User Manual

Data processing, Job & Other Tools

This manual provides an overview of data processing, job management, and additional tools such as monthly splitting and SQL utilities.

Version: 5.3.SP36

Wolters Kluwer

CCH® Tagetik

wolterskluwer.com

Table of contents

Data processing 1

Diagnostic data processing 3

Diagnostic rules 4

Validation rules page ... 4

Specific validation rules 7

Specific validation rules - Equality control groups page ... 8

Specific validation rules - Other checks page ... 10

Specific validation rules - Accounts page ... 11

Plausibility diagnostic page ... 13

Plausibility check logics page ... 14

Plausibility logics grouping page ... 19

Diagnostic data processing 20

Diagnostic checks ... 20

Control group diagnostics ... 21

Gross sign diagnostic ... 23

IC sign diagnostic ... 24

IC capacity diagnostic ... 25

FX rates diagnostic ... 26

Mandatory notes diagnostic ... 26

IC matching diagnostic ... 27

Diagnostic of authorised blocking errors ... 28

Plausibility diagnostic ... 30

Journal submission diagnostic ... 30

Draft journals diagnostic ... 31

Running diagnostic data processing 32

Entities diagnostic ... 32

Run diagnostic window - Contributor cockpit ... 32

Run diagnostic window - Consolidator cockpit - Overview tab ... 36

Run diagnostic window - Consolidator cockpit - Consolidation scenarios tab ... 40

Amounts diagnostic ... 43

Journals diagnostic ... 43
Diagnostic result window ... 44

Control Dashboard 45

Entity Dashboard ... 45
Diagnostic report ... 46
Diagnostic Error Detail ... 47

Logging diagnostic data 53

Authorisation of diagnostic errors ... 53
Diagnostic results log ... 55

Ask AI for Diagnostics 62

Error Management in Ask AI for Diagnostic ... 63

Calculation logics 64

Overview ... 64
Calculation logics list ... 64
Calculation logic, Input details ... 67
Grouping ... 67
Debug MD calculation logics ... 68
Activate calculation logics run ... 69
Run a calculation logic from workflow and from job ... 69

Run calculation logic from form 71

Processes cockpit - Contributor – Calculation logics ... 74
Processes cockpit - Consolidator – Calculation logics ... 75

Calculation logics of "Sum" type 77

How the system deletes and saves the calculated data ... 77
Definition of a "Sum" calculation logic with dimensional specifications ... 78
Definition of a set of codes ... 78
Filters ... 78
Override and reading from fixed point ... 79

Calculation logics of "Multi-Dimensional Script" type 82

Cubes and dimensions ... 82
Amounts and currencies ... 83

How does the system delete and store calculated data ... 85

Structure and definition of a TGKML script 86

Filters ... 89
Reading from fixed point ... 92
Overrides ... 94
Physical and temporary accounts ... 94
Syntactical substitution ... 95
SET expression ... 96
IF Construct ... 98
Monthly and cumulated values ... 99
Sum operator ... 101
Product operator ... 105
Parallel operators ... 109
BOUND function in a sum ... 109
Advanced use of the BOUND function ... 110

Functions and examples by topic 112

Time management ... 112
Dimension management ... 113
Write on an account node/set ... 114
LoopOnTime ... 115
Exponentiation (^) ... 116
PeriodIndex ... 117
Copy one dimension to another ... 118
Rounding and average ... 118
Currency conversion ... 120
Category pool ... 120
Swap ... 121
New Journal creation ... 122
Aggregate functions: Max/Min/Sum ... 123
DaysNumber function ... 124
Countervalue calculation ... 124

Stored procedure 126

Stored procedure definition ... 126

Database objects 134

Database Objects page ... 135
Definition of a database object ... 136
Balancing ... 137
Monthly splitting ... 139

Job 144

Loading task on Analytical Workspace ... 147
Data Transformation Package Task ... 148
ML Training and ML Execute tasks ... 148
Analytical Workspace loading task on Financial Workspace ... 149
IC Spreading Data Processing Task ... 149
ETL Run Task ... 149
Export from AW - File format Task ... 150
Diagnostic log update Task ... 150
Send Mail Task ... 151
Export to Excel ... 151

Additional Tools 153

Store data offline ... 153
Recoding ... 155
Snapshot ... 156
Unified SQL syntax ... 161
Multi File Syntax ... 165
Monthly splitting ... 167
Data tables clean-up ... 171

Data processing

CCH Tagetik provides the user with several tools to perform data processing.

There are usually two types of data processing to be performed in a CPM system:

Application data processing

Data processing operating on data without being specifically linked to a business process:

Sono :

Carry forward: Carry Forward data processing on original and consolidation scenarios allows the user to reopen initial balances and journal entries into a new fiscal year based on the closed year. (see Carry Forward)

"**Multi-Dimensional Script**" calculation logics : Complex "Multi-Dimensional Script" calculation logics (see "Multi-Dimensional Script" calculation logics)

Calculation logics: they compute specific information derived from existing data in the system (see Calculation Logics)

Break back : allows data entry via forms at the node level of aggregation structures defined on Entity, account, and custom dimensions, or only on custom dimensions (see Breack back)

Financial statement templates: a Financial statement template is an aggregation structure of accounts or customised dimensions used to create classic financial reporting templates but also KPIs and other indices useful for reporting. (see Financial statement templates)

Calculations on protected cell forms: a formula is defined on the matrix element. Its result cannot be edited by the user, but is saved to the database when the form is saved. (see Editability)

Currency conversion: the system is able to convert amounts from one currency into another (see Currency)

Algebraic operations: allow performing simple operations (e.g. add, subtract, etc.) by combining data from several scenarios. (see Algebraic Operations)

ETL : The Data Extraction, Transformation and Loading engine can perform filtering, control, data enrichment operations (see ETL Routine - Trasformazione)

DTP: a Data Transformation Package (DTP) represents a group of activities to be executed for a Data Transformation workflow task. (see Data Transformation Package)

Virtual datasets: Virtual datasets can contain fields calculated according to formulas defined by the user. (see Dataset virtuali)

Business process-specific data processing

Data processing based on built-in logics with specific functionality for business processes such as:

Consolidation: "The various consolidation processes allow the transfer of amounts from the individual Entity's financial statements through successive stages, culminating in the consolidated data (see Il bilancio consolidato)

Planning see :

- **Forecast Models**: allow a forecast scenario to be populated with a combination of actual and other forecast data (see Modelli di Forecast)
- **Cash flow** planning: enables financial planning with "what if" analyses on different simulation scenarios (see Cash Flow Planning Overview)
- **Supply Chain Planning**: allows the creation of hypothetical scenarios and simulations for SCP and CPM (seeScenario Planning)
- **Depreciation**: (see Ammortamento cespiti)
- **Spreading**: Enables calculation of the depreciation plan for new investments, organized by tax category. (see Spreading IC)

Financial close: allows you to quickly close individual financial statements and begin the consolidation process (see Financial close)

Target Matching : enables the assignment of a specific value to the data (see Impose Amount)

Allocations: data is allocated based on percentages derived from predefined rules (see Allocations)

Allocations on DTP: allows an amount originally present on a single item to be spread over several items, do as to detail it (see Analytical Allocations)

Diagnostic data processing

Purpose of the diagnostic function

The diagnostic function allows you to verify that the data entered in CCH Tagetik are correct. This check is done against validation rules defined on a general basis, but the rules can be overridden for individual processes. In this way, the diagnostic function is highly customisable.

It is used for the following purposes:

- in automatic balancing data processing, to offset unbalanced amounts that fall within certain tolerability parameters
- in the entity and journal data submission procedure, to prevent it from closing if blocking errors arise
- in the “balancing on control groups” function (available on journal rows), to manually correct unbalanced amounts.

Types of check

Checks on the accuracy of data entered in CCH Tagetik are divided into two macro categories:

- diagnostic checks
- plausibility checks, also called plausibility diagnostic

Diagnostic data processing and custom diagnostic

Depending on the page from which it is run, diagnostic data processing can be run immediately or show a run window in which further parameters can be inserted.

In the first case, the data processing run parameters are obtained from the selections that led to the specific page from which the data processing is run. In the second case, the run parameters must be specified in the run window.

Diagnostic rules

Contents

This section covers the following topics:

Validation rules page 4

Validation rules page

Admin

Application database > Data Processing tile > Diagnostic > Validation

Application database > Navigation panel > Data Processing > Diagnostic > Validation

Purpose of the page

This page allows you to set the validation rules for diagnostic checks.

Generic rules

Field	Description
Validation	
Run data processing on diagnostic	Allows you to run all data processing enabled for the process (with the exception of cash flow planning and closing) when running the diagnostics.
Also include the stored procedure	Only with **Run data processing on diagnostic** selected. Include a stored procedure that meets CCH Tagetik's minimum requirements in the data processing to be run when diagnostics is launched. The stored procedure is only run on the original scenarios/periods. The stored procedures that can be run by the diagnostic are those whose name starts with "CPM_SP_". To allow the stored procedure to read the diagnostic parameters, they must be named as follows: • PROCESS: data collection process • ENTITY_HIERARCHY: organisational hierarchy process • STEP_LIST: list of steps • ORIGINAL_SCENARIO_PERIOD_LIST: list of original scenarios/periods • ENTITY_LIST: list of entities • CONTRIBUTOR_NODE_LIST: list of contributor nodes • CONSOLIDATOR_NODE_LIST: list of consolidator nodes For more details on stored procedures, see Stored procedure in the Data processing section.
Diagnostic result: Show forms on row	Allows you to view, for every error on the control group or account in the diagnostic result window, the forms on which the control group accounts can be edited or the account itself. ***Note**: From the diagnostic result window, (non-transactional) standard forms can be edited, using Excel or web forms.*

Field	Description
Categories for which to show the result	Opens a page to select the categories for which to display the validation of the year's result, even in the absence of unbalanced amounts. ***Note**: for the **$BS_<result account code>** control group, the diagnostic will show the result, even if balanced, on the categories defined here, allowing you to view the validation of the annual result.* *(The result account code to be used is the one that is set in the process's calculation logic rules.)*
Result log	
Enable detected errors trace	Allows you to log errors returned by diagnostics so you can consult, justify and authorise them. If you have chosen to log errors, they will be inserted/added to every submission or to every diagnostic run with log update. ***Note**: only logged errors are saved and can therefore be consulted over time.*
Don't propagate the authorized blocking error information	This check looks for the presence of blocking errors authorised by users responsible for the entity on which diagnostics were run. • **Do not run**: the check is not performed. • **Blocking**: the check is performed and the presence of blocking and authorised errors is reported with blocking severity. • **Non-blocking**: the check is performed and the presence of blocking and authorised errors is marked as being of non-blocking severity. • **Only Diagnostic**: the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** (for the contributor) or **Overview** (for the consolidator) cockpit tabs; in this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed. The prerequisites for running this check are: • the traceability of errors found by diagnostics is enabled • the submission mechanism is enabled in the submission rules via authorisation of blocking errors
Journals submission validation	
Diagnose Manual journals submission	The check verifies that the manual journals belonging to the entity on which the diagnostics were run have been submitted. A prerequisite for the running of this check is that the submission of journals must be enabled on the process, i.e. task flowcharts must have been specified for journal submission in the Business Workflow mapping. • **Do not run**: the check is not performed. • **Blocking**: the check is performed and the non-submitted manual journals are reported with blocking severity. • **Non-blocking**: the check is performed and the non-submitted manual journals are marked as being of non-blocking severity. • **Only Diagnostic**: the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** (for the contributor) or **Overview** (for the consolidator) cockpit tabs; in this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed.

Field	Description
Diagnose Automatic journals submission	The check verifies that the automatic journals belonging to the entity on which the diagnostics were run have been submitted. A prerequisite for the running of this check is that the submission of journals must be enabled on the process, i.e. task flowcharts must have been specified for journal submission in the Business Workflow mapping. • **Do not run**: the check is not performed. • **Blocking**: the check is performed and the non-submitted automatic journals are reported with blocking severity. • **Non-blocking**: the check is performed and the non-submitted automatic journals are marked as being of non-blocking severity. • **Only Diagnostic**: the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** (for the contributor) or **Overview** (for the consolidator) cockpit tabs; in this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed.
IC cockpit	
IC cockpit: Balancing threshold of Compensation reconciliation accounts	The threshold, if set, is applied during the matching of relationships present in the IC Cockpit. Specifically, it will be used by the check that verifies the balancing of amounts linked to variation/detail types used on several matching rules.

Specific rules

Field	Description
Equality control groups	Opens a page to define specific rules to be applied to control groups with equality operators.
"Other" control groups	Opens a page to define specific rules to be applied to “other” control groups, or balancing control groups with non-equality equality operators.
Accounts	Opens a page to define specific control rules for accounts.

Specific validation rules

Purpose of specific rules

These rules allow you to set diagnostic data processing by table element of the control groups and accounts. These rules may be defined for the following elements:

- individual table elements, namely a control group or an account
- grouping nodes, namely for all elements contained in the node
- for all elements of the table, using the **All elements** option.

These rules allow you to validate and balance the data. However, to be used, they must first be processed to obtain all the possible combinations of table elements and categories/nodes deriving from their definition.

A preview of this data processing can be downloaded as a report in the rule definition windows.

IMPORTANT: The system processes records in the same order in which they are entered. It is therefore important to pay attention to the sequence of entries - what is added later could invalidate what was entered earlier

Validation granularity

For every table element, you can then indicate the database to be validated (category or node) and the granularity with which to validate it.

Selecting rule	Granularity	Description
Specific category	Journal	Only if the category is a journal category. The validation returns errors by journal number.
Specific category	Category	Only if the category is not a consolidation journal category. The validation returns errors by category. If the category is a pre-consolidation journal, the journal number is not reported as several journals may be being summed up together.
Node of categories	Journal	The node is deployed in the categories that it contains. The indicated granularity is applied to the entity journal categories and consolidation journal categories that are obtained. Granularity by category is applied to the amount categories that are obtained.
Node of categories	Category	The node is deployed in the categories that it contains. The indicated granularity is applied to the amount categories and entity journal categories that are obtained. The other categories are discarded.
Node of categories	Node	The node is deployed, but the algebraic sum of the categories that comprise it is validated.
All categories	Journal	All the table categories are read. The indicated granularity is applied to the entity journal and consolidation journal categories. Granularity by category is applied to amount categories.
All categories	Category	Only amount and entity journal table categories are read. The indicated granularity is applied.

Types of category nodes

The diagnostic data processing distinguishes between three types of node, or groupings, for Categories:

- **Entity groupings**. The node contains only entity categories, namely amounts and/or entity journals, of the contributor. These groupings can be validated both on original scenarios and on consolidation scenarios.
- **Consolidator entity groupings**. The node contains only entity categories, namely amounts and/or entity journals, but at least one of these is of the consolidator. These groupings can be validated both on original scenarios and on consolidation scenarios.
- **Consolidator groupings**. The node may contain just one single consolidation journal category. These groupings can only be validated on consolidation scenarios and only on the “Converted” and “Consolidable” amount.

Specific validation rules - Equality control groups page

Admin

Application database > Data Processing tile > Diagnostic > Validation > (Specific rules) Equality control groups

Application database > Navigation panel > Data Processing > Diagnostic > Validation > (Specific rules) Equality control groups

Purpose of the page

This page shows the validation rules for control groups whose equality operator is currently configured. It allows you to edit the rules and create new ones.

To override these rules for individual processes, see the Validation rules page in the Processes section.

Actions on this page

Element	Description
	Validation rules: opens a window to set and generate a preview, in Excel format, of all the elements impacted by the rule.

Attributes on the page

Element	Description
Data type	The control groups obtained by this definition are validated on the following types of data: • **All** (default) • **Gross** • **Net** • **Intercompany** This condition is intersected with what is indicated for the control groups in the relative table. See Control groups page in the Data Model section.
Control group	Indicates which elements must be validated • All elements (default) • An element of the control groups table • A node or element of the control groups groupings If **All the elements** or a node is selected, the operator of the obtained elements will be “equal to”.
Category	Indicates the categories for which the data are checked: • All elements (default) • An element of the categories table • A node or an element of the categories groupings

Element	Description
Granularity	Sets the granularity with which the data are checked and is conditional on the **Category** attribute setting: • **By node**, only if **Category** contains a node of category groupings. • **By category** (default), only if **Category** contains an amount category or an entity journal category or **All elements** or a node of category groupings. In the two latter cases, the consolidation journals are discarded from the result categories. • **By journal**, only if **Category** contains an entity journal category or a consolidation journal category or **All elements** or a node of category groupings. In the two latter cases, **By category** granularity is applied to amount-type result categories.
Severity	It allows you to define the severity to be applied to the rule, and consequently what happens if the data processing detects an error. • **Automatic** (default), the severity defined in the control groups table (**Blocking** or **Non-blocking**) is applied. • **Blocking**, the entity data are not submitted. • **Non-blocking**: the entity data are submitted but the outcome of the check is registered. • **Only Diagnostic**: the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** (for the contributor) or **Overview** (for the consolidator) cockpit tabs; in this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed. • **None**, the data are not verified. Useful option for excluding elements from a node to be deployed.
Thresholds	
Currencies for thresholds	Currency in which the threshold amounts set in this section are considered to be expressed; for the comparison, these must be converted into the currency of the entities/journals to which it is applied. The FX rate used for the conversion is the final FX rate. If the currency is not set, the currency of the relative entity or journal will be used.
Submission threshold	If set, below the threshold value, the entity is submitted even if there are errors with **Blocking** severity. In the submission step, these errors are classified as **Below threshold blocking**.
Original scenarios balancing threshold	If set, during the balancing data processing for original scenarios, the rows with unbalanced amounts below this value are balanced. Control groups with the following granularities are not balanced: • “by node” containing only journal categories • “by category” on journal categories.
Consolidation scenarios balancing threshold	If set, during the balancing data processing of consolidation scenarios, the rows with unbalanced amounts below this value are balanced. Control groups with the following granularities are not balanced: • “by node” containing only journal categories • “by category” on journal categories.



***IMPORTANT**: if thresholds are not set, the threshold on the row preceding the definition for the same data collection will be considered. If no other rows exist, the threshold will be set to zero.*

Specific validation rules - Other checks page

Admin

Application database > Data Processing tile > Diagnostic > Validation > (Specific rules) “Other” control groups

Application database > Navigation panel > Data Processing > Diagnostic > Validation > (Specific rules) “Other” control groups

Purpose of the page

This page allows you to manage validation rules for control groups whose operator is not equality. It allows you to edit the rules and create new ones.

To override these rules for individual processes, see the Validation rules page in the Processes section.

Actions on this page

Element	Description
	**Validation rules**: opens a window to set and generate a preview, in Excel format, of all the elements impacted by the rule.

Attributes on the page

Element	Description
Data type	The control groups obtained by this definition are validated on the following types of data: • **All** (default) • **Gross** • **Net** • **Intercompany** This condition is intersected with what is indicated for the control groups in the relative table. See Control groups page in the Data Model section.
Control group	Indicates which elements must be validated: • All elements (default) • An element of the control groups table • A node or element of the control groups groupings If **All elements** or a node is selected, the operator of the obtained elements will be different from “equal to”.
Category	Indicates the categories for which the data are checked: • All elements (default) • An element of the categories table • A node or an element of the categories groupings

Element	Description
Granularity	Sets the granularity with which the data are checked and is conditional on the **Category** attribute setting: • **By node**, only if **Category** contains a node of category groupings. • **By category** (default), only if **Category** contains an amount category or an entity journal category or **All elements** or a node of category groupings. In the two latter cases, the consolidation journals are discarded from the result categories. • **By journal**, only if **Category** contains an entity journal category or a consolidation journal category or **All elements** or a node of category groupings. In the two latter cases, **By category** granularity is applied to amount-type result categories.
Severity	It allows you to define the severity to be applied in the rule, and consequently what happens if the data processing detects an error. • **Automatic** (default), the severity defined in the control groups table (**Blocking** or **Non-blocking**) is applied. • **Blocking**, the entity data are not submitted. • **Non-blocking**, the entity data are submitted but errors are reported. • **Only Diagnostic**: the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** (for the contributor) or **Overview** (for the consolidator) cockpit tabs; in this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed. • **None**, the data are not verified. Useful option for excluding elements from a node to be deployed.
Thresholds	
Currencies for thresholds	Currency in which the threshold amounts set in this section are considered to be expressed; for the comparison, these must be converted into the currency of the entities/journals to which it is applied. The FX rate used for the conversion is the final FX rate. If the currency is not set, the currency of the relative entity or journal will be used.
Submission threshold	If set, below the threshold value, the entity is submitted even if there are errors with **Blocking** severity. In the submission step, these errors are classified as **Below threshold blocking**.

The thresholds are only applied to control groups whose operator is greater than, greater than/equal to, less than, less than/equal to, not equal. They are to applied to coherence operators.

***IMPORTANT**: if thresholds are not set, the threshold on the row preceding the definition for the same data collection will be considered. If no other rows exist, the threshold will be set to zero.*

Specific validation rules - Accounts page

Admin

Application database > Data Processing tile > Diagnostic > Validation > (Specific rules) Accounts

Application database > Navigation panel > Data Processing > Diagnostic > Validation > (Specific rules) Accounts

Purpose of the page

This page allows you to manage the specific validation rules of accounts for the Sign, IC sign, IC capacity, Notes and FX rates checks.

To override these rules for individual processes, see the Validation rules page in the Processes section.

Actions on this page

Element	Description
	**Validation rules**: opens a window to set and generate a preview, in Excel format, of all the elements impacted by the rule.

Attributes on the page

Element	Description
Type of check	Criteria with which the accounts to be checked are selected. • **Sign**: check only on accounts which, in the table, have **Standard sign check** not equal to **Any**. • **IC sign**: check only on accounts which, in the table, have **IC sign check** not equal to **Do not check**. • **IC capacity**: check only on accounts which, in the table, have **IC Capacity check** not equal to **Do not check**. • **FX rates**: check only on accounts which, in the table, have **FX rates check** not equal to **Do not check**. • **Notes**: check only on accounts which, in the table, have **Notes check** not equal to **Do not check**.
Account	Indicates the accounts for which the data are checked: • **All elements** (default) • An element of the accounts table • A node or an element of a hierarchy of accounts
Category	Indicates the categories for which the data are checked: • **All elements** (default) • An element of the categories table • A node or an element of the categories groupings
Granularity	Sets the granularity with which the data are checked and is conditional on the **Category** attribute setting: • **By node**, only if **Category** contains a node of category groupings. • **By category** (default), only if **Category** contains an amount category or an entity journal category or **All elements** or a node of category groupings. In the two latter cases, the consolidation journals are discarded from the result categories. • **By journal**, only if **Category** contains an entity journal category or a consolidation journal category or **All elements** or a node of category groupings. In the two latter cases, **By category** granularity is applied to amount-type result categories.
Severity	It allows you to define the severity to be applied in the rule, and consequently what happens if the data processing detects an error. • **Non-blocking** (default), the entity data are submitted but errors are reported. • **Blocking**, the entity data are not submitted. • **Only Diagnostic**: the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** (for the contributor) or **Overview** (for the consolidator) cockpit tabs; in this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed. • **None**, the data are not verified. Useful option for excluding elements from a node to be deployed.
IC capacity thresholds	
Currencies for thresholds	Currency in which the threshold amounts set in this section are considered to be expressed; for the comparison, these must be converted into the currency of the entities/journals to which it is applied. The FX rate used for the conversion is the final FX rate. If the currency is not set, the currency of the relative entity or journal will be used.

Element	Description
Submission threshold	Only applied to the IC capacity check. For the comparison, the FX rate must be converted into the currency of the entities/journals to which it is applied. The FX rate used for the conversion is the final FX rate. If set, below the threshold value, the entity is submitted even if there are errors with **Blocking** severity. In the submission step, these errors are classified as **Below threshold blocking**.
Original scenarios balancing threshold	Only applied to the IC capacity check and to accounts which, in the table, have **IC Capacity check** equal to **Check =**. If set, during the balancing data processing for original scenarios, the rows with unbalanced amounts below this value are balanced. . Accounts with the following granularities are not balanced: • "by node" containing only journal categories • "by category" on journal categories.
Dimensions check	
Entity	• **Element**: shows the outcome by entity (default). • **Contributor node**: shows the outcome on the significant nodes in the data collection process by algebraically summing the amounts linked to the entities belonging to the same nodes.
Check by Custom dimension	Can be set for all custom dimensions enabled on the environment. • **Total** (default): shows the outcome, grouping on the custom dimension. • **Lowest level elements**: shows the outcome for every element of the custom dimension. • **Hierarchy nodes**: shows the outcome by grouping on the first-level nodes of the hierarchy indicated in the **Custom dimension hierarchy** field.
Custom dimension hierarchy	The custom dimensions on which this setup can be carried out are those which are enabled in the environment's configuration. If **Check by custom dimension** is equal to **Hierarchy nodes**, an element of the custom dimensions hierarchy table must be indicated.

***IMPORTANT**: if thresholds are not set, the threshold on the row preceding the definition for the same data collection will be considered. If no other rows exist, the threshold will be set to zero.*

Plausibility diagnostic page

Application database > Processes tile > select process > > Process setup > (Data Processing section) Validations > Plausibility diagnostic >

Application database > Navbar > Processes & Workflows > Process > select process > > Process definition > (Data Processing section) Validations > Plausibility diagnostic >

Purpose of the page

This page allows you to set the plausibility diagnostics requirements, or to associate a group of plausibility logics to be run during diagnostics with every entity, contributor node and consolidator node. Also allows you to override the security level to be applied to them.

Page tabs

Field	Description
Elements	Defines the plausibility logics to be applied to entities.
Contributor nodes	Defines the plausibility logics to be applied to contributor nodes.
Consolidator nodes	Defines the plausibility logics to be applied to consolidator nodes.

Attributes tab

Field	Description
Action	• **Include** (default) • **Exclude**
Original scenarios/periods	Original scenarios/periods of the process to which to apply the overrides: • **Selected** (default) • **All**
Entity	Only for the **Elements** tab. Allows you to select an entity belonging to the process or a node of entities within the available non-time dependent entity hierarchies.
Contributor node	Only for the **Contributor nodes** tab. Allows you to select an entity node from the organisational hierarchy linked to the process.
Consolidator node	Only for the **Consolidator nodes** tab. Allows you to select an entity node from the organisational hierarchy linked to the process.
Node type	Only for the **Contributor nodes** and **Consolidator nodes** tab. Indicates the way in which to deploy the selected node.
Plausibility rules node	Valorizzare solo per azione "Includi". Allows you to select the node of logics for the plausibility checks to be applied.
Severity type	Valorizzare solo per azione "Includi". • **Do not run**: combinations with this option selected are not validated. It is used, for example, to exclude entities/nodes from a node to be deployed. • **Automatic**: severity is recovered from the definition of the plausibility logic, where established through specific comparison thresholds.
Definition result	Only appears after a definition has been saved. Shows the result of deployment of the definition, with a list of available combinations.

Plausibility check logics page

Admin

Data Processing tile > Diagnostic > Plausibility check logics > List

Navigation panel > Data Processing > Diagnostic > Plausibility check logics > List

Purpose of the page

This page allows you to define the plausibility logics. In particular, this page allows you to set:

- the reference scenario and period with which to compare the data
- the filters to be applied to the data to be checked
- the aggregation criteria for the data to be checked
- the checks to be performed, defined through an SQL expression
- the comparison thresholds to determine the severity of errors

When the data processing is run, the following elements are defined:

- the scenario and the period to be compared
- the entity for which to filter the data

Page tabs

Element	Description
Attributes	It allows you to identify the logic and indicate the elements/categories for which to run the check, as well as the comparison scenario and period.
Data to check	It allows you to specify the data type to be checked and to filter the data on which to apply the logic based on certain table elements.

Element	Description
Percentage calculation	It allows you to define whether percentage calculations must be carried out on the amounts to be compared
Comparison	It allows you to define the formula to use to perform the comparisons. Uses SQL scripts and thresholds.
Relationships	It allows you to relate every plausibility logic to one or several nodes of the plausibility logic grouping. These nodes can then be associated with each entity, contributor node and consolidator node within the process in which you want to run the plausibility diagnostic.

Fields in the Attributes tab

Element	Description
Code	Unique identifier of the plausibility logic
Description	Description of the plausibility logic
Notes	Any information in addition to the description
Check for	
Check by Entity	Can only be selected if **Check by contributor node** is not selected. If selected, the check is run on individual entities. The logic operates on the contributor entity categories (contributor amounts and contributor entity journals) and on the groupings of categories containing only contributor entity categories.
Check by contributor node	Can only be selected if **Check by Entity** is not selected. If selected, the check is run on contributor nodes, namely on nodes which are defined as **To approve** or **To submit** in the entity hierarchy within the data collection process. The data are obtained by algebraically summing the amounts linked to the entities belonging to the same nodes. The logic operates on the contributor entity categories (contributor amounts and contributor entity journals) and on the groupings of categories containing only contributor entity categories.
Include categories grouping with categories of the consolidator	Can only be selected if **Check by contributor node** is selected. If selected, the check is also run on the groupings of categories containing at least one entity category of the consolidator.
Check by consolidator node	If selected, the check is run on consolidator nodes, namely on nodes which are defined as **To consolidate** in the entity hierarchy within the consolidation process. The logic operates on the consolidator categories (consolidator amounts, consolidator entity journals, consolidation journals) and on groupings of categories containing at least one entity category of the consolidator.
Reference data	
Scenario	Indicates how to determine the scenario to compare the data. • **Same as data to process** (default): the scenario from which to read the data for the comparison is the same one used to run the data processing. • **Fixed**: the scenario from which to read the data for the comparison is the one reported in the **Fixed scenario**. • **Parametric**: the scenario from which to read the data for the comparison is the one reported in the **Parametric scenario**.
Fixed scenario	Only if **Scenario** = **Fixed**. Specifies the scenario from which to read the data.
Parametric scenario	Only if **Scenario** = **Parametric**. Scenario from which to read the data according to one of the following criteria. • **PREV**: scenario preceding the one being processed • **NEXT**: scenario following the one being processed • **REF1**... **REF5**: reference scenarios 1...5 related to the processed scenario ***Note**: if there are multiple values, they must be separated by a comma (NEXT,REF1, etc.).*

Element	Description
Period	Indicates how to determine the period for the data comparison. • **Same as data to process** (default): the period from which to read the data for the comparison is the same one used to run the data processing. • **Fixed**: the period from which to read the data for the comparison is the one reported in **Fixed period**. • **Parametric**: the period from which to read the data for the comparison is the one reported in **Parametric period**.
Fixed period	Only if **Period** = **Fixed**. Specifies the period from which to read the data.
Parametric period	Only if **Period** = **Parametric**. Period from which to read the data, between -99 and 99. This is calculated as the “difference” compared with the period of the data to be processed.
Period length	Specifies how to calculate the data for the comparison. • **Year-to-date** (default): the amount depends on the preceding months, starting from the start of the year. For example, the March amount is the sum of the January, February and March amounts. • **Periodic**: the amount only depends on the processed period. In this case, it is also necessary to specify how to calculate the previous period in the scenario.

Fields in the Data to check tab

Element	Description
Data to read	Specifies the amount that will be read from the data tables. • **Original amount**: amount in the original currency • **Currency amount**: amount in the currency of the transaction
Filter	
Account	Accounts on which the logic operates. This can be an individual account or the node of a hierarchy of accounts. If not specified, all the available accounts are considered.
Custom dimension	Dimensions on which the logic operates. For each managed dimension, this can be an individual element or the node of a hierarchy defined on the dimension. If not specified, all the elements available on the dimension are considered.
Category	Categories on which the logic operates. This can be an individual category or a category grouping node. If not specified, all the available categories are considered.
CTP entity	Counterparty entity on which the logic operates. This can be an individual entity or the node of a hierarchy defined on the entities. If not specified, all the available counterparty entities are considered. If set, it is only applied to the reading of intercompany data (amounts and entity journals).
Segment CTP Entity	Segment counterparty entities on which the logic operates. This can be an individual entity or the node of a hierarchy defined on the entities. If not specified, all the available segment counterparty entities are considered. If set, it is only applied to the reading of consolidation journals.
Counterparty custom dimension 2	Counterparty custom dimension 2 on which the logic operates. This can be an individual element or the node of a hierarchy defined on the custom dimension 2. If not specified, all the elements available on the custom dimension 2 are considered. If set, it is only applied on processes whose relationship type is “by entity - custom dimension 2” and only to the reading of intercompany data (amounts and entity journals) and to consolidation journals (where the filter acts on the segment counterparty custom dimension 2).
Currency	Currency on which the logic operates. It must be an single currency. If not specified, all the available currencies are considered.

Element	Description
Check by/Hierarchy	
Check by Account	Specifies the logic's behaviour in relation to the account. • **By element**: the logic operates by individual account. • **By total**: the logic operates on the algebraic sum of the adjusted data. • **By node**: the logic operates on the algebraic sum of the data adjusted on the accounts belonging to the first-level nodes associated with the specified hierarchy.
Check by Custom dimension	For every managed custom dimension, this specifies the logic's behaviour in relation to the custom dimension. • **By element**: the logic operates by individual element of the dimension. • **By total**: the logic operates on the algebraic sum of the data adjusted on the elements of the dimension. • **By node**: the logic operates on the algebraic sum of the data adjusted on the elements of the dimension which belong to the first-level nodes associated with the specified hierarchy.
Check by Category	Specifies the logic's behaviour in relation to the category. • **By element**: the logic operates by individual category. • **By analytic element**: the logic operates by individual entity journal or consolidation journal. If the category is amount, the logic operates by individual category. • **By total**: the logic operates on the algebraic sum of the data adjusted on the categories. • **By node**: the logic operates on the algebraic sum of the data adjusted on the categories belonging to the first-level nodes associated with the specified hierarchy.
Check by CTP entity	Specifies the logic's behaviour in relation to the counterparty entity. • **Gross**: the logic only analyses gross amounts. • **By element**: the logic operates by individual counterparty entity. • **By total**: the logic operates on the algebraic sum of the data adjusted on the counterparty entities. • **By node**: the logic operates on the algebraic sum of the data adjusted on the counterparty entities belonging to the first-level nodes associated with the specified hierarchy.
Check by Segment CTP entity	Specifies the logic's behaviour in relation to the segment counterparty entity. • **By element**: the logic operates by individual segment counterparty entity. • **By total**: the logic operates on the algebraic sum of the data adjusted on the segment counterparty entities. • **By node**: the logic operates on the algebraic sum of the data adjusted on the segment counterparty entities belonging to the first-level nodes associated with the specified hierarchy.

Fields in the Percentage calculation tab

Element	Description
Check "ratio %"	Specifies whether percentage calculations must be carried out on the amounts to be compared • **No**: no calculations are performed. • **Yes**: the amounts to be compared are added up according to what is set in the **Sum up** section. The percentage of every element or node is then calculated. Enables the following fields.
Sum up/Hierarchy	
Sum up Account	Specifies how the account contributes to determining the totals with which to divide the individual amounts to be compared. • **No**: the logic does not consider the account when determining the totals. • **By element**: the logic determines a total for every account. • **By node**: the logic determines a total for every first-level node associated with the specified hierarchy to which the accounts belong.
Sum up Custom dimension	Fir every custom dimension managed in the environment, it specifies how the elements of the dimension contribute to determining the totals with which to divide the individual amounts to be compared. • **No**: the logic does not consider the elements of the dimension when determining the totals. • **By element**: the logic determines a total for every element of the dimension. • **By node**: the logic determines a total for every first-level node associated with the specified hierarchy to which the dimension elements belong.

Fields in the Comparison tab

Element	Description
Preliminary expression	Optional check, the same rules as the **Plausibility check** are applied.
Plausibility check	Mandatory check Expression in SQL syntax to be applied to the amounts to be compared. The amounts related to the first data collection must be referred to using string “A1”, while those related to the second data collection must be referred to using string “A2”. *Case when A1<>0 then ABS((A2-A1)/A1)*100 end*
Check currency	Currency used to compare the result of the preliminary check expression and the plausibility check expression. If defined, it converts the amounts accordingly. The amount of the thresholds defined below is understood to be expressed in that currency, otherwise it is expressed in the currency of the entities/journals to which the check is applied. The FX rate used for conversion is the FX rate indicated for the account of the data collection in question.
Threshold for preliminary check	Threshold used to compare the result of the preliminary check expression, if specified. If the result of the expression is lower than the indicated threshold, the subsequent checks will not be carried out and the diagnostic will not record errors for the data collection in question.
Threshold for plausibility check: trivial error	Threshold used to compare the result of the plausibility check expression and establish whether the error is trivial. If the outcome of the expression is lower than the indicated threshold, the diagnostic will assign “Warning” severity to the data collection in question.
Threshold for plausibility check: non-blocking error	Threshold used to compare the outcome of the plausibility check expression and establish whether the error is non-blocking. If the result of the expression is above the trivial error threshold and below the indicated threshold, the diagnostic will assign “Non-blocking” severity to the data collection in question.

Element	Description
Threshold for plausibility check: blocking error	Threshold used to compare the outcome of the plausibility check expression and establish whether the error is blocking. If the result of the expression is above the non-blocking error threshold and below the indicated threshold, the diagnostic will assign “Blocking” severity to the data collection in question.
Threshold exceptions for currency	Opens a page to define specific thresholds for the desired currencies. Useful if **Check currency** is not defined, and the amounts are therefore expressed in the currency of the data collection entities/journals subject to the check.

***IMPORTANT**: in order for the plausibility check to be performed correctly, at least one threshold must be specified. Moreover, the values of the thresholds set must be incremental, that is, the blocking error threshold must be greater than the non-blocking error threshold and the non-blocking error threshold must be greater than the trivial error threshold.*

Plausibility logics grouping page

Admin

Data Processing tile > Diagnostic > Plausibility check logics > Grouping

Navigation panel > Data Processing > Diagnostic > Plausibility check logics > Grouping

Purpose of the page

This page allows you to customise the “grouping” aggregation structure of the predefined plausibility checks ($ - Plausibility checks). This customisation defines the nodes and establishes which ones new plausibility logics must be related to by default.

The nodes of logics are then associated to each entity, contributor node and consolidator node within the process in which the plausibility diagnostic is run. See Plausibility diagnostic page in the Processes section.

Diagnostic data processing

Contents

This section covers the following topics:

Diagnostic checks 20
Control group diagnostics 21
Gross sign diagnostic 23
IC sign diagnostic 24
IC capacity diagnostic 25
FX rates diagnostic 26
Mandatory notes diagnostic 26
IC matching diagnostic 27
Diagnostic of authorised blocking errors 28
Plausibility diagnostic 30
Journal submission diagnostic 30
Draft journals diagnostic 31

Diagnostic checks

Introduction

Diagnostic data processing runs checks on the following elements:

- Gross, IC and net control groups
- Gross sign
- IC sign
- IC Capacity
- FX rate
- Mandatory notes
- IC matching
- Authorized blocking errors
- Plausibility checks
- Journals submission
- Draft journals

Depending on the elements to be verified, the checks are run from different points in the application.

Element on which the diagnostic check is run	Data processing starting point
Entities, contributor nodes and consolidator nodes on original scenarios	Contributor cockpit (Contributor Cockpit page in the Processes section) > Actions > Diagnostic Consolidator cockpit (Consolidator cockpit page in the Processes section) > Actions > Diagnostic
Entities, contributor nodes and consolidator nodes on consolidation scenarios	Consolidator cockpit - Consolidation scenarios (Consolidation scenarios tab in the Processes section) > Actions > Diagnostic
Journals	Entity and consolidation journals data entry page (Actions from journal headers in the Data collection section) in the Contributor or Consolidator cockpit
Amounts	Amounts and IC amounts data entry page (Gross Amounts page and IC Amounts page in the Data collection section) in the Contributor or Consolidator cockpit

Control group diagnostics

Control description

The control groups diagnostic checks the balance of the rule that is obtained from the definition of the control group. Then it compares the total amounts associated with the 'parent' accounts to the total amounts associated with the 'child' accounts, based on the operator selected for the control group.

This type of diagnostic analyzes the gross, IC and net amounts, depending on the type of control group being analyzed.

Control operators

The check uses the following operators:

Operator	Description
= (equal)	The amount of the 'child' accounts must be equal to the amount of the 'parent' accounts.
< (less than)	The amount of the 'child' accounts must be less than the amount of the 'parent' accounts.
> (greater than)	The amount of the 'child' accounts must be greater than the amount of the 'parent' accounts.
> = (greater than or equal)	The amount of the 'child' accounts must be greater than or equal to the amount of the 'parent' accounts.
<= (less than or equal)	The amount of the 'child' accounts must be less than or equal to the amount of the 'parent' accounts.
<> (not equal)	The amount of the 'child' accounts must be different from the amount of the 'parent' accounts.
Coherence	If the amount of the 'parent' accounts is different from zero, the amount of the 'child' accounts must also be different from zero.
Bi-directional coherence	If the amount of the 'parent' accounts is different from zero, the amount of the 'child' accounts must also be different from zero. If the amount of the 'parent' accounts is zero, the amount of the 'child' accounts must also be zero.
Coherence by element	If a "parent" account different from zero exists, then a "child" account different from zero must exist as well.
Bi-directional coherence for element	If a "parent" account different from zero exists, then a "child" account different from zero must exist as well. If all 'parent' accounts are equal to zero, all 'child' accounts must also be equal to zero.

For 'existence' control groups, i.e. those with operator < (less than), > (greater than) and <> (different) for which there is no data by scenario/period, entity and control group, the diagnostic generates reports by category/grouping as defined in the validation rules.

Consequently, in the absence of rows or with zero rows, the data are analysed taking into account the control group settings.

If, for example, the control group includes checking custom dimensions by elements, the generated reports are not divided only by category/grouping, but also by custom dimension.

Control groups to be diagnosed

The diagnostic is only performed on the control groups associated with the selected process.
For each entity, i.e. Entity, contributor node, consolidator node, only the control groups present in the process data model associated with that entity are subjected to diagnostics.

As for the control group used to verify the income statement result ($BS_), it is possible to display its result even when balancing is achieved. For this purpose, it is necessary to specify the categories for which the result should be displayed even when balancing occurs (Categories to be diagnosed).

***Note**: The result account code to be used is the one defined in the calculation logics of the process.*

Diagnostic and control groups list

The following attributes of the control group master data are relevant for the execution of diagnostic data processing:

Attribute	Description
Data types to check	Specifies whether the check is to be performed on gross, net or IC amounts.
Check original data on	Specifies whether the check on gross amounts, on amount categories, is to be performed by local currency or by transaction currency.
Check original data by	Specifies whether the control group is to be diagnosed on the Entity, on the contributor node, or on both.
Check consolidation journals by Entity	If enabled, the check on consolidation journals is run by journal and Entity, otherwise only by journal.

For more details, see Control Groups Control in the Data Model section.

Gross amounts check

The check on gross amounts is performed on control groups with **Data Type to Check** = *Gross* and is performed on both the original and consolidation scenarios. It reports any unbalanced amounts on the gross phase present in the process data model of the entity being diagnosed.

It is performed on all category types (amount, Entity journals, Consolidation journals) and on all category groupings (entity grouping, consolidator entity grouping, consolidator grouping).

It can be carried out either on the individual elements of the custom dimensions or on the top-level nodes of the custom dimensions or by adding up the custom dimensions as specified in the control group list. Moreover, it takes into account any restrictions on custom dimensions defined on the control group itself.

IC amounts check

The check on IC amounts is performed on control groups with **Data type to check** = *Intercompany* and is only performed on the original scenarios. It reports any unbalanced amounts on the IC phase present in the process data model of the entity being diagnosed.

It is performed by Ctp Entity, by checking only the counterparties defined in the **counterparty rule for IC checks** in the control group list, if any. This setting is ignored for contributor nodes, where the check is performed by Ctp node.

It is performed only on the Entity journals and amount categories and only on the category groupings of Entity and consolidator Entity.

It can be carried out either on the individual elements of the custom dimensions or on the top-level nodes of the custom dimensions or by adding up the custom dimensions as specified in the control group list. Moreover, it takes into account any restrictions on custom dimensions defined on the control group itself.

Net amounts check

The check on net amounts is performed on control groups with **Data type to check** = *Net* and is only performed on the original scenarios. It reports any unbalanced amounts in the maximum phase between gross and IC (capacity phase) within the process data model of the entity being diagnosed

It is performed only on the Entity journals and amount categories and only on the category groupings of Entity and consolidator Entity.

It can be carried out either on the individual elements of the custom dimensions or on the top-level nodes of the custom dimensions or by adding up the custom dimensions as specified in the control group list. Moreover, it takes into account any restrictions on custom dimensions defined on the control group itself.

Categories to be diagnosed

The categories and category groupings to be diagnosed are defined in the validation rules of the processin the Processes section). For each category and each grouping, the following items are specified:

- severity (**Automatic**, **Blocking**, **Non-blocking**, **Diagnostic only**, **Do not run**),

***Note**: If **Automatic** is selected, the information is retrieved from the control group list, where it can only be **Blocking** or **Non-Blocking**.*

- any tolerance thresholds for diagnostic run by automatic balancing data processing or by the data submission procedure

Entity groupings are those only containing entity categories, i.e. amounts and/or entity journals, of the contributor. These groupings can be validated both on original scenarios and on consolidation scenarios.

Consolidator entity groupings are those which only contain entity categories, i.e. amounts and/or entity journals, where at least one is of the consolidator. These groupings can be validated both on original scenarios and on consolidation scenarios.

The consolidator groupings are those which also contain a single consolidation journal category. These groupings can only be validated on consolidation scenarios and only on the “Converted” and “Consolidable” amount.

Gross sign diagnostic

Description of the check

This check verifies that the sign of the gross amounts entered for the account and custom dimensions is consistent with what is indicated as the **Standard sign check** in the accounts table. It only checks the accounts associated with the selected process and, for entities (company, contributor node, consolidator node), it only checks those present in the data model process associated with the same entity.

The check is run on both original scenarios and consolidation scenarios, and reports any unbalanced amounts in the gross step present in the data model process of the entity subject to the diagnostic. The check can be run on individual elements of the destinations, on first-level nodes of the destinations, or by summing the destinations according to what is indicated in the process’s validation rules.

The check is run on all category types (amount, entity journals, consolidation journals) and on all category groupings (entity grouping, consolidator entity grouping, consolidator grouping).

Accounts table and diagnostic

The following attributes of the accounts are significant for running diagnostic data processing:

Attribute	Description
Standard sign check	Specifies the sign that may be used to insert amounts. The gross sign diagnostic is only run on accounts whose setting is not **Any** (Account table in the Data Model section).
Accounting class signs	Signs used by the accounting classes (e.g. payables, revenues, costs...) in the reporting forms. The diagnostic takes account of what is indicated in **Accounting class signs: Debit**. You can specify that the Debit accounting class is to be indicated as negative or as positive. (Account dimension settings in the Data Model section).

The check is run according to the following logic:

If...	AND...	Then...
Standard sign check = **Debit**	the debit sign is positive.	the diagnostic verifies that the amount is greater than or equal to zero.
Standard sign check = **Debit**	the debit sign is negative.	the diagnostic verifies that the amount is less than or equal to zero.
Standard sign check = **Credit**	the debit sign is positive.	the diagnostic verifies that the amount is less than or equal to zero.
Standard sign check = **Credit**	the debit sign is negative.	the diagnostic verifies that the amount is greater than or equal to zero.

Categories to subject to the diagnostic

The categories and category groupings to subject to the diagnostic are defined in the process’s validation rules (Validation rules page in the Processes section).
Severity is indicated for each category and each grouping (**Blocking**, **Non-blocking**, **Only Diagnostic**, **None**)

Entity groupings are those only containing entity categories, i.e. amounts and/or entity journals, of the contributor. These groupings can be validated both on original scenarios and on consolidation scenarios.

Consolidator entity groupings are those which only contain entity categories, i.e. amounts and/or entity journals, where at least one is of the consolidator. These groupings can be validated both on original scenarios and on consolidation scenarios.

The consolidator groupings are those which also contain a single consolidation journal category. These groupings can only be validated on consolidation scenarios and only on the “Converted” and “Consolidable” amount.

IC sign diagnostic

Description of the check

This check verifies congruence between the sign of the intercompany amounts entered for the account, custom dimensions and counterpart and what is indicated as the **IC sign check** in the accounts table.
It only checks the accounts associated with the process that has been selected and, for entities (company, contributor node, consolidator node), it only checks those present in the data model process associated with the same entity.

The check is only run on original scenarios. It can be run on individual elements of the destinations, on first-level nodes of the destinations, or by summing the destinations according to what is indicated in the process’s validation rules.

The check is only run on the amount and entity journal categories, and only on entity category and consolidator entity groupings.

Accounts table and diagnostic

The following attributes of the accounts table are significant for running diagnostic data processing:

Attribute	Description
IC sign check	This specifies the method used to check the signs of IC entries. The IC sign diagnostic is only run on accounts whose setting is not **Do not check** (Account table in the Data Model section).
Accounting class signs	Signs used by the accounting classes (e.g. payables, revenues, costs...) in the reporting forms. The diagnostic takes account of what is indicated in **Accounting class signs: Debit** (Account dimension settings in the Data Model section) where you can define the sign of the Debit accounting class (and the corresponding credit class).

The check is run according to the following logic:

If...	AND...	Then...
IC sign check = **Debit**	the debit sign is positive.	the diagnostic verifies that the amount is greater than or equal to zero.
IC sign check = **Debit**	the debit sign is negative.	the diagnostic verifies that the amount is less than or equal to zero.
IC sign check = **Credit**	the debit sign is positive.	the diagnostic verifies that the amount is less than or equal to zero.
IC sign check = **Credit**	the debit sign is negative.	the diagnostic verifies that the amount is greater than or equal to zero.
IC sign check = **As Gross amount**	the amounts are not equal to zero.	the diagnostic verifies that they are both greater than zero or both less than zero. That is, it verifies congruence between the sign of the amounts entered for the account, custom dimensions and counterparty on the IC data and the sign of the amounts entered for the account and custom dimensions on the gross data.

If **IC sign check** is Debit or Credit, the diagnostic reports any unbalanced amounts on the IC step present in the data model process of the entity subject to the diagnostic.

If **IC sign check** is **As Gross amount**, the diagnostic reports any unbalanced amounts in the highest out of the gross step and the IC step (capacity step) present in the data model process of the entity subject to the diagnostic.

Categories to subject to the diagnostic

The categories and category groupings to subject to the diagnostic are defined in the process's validation rules (Validation rules page in the Processes section).
Severity is indicated for each category and each grouping (**Blocking**, **Non-blocking**, **Only Diagnostic**, **None**)

Entity groupings are those only containing entity categories, i.e. amounts and/or entity journals, of the contributor.

Consolidator entity groupings are those which only contain entity categories, i.e. amounts and/or entity journals, where at least one is of the consolidator.

IC capacity diagnostic

Description of the check

This check verifies that the total of the intercompany amounts entered by account and destination is an "of which" of the total of the gross amounts entered, also by account and destination. In contrast to the IC sign diagnostic, it does not work by counterparty but by account. It only checks the accounts associated with the process that has been selected and, for entities (company, contributor node, consolidator node), it only checks those present in the data model process associated with the same entity.

The check is only run on original scenarios and reports any unbalanced amounts in the highest out of the gross step and the IC step (capacity step) present in the data model process of the entity subject to the diagnostic.

It can be run on individual elements of the destinations, on first-level nodes of the destinations, or by summing the destinations according to what is indicated in the process's validation rules.

The check is only run on the amount and entity journal categories, and only on entity category and consolidator entity groupings.

Accounts table and diagnostic

The following attributes of the accounts table are significant for running diagnostic data processing:

Attribute	Description
IC capacity check	The IC capacity diagnostic is only run on accounts whose setting is not **Do not check** (Account table in the Data Model section).

The check is run according to the following logic:

If...	Then...
IC Capacity check = **Check <=**	the IC amounts must have the same sign as the gross amounts and be of a lower or equal absolute value.
IC Sign Check = **Check =**	IC amounts must be equal to the gross amount.

In entity journals, the "Check <=" is always satisfied, given that it compares the sum of all the rows with the sum of only rows with a defined counterparty. On the other hand, the "Check =" is only satisfied if the counterparty entity is defined on all the rows.

Categories to subject to the diagnostic

The categories and category groupings to subject to the diagnostic are defined in the process's validation rules (Validation rules page in the Processes section).
Severity is indicated for each category and each grouping (**Blocking**, **Non-blocking**, **Only Diagnostic**, **None**)

Entity groupings are those only containing entity categories, i.e. amounts and/or entity journals, of the contributor.

Consolidator entity groupings are those which only contain entity categories, i.e. amounts and/or entity journals, where at least one is of the consolidator.

FX rates diagnostic

Description of the check

For every gross and/or IC amount, this check verifies coherence between the FS currency amount and the document currency amount. It verifies coherence by checking that the two amounts have the same sign and, in particular, that the FX difference between the document currency amount and the FS currency amount does not exceed a percentage specified in the currencies list (**Percentage difference**). For more details, see Currencies list in the Data Model section.

The final FX rate is always used for this check, regardless of the account.

It only checks the accounts associated with the selected process and, for entities (company, contributor node, consolidator node), it only checks the accounts present in the data model process associated with the same entity.

The check is only run on original scenarios, and on gross data; it reports any unbalanced amounts in the gross step present in the data model process of the entity subject to diagnostic. On IC data, it reports any unbalanced amounts on the IC step present in the data model process of the entity subject to the diagnostic.

It can be run on individual elements of the destinations, on first-level nodes of the destinations, or by summing the destinations according to what is indicated in the process's validation rules.

The check is only run on the amount and entity journal categories, and only on entity category and consolidator entity groupings.

Accounts table and diagnostic

The following attributes of the accounts table are significant for running diagnostic data processing:

Attribute	Description
FX rates check	The FX rates diagnostic is only run on accounts whose setting is not **Do not check** (Account table in the Data Model section).

The check is run according to the following logic:

If...	Then...
FX rates check = **Check only IC amounts**	the FX rates check must be run for intercompany data only.
FX rates check = **Check Gross and IC amounts**	the FX rates check must be run for both intercompany data and gross data.

Categories to subject to the diagnostic

The categories and category groupings to subject to the diagnostic are defined in the process's validation rules (Validation rules page in the Processes section). Severity is indicated for each category and each grouping (**Blocking**, **Non-blocking**, **Only Diagnostic**, **None**)

Entity groupings are those only containing entity categories, i.e. amounts and/or entity journals, of the contributor.

Consolidator entity groupings are those which only contain entity categories, i.e. amounts and/or entity journals, where at least one is of the consolidator.

Mandatory notes diagnostic

Description of the check

This check verifies the presence of notes justifying the values entered on the gross amounts and on IC amounts. The check is only run if the amounts are not equal to zero. When the amounts are equal to zero, it is not mandatory to enter a justification.

The check only verifies the accounts associated with the selected process and, for entities (company, contributor node, consolidator node), it only checks the accounts present in the data model process associated with the same entity.

The check is only run on original scenarios, and on gross data; it reports any unbalanced amounts in the gross step present in the data model process of the entity subject to the diagnostic.
On IC data, it reports any unbalanced amounts on the IC step present in the data model process of the entity subject to the diagnostic.

It can be run on individual elements of the destinations, on first-level nodes of the destinations, or by summing the destinations according to what is indicated in the process's validation rules.

The check is only run on the amount category and only on entity category and consolidator entity groupings.

Accounts table and diagnostic

The following attributes of the accounts table are significant for running diagnostic data processing:

Attribute	Description
Notes check	The mandatory notes diagnostic is only run on accounts whose setting is not **Do not check** (Account table in the Data Model section).

The check is run according to the following logic:

If...	Then...
Notes check = **Check only Gross amounts**	the notes check must only be run for gross data.
Notes check = **Check Gross and IC amounts**	the notes check must be run for both intercompany data and gross data.

Categories to subject to the diagnostic

The categories and category groupings to subject to the diagnostic are defined in the process's validation rules (Validation rules page in the Processes section).
Severity is indicated for each category and each grouping (**Blocking**, **Non-blocking**, **Only Diagnostic**, **None**)

Entity groupings are those only containing entity categories, i.e. amounts and/or entity journals, of the contributor.

Consolidator entity groupings are those which only contain entity categories, i.e. amounts and/or entity journals, where at least one is of the consolidator.

IC matching diagnostic

Description of the check

Intercompany matching is carried out via a special cockpit which shows IC data by entity/counterparty entity (or entity-custom dimension2/entity-counterparty custom dimension2), which the manager users from the entity and the counterparty entity use to confirm that the loaded IC data is correct. See Intercompany Cockpit page in the Consolidation section.

This check does not go into the details of the declared IC data, but simply verifies that all the possible relationships that are/can be calculated by the cockpit have been matched. It is only run on original scenarios and reports any unbalanced amounts on the step to which the cockpit is related.

***Note**: one or several intercompany cockpits can be associated with every step of the process.*

Entity elements table and IC and diagnostic cockpits

The following attributes are significant for running diagnostic data processing:

Attribute	Description
IC Cockpits elements table	
IC matching for submission check	The IC matching diagnostic is only run for IC cockpits whose setting is not equal to **None** (Create IC Cockpits page in the Consolidation section).

Attribute	Description
Entity elements table	
IC matching	**Mandatory**: failure to run IC matching will block submission. **Optional**: IC matching is not mandatory for the entity because it cannot carry out that activity in time for the closing date of the relative step. *If A and B are entities for which IC matching is not mandatory while it is mandatory for entity C, then:* • *C does not have to match its own intercompany entries to A and B* • *A and B must match their own intercompany entries to C* • *in cases of IC relationships between A and B, the first to submit data does not have to match, while the other is obliged to do so* **Automatic**: relationships in which at least one of the two entities has this option active are matched automatically, consequently: • they do not appear in the IC cockpit • failure to run matching does not block submission See Entities list in the Data Model section.

The check is run according to the following logic:

If...	Then...
IC matching for submission check = **Non blocking**	the check is run and unmatched relationships are reported with **Non-blocking** severity.
IC matching for submission check = **Blocking**	the check is run and unmatched relationships are reported with **Blocking** severity.
IC matching for submission check = **Only Diagnostic**	the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** (for the contributor) or **Overview** (for the consolidator) cockpit tabs; in this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed.

Diagnostic of authorised blocking errors

Prerequisites

This check can only be run if the diagnostic result log is managed with error authorisation on the process subject to the diagnostic. Namely, if the following conditions are met:

- in the process's validation rules, **Enable detected errors trace** is not **Don't manage**,
- in the process's submission rules, the **With authorization of blocking errors** option is active
- in the process's validation rules, the **Don't propagate the authorized blocking error information** parameter is not set to **None**,

Validation and diagnostic rules

The following attributes are significant for running diagnostic data processing:

Attribute	Description
Validation	
Don't propagate the authorised blocking error information	The authorised blocking errors diagnostic is only run on processes that are not set to **None** (See and Submission Rules page in the Processes section)

The check is run according to the following logic:

If...	Then...
Don't propagate the authorized blocking error information = **None**	the check will not be run.
Don't propagate the authorized blocking error information = **Blocking**	the check is performed and the presence of authorised blocking errors is reported with **Blocking** severity.
Don't propagate the authorized blocking error information = **Non blocking**	the check is performed and the presence of authorised blocking errors are reported with **Non blocking** severity.
Don't propagate the authorized blocking error information = **Only Diagnostic**	the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** (for the contributor) or **Overview** (for the consolidator) cockpit tabs; in this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed.

Description of the check

When the diagnostic checks detect blocking errors, these can be authorised by manager users from the parent entity of the entity on which the error is detected, namely the same users that can reject the entity.

The authorised blocking error diagnostic verifies, for every entity subject to the diagnostic, whether there are any authorised blocking errors on the child entity and, if there are, it reports an error on the same entity. The error will gradually move up the hierarchy and will, in each case, need authorisation from the manager of the parent entity of the company to which it flows. This makes such errors visible to other users, including at higher levels of the organisational hierarchy associated with the process.

If...	Then...
the diagnostic check is run on an entity	its child entities can only be entity journals, provided that, in the process's submission rules, the **User enabled for Journals rejection** is equal to **User on the entity to which the journal belongs**.
the diagnostic check is run on a contributor node	its children can be: • entities • contributor nodes • entity journals belonging to the child entities of the node if, in the process's submission rules, the **User enabled for Journals rejection** is equal to **User on the parent node of the entity to which the journal belongs**
the diagnostic check is run on a consolidator node	its children can be: • entities • contributor nodes • consolidator nodes • entity journals or consolidation journals belonging to the same node if, in the process's submission rules, the **User enabled for Journals rejection** is equal to **User on the entity to which the journal belongs** • entity journals or consolidation journals belonging to child consolidator nodes of the node if, in the process's submission rules, the **User enabled for journals rejection** is equal to **User on the parent node of the entity to which the journal belongs** • entity journals belonging to the child entities of the node if, in the process's submission rules, the **User enabled for journals rejection** is equal to **User on the parent node of the entity to which the journal belongs**

The check is only run on original scenarios and reports any unbalanced amounts on the same step on which the errors have been authorised.

For the consolidator node diagnostic, if the child entity errors have been authorised on contributor steps, the report is made on the first consolidator or mixed step following the contributor step.

Plausibility diagnostic

Prerequisites

This check can only be run if the plausibility diagnostic is enabled on the process. See Plausibility diagnostic page.

Description of the check

This check verifies the amounts of a data collection related to a scenario/period against the amounts of the same data collection related to another scenario/period. This comparison generates error reports of varying severity, or warning messages.

For example, by setting dedicated logics, you can check the following aspects:

- that there are no percentage differences above a certain threshold between actual data and the budget data
- that the actual cumulated P&L data are neither less than, nor excessively greater than, that of the previous month
- that there are no percentage differences greater than a certain percentage
- that the items of a financial statement on a certain scenario are balanced with the same items which, on another scenario, are subdivided on an additional dimension (such as the Business Lines for the purposes of segment reporting)
- that the items of a financial statement or budget allocated to cost centres on specific categories are balanced with the total of the same items before the allocations

For these checks, the comparison rules related to the node of the plausibility rules grouping indicated on the process are applied. The severity of the errors is defined by the thresholds set in the plausibility logics table. See Plausibility check logics page.

The check is run for entities which, in the plausibility execution plan defined on the process, have **Severity type** equal to **Automatic**.

The check on the gross data is only run on original scenarios, and reports any unbalanced amounts in the gross step present in the data model process of the entity subject to the diagnostic. The check on IC data reports any unbalanced amounts on the IC step present in the data model process of the entity subject to the diagnostic.

Journal submission diagnostic

Description of the check

Journal submission is activated on the process in the Business Workflow mapping and you must specify a journal submission task workflow for every entity (company or consolidator node) for which you want to submit the journals. See Business Workflow Concepts and Validations in the Processes section.

> ***Note**: just as you must submit an entity before submitting any contributor node to which it belongs, you must submit a journal before submitting the parent entity to which it belongs.*

This check does not go into the details of the amounts adjusted by the journals, but simply verifies that the journals related to the entities to be checked have all been submitted.

Journals that have not been submitted are reported with different severities for manual journals and automatic journals, according to what is indicated in the process's validation rules. The check is run according to the set-up of the **Diagnose Manual journals submission** and **Diagnose Automatic journals submission**.

If...	Then...
Blocking	the diagnostic is run and non-submitted journals are reported with **Blocking** severity
Non blocking	the diagnostic is run and non-submitted journals are reported with **Non blocking** severity

If...	Then...
Only Diagnostic	the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** (for the contributor) or **Overview** (for the consolidator) cockpit tabs; in this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed.
Do not run	the diagnostic will not be run.

The check is only run on original scenarios and reports any unbalanced amounts in the step specified in the process's Business Workflow mapping for the entity subject to the diagnostic. For processes in which the data model process is defined by account/category, journal non-submission errors are only reported on the steps managed for the category related to the journal.

Draft journals diagnostic

Description of the check

The functionality related to draft journals is activated in the process's journal data entry rules. It is defined by category and is set in the dedicated page: categories for draft journals (Categories for draft journals page in the Data Collection section), where the functionality can be enabled for Entity journals, Consolidator entity journals and Consolidation journals categories.

This check verifies that, for the entities to be checked, namely entities and consolidator nodes, there are no journals that still have **Draft** status.

The check cannot be configured, that is, it cannot be omitted from the diagnostic checks and always produces errors with **Blocking** severity.

It is only run on original scenarios and reports any unbalanced amounts on the lowest step of the accounts present in the "draft" part of the journal.

Running diagnostic data processing

Contents

This section covers the following topics:

Entities diagnostic 32
Run diagnostic window - Contributor cockpit 32
Run diagnostic window - Consolidator cockpit - Overview tab 36
Run diagnostic window - Consolidator cockpit - Consolidation scenarios tab 40
Amounts diagnostic 43
Journals diagnostic 43
Diagnostic result window 44

Entities diagnostic

Description

This is the broadest diagnostic, which runs all of the checks on the entities and on the nodes. It allows you to customise the checks to be run via the dedicated run windows, accessible from the cockpits.

Run diagnostic window - Contributor cockpit

User

Processes tile > select process > Contributor > Detail summary tab > select entity on which to run the diagnostic > (Actions) Diagnostic > Diagnose

Navigations panel > Processes > select process > Contributor > Detail summary tab > select entity on which to run the diagnostic > (Actions) Diagnostic > Diagnose

Purpose of window

It allows you to set filters to run diagnostic data processing on original scenarios/periods and on entities and contributor nodes of the selected process.

Fields in window

Depending on the selected checks, some options can be disabled.

Element	Description
Filters	
Original scenarios/periods	For scenario/period submission processes, it contains the scenario/period selected on accessing the cockpit. Not editable. For single submission processes, it contains the last significant scenario/period of the process. Editable. The diagnostic can only be run on scenarios/periods that are not stored off-line and are visible to the user.

Element	Description
Entities	Contains the entities selected in the cockpit. Not editable. The diagnostic can only be run on entities that meet the following requirements: • they must belong to the process and at least one of the selected scenarios/periods • they cannot be only counterparty or only consolidator • they must be visible to the user
Contributor nodes	Contains the contributor nodes selected in the cockpit. Not editable. The diagnostic can only be run on nodes that meet the following requirements: • they must belong to the selected process • they must be to be submitted in the data collection process • they must be visible to the user
Advanced filters	
Steps	• **All** (default): the diagnostic is run on all steps of the business workflow associated with the process. • **Up to**: the diagnostic is run on the selected steps and all preceding steps. • **Selected**: the diagnostic is run on the selected steps. The diagnostic is only run on contributor and mixed steps.
Control groups	You can only select control groups which meet the following requirements: • they must belong to the process and the selected steps • they must be set in the table as to be diagnosed on the gross, intercompany or net data If no filter is set, all the control groups which meet the above conditions will be subject to the diagnostic. Any filter that is set will only act on the gross, IC and net control group diagnostic.
Categories	The diagnostic can only be run on categories that meet the following requirements: • they must belong to the selected process • they cannot be children of other categories • they must be visible to the user If not set, the diagnostic will be run on all the categories which meet the above conditions. The diagnostic is only run on the contributor categories. The filter is applied to all checks, except for IC matching, authorised blocking errors and plausibility.
Category nodes	You can only select category nodes used in defining the process's validation rules with "by node" granularity. If not set, the diagnostic will be run on all the category nodes which meet the above conditions. The diagnostic is only run on the category nodes containing only contributor categories. For every node, the categories are considered up to the last level, provided they belong to the process and are visible to the user. The filter is applied to all checks, except for IC matching, authorised blocking errors and plausibility.

Element	Description
Forms	You can only select forms which meet the following requirements: • they must belong to the process and the selected steps • they must be standard forms The filter is applied to all checks, except for IC matching, authorised blocking errors, journal submission and plausibility. If set, they have the following impacts: • The control groups diagnostic will only analyse control groups associated with the selected process. • The gross sign, IC sign, IC capacity, FX rate and mandatory notes diagnostic will only analyse the accounts associated with the selected forms. If not set, the diagnostic will be run on all the control groups and accounts, without taking account of whether or not they belong to the forms.
Plausibility logics	Can only be set if the plausibility diagnostic is active on the process. You can only select the plausibility logics present in the execution plan of entities or contributor nodes for the selected process. If not set, the diagnostic will be run on all the plausibility logics which satisfy the above condition. The filter is only applied to the plausibility diagnostic.
Errors threshold	• **None** (default): the diagnostic does not apply threshold filters. • **Above the Balancing threshold**: the diagnostic only reports unbalanced amounts above the balancing threshold set in the process's validation rules. • **Above the Submission threshold**: the diagnostic only reports unbalanced amounts above the submission threshold set in the process's validation rules. • **Above the specified threshold**: customised threshold and currency. If the currency is specified, the threshold amount is understood to be expressed in that currency and for the comparison it must be converted to the currency of the entity to which it is applied. The FX rate used for the conversion is the final FX rate. The filter is only applied to the gross, IC and net control groups diagnostic and to the IC capacity diagnostic.
Threshold	(Only if **Errors threshold** = **Above the specified threshold**) Indicates the value of the customised threshold.
Currency	(Only if **Errors threshold** = **Above the specified threshold**) Indicates the currency to be applied to value of the customised threshold.
Severity	Severity of unbalanced amount reported by the diagnostic. You can select one or several, but at least one must be selected. • **Blocking**: the entity data are not submitted. • **Non-blocking**: the entity data are submitted but errors are reported. • **Only Diagnostic**: the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** cockpit tabs (for the contributor). In this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed.
Checks	
Runs the selected check. At least one check must be selected.	
Select all	Selects all the listed checks.
Result	Runs the filtered control groups diagnostic only on the control group used for checking the annual result. This control group is $BS_<result account code>. Use the result account code that is set in the process's calculation logics. Moreover, in the process's validation rules, it is possible to specify the categories for which to show the result of this control group, including in the presence of balancing.

Element	Description
Gross control groups	Runs the diagnostic on control groups which, in the table, require the **Gross** data check.
Gross sign	Runs the gross sign diagnostic for accounts which, in the table, have a **Standard sign check** not equal to **Any**.
Net control groups	Runs the diagnostic on control groups which, in the table, require the **Net** data check.
IC control groups	Runs the diagnostic on control groups which, in the table, require the **Intercompany** data check.
IC sign	Runs the IC sign diagnostic for accounts which, in the table, have **IC sign check** not equal to **Do not check**.
IC Capacity	Runs the IC capacity diagnostic for accounts which, in the table, have **IC Capacity check** not equal to **Do not check**.
IC matching	Runs the IC matching check for cockpits which, in the table, have **IC matching for submission check** not equal to **None**.
FX rate	Runs the FX rates diagnostic for accounts which, in the table, have **FX rates check** not equal to **Do not check**.
Mandatory notes	Runs the mandatory notes diagnostic for accounts which, in the table, have **Notes check** not equal to **Do not check**.
Authorized blocking errors	Runs the authorised blocking errors diagnostic in the following cases: • In the process's validation rules, **Enable detected errors trace** is not equal to **Don't manage**. • In the process's validation rules, **Don't propagate the authorized blocking error information** is not equal to **None**. • In the process's submission rules, the **Override submission - With authorization of blocking errors** option is selected.
Plausibility checks	Runs the plausibility diagnostic for entities for which an execution plan has been defined on the process. Only present if the plausibility diagnostic is active on the process.
Journals submission	Runs the journal submission diagnostic in the following cases: • Journal submission has been enabled in the process's Business Workflow mapping, or a task workflow has been specified for at least one entity. • In the process's validation rules, **Diagnose Manual journals submission** and/or **Diagnose Automatic journals submission** is not equal to **None**.
Options	
Show the code of the form on the error row	For every error on the control group or account, shows the forms on which the control group accounts can be edited or the account itself. ***Note**: "standard" forms are editable from the diagnostic result window via both Excel and web forms.*
Also show balanced data	If selected, the control groups and accounts diagnostic also shows data which balance, and not unbalanced amounts only. In this case, it is not possible to run the IC matching, authorised blocking errors, plausibility and journal submission diagnostic. Moreover, you cannot request that the log be updated. Option only available if function 0004099 (Processes - Cockpit - Diagnostic - Also show balanced data) is active in the user role.
Update log	If selected, the content of the diagnostic reports log is updated at the end of the diagnostic. Option only available if, in the process's validation rules, **Enable detected errors trace** is not equal to **Don't manage**.

Actions in window

Element	Description
Save new filters set	Saves the run settings to be able to reuse them. ***Note**: the filters are saved by process and do not act on entities, for which the selection made in the cockpit always applies.*
Run	Runs the diagnostic according to the defined filters and settings.
Cancel	Closes the window without saving the settings.

Run diagnostic window - Consolidator cockpit - Overview tab

User

Processes tile > select process > Consolidator > Overview tab > select entity or node on which to run the diagnostic > (Actions) Diagnostic > Diagnose

Navigation panel > Processes > select process > Consolidator > Overview tab > select entity or node on which to run the diagnostic > (Actions) Diagnostic > Diagnose

Purpose of window

It allows you to set filters to run diagnostic data processing on all the cockpit's entities and nodes and on all steps of the process.

Fields in window

Depending on the selected checks, some options can be disabled.

Element	Description
Filters	
Original scenarios/periods	For scenario/period submission processes, it contains the scenario/period selected on accessing the cockpit. Not editable. For single submission processes, it contains the last significant scenario/period of the process. Editable. The diagnostic can only be run on scenarios/periods that are not stored off-line and are visible to the user.
Consolidator nodes	Shows the consolidator nodes selected in the cockpit and cannot be edited. The diagnostic can only be run on consolidator nodes that meet the following requirements: • they must belong to the selected process • they must be to be consolidated • they must be visible to the user
Run on Consolidation Scenario	Allows you to choose the types of scenario on which to run the diagnostic. • **No** (default): the diagnostic is only run on original scenarios/periods. • **Only**: the diagnostic is only run on consolidation scenarios/periods. • **Also**: the diagnostic is run on both original scenarios/periods and consolidation scenarios/periods.
Consolidation scenarios/periods	If **Run on Consolidation Scenario** = **Only** or **Also**, one or more consolidation scenarios/periods must be selected. The scenarios will be chosen from among those which are visible to the user and linked to the selected original scenarios/periods.
Consolidation data	If **Run on Consolidation Scenario** = **Only** or **Also**, one or more data types from among those which lead to the following types of consolidation must be selected: • **Proportional** • **Converted** • **Consolidable**
Run on Contributor Data	• **No** (default): the diagnostic is only run on consolidator nodes. • **Only**: the diagnostic is only run on entities and contributor nodes. • **Also**: the diagnostic is run on all entities (companies, contributor nodes and consolidator nodes).

<table>
<tr><th>Element</th><th>Description</th></tr>
<tr><td>Entities</td><td>If Run on Contributor Data = Only or Also.
Not editable. The field is defined with the entities that meet the following requirements:
they must belong to one of the selected consolidator nodes
they must belong to the process and at least one of the selected scenarios/periods
they cannot be only counterparty or only consolidator
they must be visible to the user</td></tr>
<tr><td>Contributor nodes</td><td>If Run on Contributor Data = Only or Also.
Not editable. The diagnostic can only be run on nodes that meet the following requirements:
they must belong to the selected process
they must be to be submitted in the data collection process
they must be visible to the user</td></tr>
<tr><td colspan="2">Advanced filters</td></tr>
<tr><td>Steps</td><td>All (default): the diagnostic is run on all steps of the business workflow associated with the process.
Up to: the diagnostic is run on all the preceding steps, including the present one.
Selected: the diagnostic is run on the selected steps.
The diagnostic is run according to the following criteria:<table><tr><th>If...</th><th>Then...</th></tr><tr><td>Run on Contributor Data = No</td><td>the diagnostic is only run on the consolidator or mixed steps.</td></tr><tr><td>Run on Contributor Data = Only</td><td>the diagnostic is only run on the contributor or mixed steps.</td></tr><tr><td>Run on Contributor Data = Also</td><td>the diagnostic is run on the contributor, consolidator or mixed steps.</td></tr></table></td></tr>
<tr><td>Control groups</td><td>The diagnostic can only be run on control groups that meet the following requirements:
they must belong to the process and the selected steps
in the table, they are set as to diagnose on the gross, intercompany or net data
If not set, the diagnostic will be run on all the control groups which meet the above conditions.
The filter is only applied to the gross, IC and net control groups diagnostic.</td></tr>
<tr><td>Categories</td><td>The diagnostic can only be run on categories that meet the following requirements:
they must belong to the selected process
they cannot be children of other categories
they must be visible to the user
If not set, the diagnostic will be run on all the categories which meet the above conditions.
The diagnostic is run according to the following criteria:<table><tr><th>If...</th><th>Then...</th></tr><tr><td>Run on Contributor Data = No</td><td>the diagnostic is only run on the consolidator categories.</td></tr><tr><td>Run on Contributor Data = Only</td><td>the diagnostic is only run on the contributor categories.</td></tr><tr><td>Run on Contributor Data = Also</td><td>the diagnostic is run on all contributor and consolidator categories.</td></tr></table>The filter is applied to all checks, except for IC matching, authorised blocking errors and plausibility.</td></tr>
</table>

<table>
<tr><th>Element</th><th>Description</th></tr>
<tr><td>Category nodes</td><td>You can only select category nodes used in defining the process's validation rules with "by node" granularity.
If not set, the diagnostic will be run on all the category nodes which meet the above conditions.
The diagnostic is run according to the following criteria:
<table><tr><th>If...</th><th>Then...</th></tr><tr><td>Run on Contributor Data = No</td><td>the diagnostic is only run on category nodes containing at least one consolidator category.</td></tr><tr><td>Run on Contributor Data = Only</td><td>the diagnostic is only run on the category nodes containing only contributor categories.</td></tr><tr><td>Run on Contributor Data = Also</td><td>the diagnostic will be run on all the category nodes.</td></tr></table>For every node, the categories are considered up to the last level, provided they belong to the process and are visible to the user.
The filter is applied to all checks, except for IC matching, authorised blocking errors and plausibility.</td></tr>
<tr><td>Forms</td><td>It is only possible to select forms which meet the following requirements:
they must belong to the process and the selected steps
they must be standard forms
The filter is applied to all checks, except for IC matching, authorised blocking errors, journal submission and plausibility.
If set, they have the following impacts:
The control groups diagnostic will only analyse control groups associated with the selected process.
The gross sign, IC sign, IC capacity, FX rate and mandatory notes diagnostic will only analyse the accounts associated with the selected forms.
If not set, the diagnostic will be run on all the control groups and accounts, without taking account of whether or not they belong to the forms.</td></tr>
<tr><td>Plausibility logics</td><td>Can only be set if the plausibility diagnostic is active on the process.
The diagnostic is run according to the following criteria:
<table><tr><th>If...</th><th>Then...</th></tr><tr><td>Run on Contributor Data = No</td><td>the selectable plausibility logics are those which are present in the execution plan of the consolidator nodes for the selected process.</td></tr><tr><td>Run on Contributor Data = Only</td><td>the selectable plausibility logics are those which are present in the execution plan of the entities or contributor nodes for the selected process.</td></tr><tr><td>Run on Contributor Data = Also</td><td>the selectable plausibility logics are those which are present in the execution plan of the entities, contributor nodes or consolidator nodes for the selected process.</td></tr></table>If not set, the diagnostic will be run on all the plausibility logics which satisfy the above condition.
The filter is only applied to the plausibility diagnostic.</td></tr>
</table>

Element	Description
Errors threshold	• **None** (default): the diagnostic does not apply threshold filters. • **Above the Balancing threshold**: the diagnostic only reports unbalanced amounts above the balancing threshold set in the process's validation rules. • **Above the Submission threshold**: the diagnostic only reports unbalanced amounts above the submission threshold set in the process's validation rules. • **Above the specified threshold**: customised threshold and currency. If the currency is specified, the threshold amount is understood to be expressed in that currency and for the comparison it must be converted to the currency of the entity to which it is applied. The FX rate used for the conversion is the final FX rate. The filter is only applied to the gross, IC and net control groups diagnostic and to the IC capacity diagnostic.
Threshold	(Only if **Errors threshold** = **Above the specified threshold**) Indicates the value of the customised threshold.
Currency	(Only if **Errors threshold** = **Above the specified threshold**) Indicates the currency to be applied to value of the customised threshold.
Severity	Severity of unbalanced amount reported by the diagnostic. At least one severity must be selected. • **Blocking**: the entity data are not submitted. • **Non-blocking**: the entity data are submitted but errors are reported. • **Only Diagnostic**: the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** cockpit tabs (for the contributor). In this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed.
Checks	
Runs the selected check. At least one check must be selected. If **Run on Consolidation Scenario** = **Only**, you can only run the Result, Gross control groups and Gross sign diagnostic.	
Select all	Selects all the listed checks.
Result	Runs the filtered control groups diagnostic only on the control group used for checking the annual result. This control group is $BS_<result account code>. Use the result account code that is set in the process's calculation logics. Moreover, in the process's validation rules, it is possible to specify the categories for which to show the result of this control group, including in the presence of balancing.
Gross control groups	Runs the diagnostic on control groups which, in the table, require the **Gross** data check.
Gross sign	Runs the gross sign diagnostic for accounts which, in the table, have a **Standard sign check** not equal to **Any**.
Net control groups	Runs the diagnostic on control groups which, in the table, require the **Net** data check.
IC control groups	Runs the diagnostic on control groups which, in the table, require the **Intercompany** data check.
IC sign	Runs the IC sign diagnostic for accounts which, in the table, have **IC sign check** not equal to **Do not check**.
IC Capacity	Runs the IC capacity diagnostic for accounts which, in the table, have **IC Capacity check** not equal to **Do not check**.
IC matching	Runs the IC matching check for cockpits which, in the table, have **IC matching for submission check** not equal to **None**.
FX rate	Runs the FX rates diagnostic for accounts which, in the table, have **FX rates check** not equal to **Do not check**.
Mandatory notes	Runs the mandatory notes diagnostic for accounts which, in the table, have **Notes check** not equal to **Do not check**.

Element	Description
Authorized blocking errors	Runs the authorised blocking errors diagnostic in the following cases: • In the process's validation rules, **Enable detected errors trace** is not equal to **Don't manage**. • In the process's validation rules, **Don't propagate the authorized blocking error information** is not equal to **None**. • In the process's submission rules, the **Override submission - With authorization of blocking errors** option is selected.
Plausibility checks	Runs the plausibility diagnostic for entities for which an execution plan has been defined on the process. Only present if the plausibility diagnostic is active on the process.
Journals submission	Runs the journal submission diagnostic in the following cases: • Journal submission has been enabled in the process's Business Workflow mapping, or a submission task workflow has been specified for at least one entity. • In the process's validation rules, **Diagnose Manual journals submission** and/or **Diagnose Automatic journals submission** is not equal to **None**.
Options	
Show the code of the form on the error row	For every error on the control group or account, shows the forms on which the control group accounts can be edited or the account itself. ***Note**: "standard" forms can be edited from this window via both Excel and web forms.*
Also show balanced data	If selected, the control groups and accounts diagnostic also shows data which balance, and not unbalanced amounts only. In this case, it is not possible to run the IC matching, authorised blocking errors, plausibility and journal submission diagnostic. Moreover, you cannot request that the log be updated. Option only available if function 0004099 (Processes - Cockpit - Diagnostic - Also show balanced data) is active in the user role.
Update log	If selected, the content of the diagnostic reports log is updated at the end of the diagnostic. Option only available if, in the process's validation rules, **Enable detected errors trace** is not equal to **Don't manage**. Can only be selected if **Run on Consolidation Scenario** is not equal to **Only**, as the diagnostic reports log is only managed for the original scenarios/periods.

Actions in window

Element	Description
Ok	Runs the diagnostic according to the defined filters and settings.
Cancel	Closes the window without saving the settings.

Run diagnostic window - Consolidator cockpit - Consolidation scenarios tab

User

Processes tile > select process > Consolidator > Consolidation scenarios tab > select entity or node on which to run the diagnostic > (Actions) Diagnostic > Diagnose

Navigation panel > Processes > select process > Consolidator > Consolidation scenarios tab > select entity or node on which to run the diagnostic > (Actions) Diagnostic > Diagnose

Purpose of window

It allows you to set filters to run diagnostic data processing on consolidation scenarios.

Fields in window

Depending on the selected checks, some options can be disabled.

<table>
<tr><th>Element</th><th>Description</th></tr>
<tr><td colspan="2">Filters</td></tr>
<tr><td>Consolidation scenarios</td><td>Contains one or several consolidation scenarios/periods from among those visible to the user, linked to the selected original scenario/period and in which the perimeter contains at least one entity belonging to the selected consolidator nodes.</td></tr>
<tr><td>Data to diagnose</td><td>• All (default): the diagnostic is run on all entities (companies, contributor nodes and consolidator nodes).
• Consolidator: the diagnostic is only run on consolidator nodes.
• Contributor: the diagnostic is only run on contributor nodes.</td></tr>
<tr><td>Consolidator nodes</td><td>If Data to diagnose = All or Consolidator, it shows the consolidator nodes selected in the consolidation scenarios cockpit and is not editable.
The diagnostic can only be run on consolidator nodes that meet the following requirements:
• they must belong to the selected process
• they must be to be consolidated
• they must be visible to the user</td></tr>
<tr><td>Contributor nodes</td><td>Only if Data to diagnose = All or Contributor.
Not editable. The diagnostic can only be run on nodes that meet the following requirements:
• they must belong to one of the selected consolidator nodes
• they must have at least one entity belonging to at least one of the selected consolidation scenarios
• they must belong to the selected process
• they must be to be submitted in the data collection process
• they must be visible to the user
Not editable.</td></tr>
<tr><td>Entities</td><td>If Data to diagnose = All or Contributor.
Not editable. the field is defined with the entities that meet the following requirements:
• they must belong to one of the selected consolidator nodes
• they must belong to the process and at least one of the selected consolidation scenarios/periods
• they cannot be only counterparty or only consolidator
• they must be visible to the user</td></tr>
<tr><td colspan="2">Advanced filters</td></tr>
<tr><td>Steps</td><td>• All (default): the diagnostic is run on all steps of the business workflow associated with the process.
• Up to: the diagnostic is run on the selected steps and all preceding steps.
• Selected: the diagnostic is run on the selected steps.
The diagnostic is run according to the following criteria:
<table><tr><th>If...</th><th>Then...</th></tr><tr><td>Data to diagnose = All</td><td>the diagnostic is run on all steps (contributor, consolidator and mixed).</td></tr><tr><td>Data to diagnose = Consolidator</td><td>the diagnostic is only run on the consolidator and mixed steps.</td></tr><tr><td>Data to diagnose = Contributor</td><td>the diagnostic is only run on the contributor and mixed steps.</td></tr></table></td></tr>
</table>

Element	Description
Errors threshold	• **None** (default): the diagnostic does not apply threshold filters. • **Above the Balancing threshold**: the diagnostic only reports unbalanced amounts above the balancing threshold set in the process's validation rules. • **Above the Submission threshold**: the diagnostic only reports unbalanced amounts above the submission threshold set in the process's validation rules. • **Above the specified threshold**: customised threshold and currency. If the currency is specified, the threshold amount is understood to be expressed in that currency and for the comparison it must be converted to the currency of the entity to which it is applied. The FX rate used for the conversion is the final FX rate. The filter is only applied to the gross,
Threshold	(Only if **Errors threshold** = **Above the specified threshold**) Indicates the value of the customised threshold.
Currency	(Only if **Errors threshold** = **Above the specified threshold**) Indicates the currency to be applied to value of the customised threshold.
Severity	Severity of unbalanced amount reported by the diagnostic. At least one severity must be selected. • **Blocking**: the entity data are not submitted. • **Non-blocking**: the entity data are submitted but errors are reported. • **Only Diagnostic**: the check is only run if explicitly requested by manually running the diagnostic from the **Detail summary** cockpit tabs (for the contributor). In this case it is possible to customise the filters before running the data processing. However, whenever diagnostics are run automatically (from the cockpit/from other data processing), the check is not performed.
Show the code of the form on the error row	For every error on the control group or account, shows the forms on which the control group accounts can be edited or the account itself. ***Note**: "standard" forms can be edited from the diagnostic result window via both Excel and web forms.*
Also show balanced data	If selected, the control groups and accounts diagnostic also shows data which balance, and not unbalanced amounts only. Option only available if function 0004099 (Processes - Cockpit - Diagnostic - Also show balanced data) is active in the user role.
Checks	
Runs the selected check. At least one check must be selected.	
Select all	Selects all the listed checks.
Result	Runs the filtered control groups diagnostic only on the control group used for checking the annual result. This control group is $BS_<result account code>. Use the result account code that is set in the process's calculation logics. Moreover, in the process's validation rules, it is possible to specify the categories for which to show the result of this control group, including in the presence of balancing.
Gross control groups	Runs the diagnostic on control groups which, in the table, require the **Gross** data check.
Gross sign	Runs the gross sign diagnostic for accounts which, in the table, have a **Standard sign check** not equal to **Any**.
Amount type	One or more amount types must be selected: • **Consolidable** (preselected) • **Converted** • **Proportional**

Actions in window

Element	Description
Save new filters set	Saves the run settings to be able to reuse them. ***Note**: the filters are saved by process and do not act on entities, for which the selection made in the cockpit always applies.*
Run	Runs the diagnostic according to the defined filters and settings.
Cancel	Closes the window without saving the settings.

Amounts diagnostic

Description

This diagnostic runs all the diagnostic checks on the gross amounts, while it only runs the following checks on intercompany amounts:

- IC control groups
- IC sign
- IC Capacity
- FX rate
- IC matching
- Mandatory notes
- Authorized blocking errors

Run method

This diagnostic is run from the contributor and consolidator cockpits. For gross amounts diagnostic, see the Gross Amounts page, for intercompany amount diagnostic, see IC Amounts page in the Data collection section.

It does not require you to set parameters when it is run. The entities to be subject to the diagnostic (companies and/or consolidator nodes) are set automatically, depending on the category and the entity of every selected row:

If...	Then...
the category is contributor	the entity diagnostic is run.
the category is consolidator	the diagnostic is run on the consolidator node to which the entity belongs

Journals diagnostic

Description

This diagnostic runs the following checks on journals:

- Gross, net and IC control groups
- Gross sign
- IC sign
- IC Capacity
- FX rate
- Plausibility checks

Run method

This diagnostic can be run according to several methods:

- from journal headers from the **Diagnostic** action (Actions from journal headers in the Data collection section), updating or not updating the error log,
- from journal rows from the **Diagnostic** action (Actions from journal rows in the Data collection section), updating or not updating the error log.

You do not have to set parameters when running it, as these are calculated automatically based on previous configurations.

If the **Diagnose and update log** option is selected, the error log will also be updated.

This update is only available if, in the process's validation rules, **Enable detected errors trace** is not equal to **Don't manage** (Validation rules page in the Processes section).

Diagnostic result window

At the end of the diagnostic data processing, CCH Tagetik automatically opens a window displaying the detected errors.

The user, via the Point of view (see User Settings), can choose whether to display a summary dashboard, to better understand how errors are distributed (organised by control or entity), or directly the detail of errors.

Control Dashboard

Purpose of the page

This dashboard displays a card for each control on which the diagnostic has detected errors, detailing the type of error:

- Blocking: it will not be possible to submit entity data on which errors are present with this severity
- Authorised (only if the option "Override Submission: With authorization of blocking errors" is activated in the process submission rules): blocking errors, but which have been authorised
- Below threshold: blocking errors, but where unbalanced amount is below the submission threshold set on the process validation rules
- Submitted: blocking errors, but relating to journals that have been submitted
- Non-blocking
- Balanced (only if the diagnostic has been run with the option 'Also show balanced data' or if in the process validation rules, categories have been specified for which to show the result even if there is a balancing)

By selecting a card, you can view the error details filtered by the selected check. See Diagnostic Error Detail

Buttons on the page

Element	Description
	It allows filters on the main dimensions. Filters affect the display of the dashboard and reports.
	Allows you to select the dashboard type: Controls or Entities.
	It allows reports to be extracted from the diagnostic result. See Diagnostic Reports
	This shows the list of entities/steps for which the diagnostic has not detected any errors.

Filters on the page

From the dashboard, it is possible to make filters for:

- scenario/period: allows filtering by amount type (original, proportional, converted, consolidated) and/or by the scenarios/periods on which errors were detected
- step: allows filtering for a single step among those on which errors were detected
- entities: allows you to filter alternatively by entities (also choosing whether you want to display the result also for consolidator entities), contributor nodes or consolidator nodes on which errors were detected
- category/node category/journal: allows filtering alternatively by category, category node or journal on which errors have been detected

Entity Dashboard

Purpose of the page

This dashboard displays a row for each entity on which the diagnostician has detected errors.

Each row provides the following information:

- Entity type: Entity ; Contributor node; Consolidator node
- Entity code and description

- Total number of errors and their distribution (number of blocking, authorised, below threshold, submitted , non-blocking errors, balanced)

By selecting a row, the filtered error detail for the selected entity can be displayed. See Diagnostic Error Detail

Buttons on the page

Element	Description
	It allows filters on the main dimensions. Filters affect the display of the dashboard and reports.
	Allows you to select the dashboard type: Controls or Entities.
	It allows reports to be extracted from the diagnostic result. See Diagnostic Reports. -
	This shows the list of entities/steps for which the diagnostic has not detected any errors.

Filters on the page

From the dashboard, it is possible to make filters for:

- scenario/period: allows filtering by amount type (original, proportional, converted, consolidated) and/or by the scenarios/periods on which errors were detected
- step: allows filtering for a single step among those on which errors were detected
- control: allows filtering of the various diagnostic controls on which errors have been detected
- category/node category/journal: allows filtering alternatively by category, category node or journal on which errors have been detected

Diagnostic report

Introduction

Reports can be generated from the diagnostics result window in order to carry out various types of analysis on the errors detected.

Excel report by Scenario/Period.

This report displays the detail of errors detected by the diagnostics, organised by scenario/period and step.

It is the most comprehensive report (e.g. it also contains descriptions of all dimensions) and allows the data to be analysed in various ways, thanks to the possibility of filters.

Excel Report by Check to Perform

This report displays the detail of errors detected by the diagnostic, sorted by control.

The data are organised in such a way that they can be printed.

PDF Report by Check to Perform

This report displays the detail of errors detected by the diagnostic, sorted by control, in PDF format.

The data are organised in such a way that they can be printed.

This report is the one that is saved upon entity submission when the option 'Store diagnostic result on submission' is enabled in the process submission rules.

Control groups drill down report

This report displays, for each control group, the details of the data rows that originated the unbalanced amount.

If there are a large number of reports on control groups, it may take a long time to generate the report.

For this reason, the report is only available if the diagnostician has detected control group errors and if the diagnostician has not been run with the option 'Also show balanced data'.

Diagnostic Error Detail

Purpose of the page

This page allows the detail of errors detected by the diagnostic to be displayed.

The page can be accessed:

- by selecting individual cards in the check dashboard
- by selecting individual entities in the entity dashboard
- after running the diagnostic on journals
- from the entity/journal submission result window

Actions on this page

Element	Description
	Show Control group information. Only available for single selection and only for Control Group errors. Opens a window showing the main information on the control group, including a list of its parent and child accounts. **Drill Down.** Available only for single selection and only for errors of Control Groups and Plausibility Checks (for the latter, only if the error is on a contributor node). For Control Groups, it opens a window displaying the amount and/or journal rows that generated the error, divided between parent and child accounts. For Plausibility Checks, it opens a window displaying the amounts divided among the various Entities belonging to the node. **Show forms.** Only available for single-selection. Moreover, it is only present for Control Group and Account errors (Gross Sign, IC Sign, IC Capacity, FX Rate, Mandatory Notes). Opens a window showing the forms in which it should be possible to edit the control group or account, with the option to open the web/excel form. **Comment error.** Only present if the diagnostic was performed with the 'Update Log' option. Available for single and multiple selection. In the case of a single selection, it opens a window where you can view and manage the comments present for the error. In the case of multiple selection, a window opens where you can write a comment that will be entered on all selected errors. In this case, the menu item is disabled if rows are selected on locked or display-only scenarios/periods.
	Comment on all records. Only present if the diagnostic was performed with the 'Update log' option and if the displayed scenarios/periods are not locked or in display-only mode. Opens a window where you can write a comment on all displayed errors. **Export to Excel** Present only on the 'All Checks' grid and only if the page was opened following the execution of the diagnostic on journals or from the entity/ journal submission result window. Exports a report showing the details of all errors detected by the diagnostic, organised by scenario/period and step.

Attributes on the page

Element	Description
Blocking	Indicates whether the error is blocking and therefore it will not be possible to submit the entity data. Non-filterable/sortable.
Balancing	Only present if the diagnostic was performed with the option 'Also show balanced data'. In the case of unbalanced amount, it displays the symbol . Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes.
In progress	Only present if there is at least one category for the process for which the status 'Draft journal' can be managed. If the error relates to a draft journal, it displays the symbol .
Scenarios	Code and description of the scenario.
Period	Code and description of the period.
Step	Step code.
Check	Present only on the 'All Checks' grid. The possible values are: • Gross control groups • IC control groups • Net control groups • Gross sign • IC sign • IC Capacity. • FX rate • Mandatory notes • Plausibility checks • Authorized blocking errors • IC matching • Journals submission • Draft Journals
Source control group	Code and description of the control group. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups.
Error detail	Notes in the control group list. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups.
Accounts	Account code and description. Filled only for Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks.
Account node	Code (hierarchy|node) and description of the accounts node. Filled only for Plausibility Check errors.
Entity	Entity code and description.
Entity node	Code (hierarchy|node) and description of the Entity node.

Element	Description
Entity type	The possible values are: • Entity • Contributor nodes • Consolidator Nodes
Category	Category code and description. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks, journal Submission, Draft journals.
Category node	Code (hierarchy|node) and description of the category node. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks.
Journal number	Journal code and description. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Plausibility Checks, Authorised Blocking Errors, Journal submission, Draft journals.
Custom dimension 1...5	Only present if the custom dimension is managed. Custom dimension code and description. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks. Custom dimension 2 is also defined for IC Matching errors.
Custom dimension 1...5 node	Only present if the custom dimension is managed. Code (hierarchy|node) and description of the custom dimension node. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks.
CTP entity	CTP Entity code and description. Filled only for errors in IC Control Groups, IC Sign, FX rate, Mandatory Notes, IC Matching, Plausibility Checks.
CTP Entity Node	Code (hierarchy|node) and description of the CTP Entity node. Filled only for errors in IC Control Groups, Plausibility Checks.
CTP Custom dimension 2	Only present if custom dimension 2 is managed. CTP custom dimension 2 code. Filled only for errors in IC Control Groups, IC Sign, FX rate, Mandatory Notes, IC Matching, Plausibility Checks.
Unit	Currency code. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks.
Amount to detail	Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups.
Detail amount	Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups.
Difference	For errors on control groups, it is the difference between the detail amount and the amount to be detailed. For plausibility errors, it is the difference between the amount to be compared and the reference amount. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Plausibility Checks. Non-filterable/sortable.
Amount	Filled only for Gross Sign, FX rate, Mandatory Notes errors.
Gross amount	Valued only for IC Sign, IC Capacity errors.
IC amount	Valued only for IC Sign, IC Capacity errors.

Element	Description
Invoice currency	Invoice currency code. Filled only for FX rate errors.
Invoice Currency amount	Filled only for FX rate errors.
Operator	Control group operator. The possible values are: • = • < • <= • > • >= • <> • Coherence • Bi-directional coherence • Coherence by element • Bi-directional coherence for element Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups.
Sign	Check of sign specified for the account. The possible values are: • Debit • Credit Filled only for gross sign errors.
IC Capacity	IC capacity check specified for the account. The possible values are: • Check <= • Check = Filled only for IC Capacity errors.
IC sign	Check of IC sign specified for the account. The possible values are: • Debit • Credit • As Gross amount Filled only for IC Sign errors.
Cockpit	Cockpit code and description. Filled only for IC Matching errors.
Authorized Entity	Code and description of the authorized Entity. Filled only for authorized blocking errors.
Authorized Entity node	Code (hierarchy|node) and description of the authorized Entity node. Filled only for authorized blocking errors.
Rule	Code and description of the plausibility rule. Filled only for Plausibility Check errors.
Ctp entity for Segment	code and description of the CTP Entity for Segment. Filled only for Plausibility Check errors.

Element	Description
Ctp entity for Segment node	Code (hierarchy|node) and description of the Ctp entity for Segment node. Filled only for Plausibility Check errors.
Reference Scenario	Reference scenario code. Filled only for Plausibility Check errors.
Reference period	Reference period code. Filled only for Plausibility Check errors.
Comparison evaluation	Comparison currency code. Filled only for Plausibility Check errors.
Amount to compare	Filled only for Plausibility Check errors.
Reference amount	Filled only for Plausibility Check errors.
Expression	Result of the plausibility rule preliminary check. Filled only for Plausibility Check errors.
Materiality check expression	Plausibility check result. Filled only for Plausibility Check errors.
Row type	The possible values are: • Amount • Journal • Aggregated Filled only for Plausibility Check errors.
Data type	The possible values are: • Gross amount • IC amount • Gross Adjustment • IC adjustment • Gross aggregate • IC aggregate • Cause Filled only for Plausibility Check errors.
Elimination code	Elimination logic code. Filled only for Plausibility Check errors.
Amount Type	The possible values are: • Original • Proportional • Converted • Consolidable

Element	Description
Error type	The possible values are: • Blocking: Entity data cannot be submitted • Non blocking • Diagnostic only: the check was performed because it was explicitly requested through manual execution of the diagnostic • Warning • Authorized Blocking : blocking error, but which has been authorised • Submitted: blocking error, but relating to a journal that has been submitted • Blocking Below Threshold: Blocking error, but where the unbalanced amount is below the submission threshold set on the process validation rules • Result: this is not an error; the row was reported as it related to the control group of the year result
Further information	Only present if the diagnostic was performed with the 'Update Log' option. Indicates whether there are any comments about the error.
Second last comment	Only present if the diagnostic was performed with the 'Update Log' option. Non-filterable/sortable.
Last comment	Only present if the diagnostic was performed with the 'Update Log' option. Non-filterable/sortable.
Last comment user	Only present if the diagnostic was performed with the 'Update Log' option. Non-filterable/sortable.
Last comment date	Only present if the diagnostic was performed with the 'Update Log' option. Non-filterable/sortable.
Note	The possible values are: • FX rate is not present in the FX rates table • FX rate is not present in the FX rates table for the previous scenario • Incorrect setup of the reference scenario/period Filled only for Plausibility Check errors.
Rule notes	Notes in the plausibility rule list. Filled only for Plausibility Check errors.
Form	Only present if the diagnostic was run with the option 'Show the code of the form on the error row'. Form code. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes.

Logging diagnostic data

Introduction

The diagnostic check generates information for immediate use, The results are shown but not saved. To see the results again, you must run the data processing again. To save the data returned by the diagnostic, you must activate a log results function, which generates a log.

This function is only available for the original scenarios and is activated by process by the validation rules (Validation rules page in the Processes section) according to the following criteria:

If...	Then...
Enable detected errors trace = **Don't manage**	the errors detected by the diagnostic check will not be logged.
Enable detected errors trace = **Only manage for blocking signals**	only errors with **Blocking** severity will be logged.
Enable detected errors trace = **Manage the table for all signals**	all the errors detected by the diagnostic check will be logged.

If the log function is active, the data in the log is updated on every submission or running of the diagnostic checks where the **Update log** option is selected.

Authorisation of diagnostic errors

Introduction

For processes in which the logging of diagnostic data is active, you can manually authorise blocking errors on the data. This operation changes the severity of the error from **Blocking** to **Authorized blocking** and allows you to submit the entity regardless.

This function is activated by the process's submission rules with the **Override submission - With authorization of blocking errors** option (Submission rules page in the Processes section).

The detected errors can only be authorised by manager users from the parent entity of the entity on which the error is detected, namely the same users that can reject the entity. Comments can be inserted on authorised errors, but if this is not done, a note that it is an authorised error is inserted in any case.

***IMPORTANT**: it is possible to authorise several errors at once, but only if they can all be authorised. Otherwise, the system will return an error message: "It was not possible to authorise some errors".*

To find out the reason why an error cannot be authorised, try to authorise the individual error: the specific reason why the error cannot be authorised will be indicated.

Whenever the content of the log is updated, an authorised error that was previously present in the log may no longer be detected by the diagnostic checks, which leads to the following behaviour:

If...	Then...
there are no comments for the error.	the error will be removed from the log.
there are comments for the error.	the error will be saved and marked as "Correct". Correct errors will no longer be considered, so if the same error arises again, it will be considered to be a new error. ***Note**: correct errors are not normally shown, due to a pre-set filter on the log. In any case, CCH Tagetik can be configured to show them.*

For processes that manage the diagnostic log with authorisation of blocking errors, it may be useful to activate the authorised blocking errors diagnostic check (Authorised blocking errors diagnostic).

Requirements for authorising an error

To be able to authorise an error, the following conditions must be fulfilled:

- The error must have **Blocking** severity.
- The error cannot have already been corrected, or be an unbalanced amount that is no longer present in the system but still stored in the log due to the presence of comments.
- The error cannot have already been authorised, unless it was authorised but the authorisation was then invalidated.
- The entity to which the error relates cannot be submitted.
- If the error concerns a journal of an entity for which journal submission is required, the journal cannot be submitted.
- There must be a justification for the error, inserted by a user other than the authorising user.
- The user must be enabled to authorise errors on the relative entity, or must be a manager of the parent entity of the entity to which the error relates.

Information on authorised errors

When an error is authorised, the following information is tracked in the log:

Attribute	Description
Authorized	This attribute indicates whether or not the error has been authorised (= true).
Authorized amount	Amount at the time of authorisation. • For errors related to control groups: difference between the amount of the child accounts and the amount of the parent accounts • For errors related to the gross sign: amount • For errors related to the IC sign: IC amount • For errors related to the IC capacity: difference, as an absolute value, between the IC amount and the gross amount
User authorization	User that performed the authorisation
Date authorization	Date and time at which the error was authorised

***Note**: the data related to authorised errors remain editable, and consequently some errors may no longer exist or amounts may change when the log is subsequently updated.*

Removal of error authorisations

Error authorisations can be removed. Only the users who performed the authorisation are allows to do this, namely manager users of the parent entity of the entity on which the error is detected. Comments can be inserted on errors for which authorisation is removed, but if this is not done, a note that the authorisation was removed is inserted in any case.

***IMPORTANT**: you can remove the authorisation on several errors at the same time. If this is not possible on some errors, the operation is interrupted and an error message is shown: "It was not possible to remove the authorisation of some errors".*

To find out the actual reason why the authorisation of an error cannot be removed, you can attempt to authorise the individual error. In this case, the system will indicate the reason why the authorisation cannot be removed for the specific error.

To find out the reason why the authorisation of an error cannot be removed, try to authorise the individual error: the specific reason why the authorisation of an error cannot be removed will be indicated.

You cannot submit entities that have errors that were initially authorised and have then been removed.

Requirements for removing the authorisation of an error

To be able to remove the authorisation of an error, the following conditions must be fulfilled:

- The error must have **Blocking** severity.
- The error cannot have already been corrected, or be an unbalanced amount that is no longer present in the system but still stored in the log due to the presence of comments.

- The error must have already been authorised and the authorisation cannot have been invalidated or removed.
- The entity to which the error relates cannot be submitted.
- If the error concerns a journal of an entity for which journal submission is required, the journal cannot be submitted.
- The user must be enabled to remove the authorisation, or be a manager of the parent entity of the entity to which the error relates.

Invalidation of errors

The invalidation of errors is an automatic operation run by CCH Tagetik every time that the log is updated, if one of the following conditions is fulfilled:

- Unbalanced amounts have changed sign.
- The value of the unbalanced amount has increased in absolute terms.

Diagnostic results log

User

Process tile > select process > Contributor > Detail summary tab or Step tab > select entity > (Actions) Diagnostic > Diagnostic results log

Process tile > select process > Consolidator > Overview tab or Step tab > select entity > (Actions) Diagnostic > Diagnostic results log

Processes Tile > select process with dimension by step > select entity > User Tools > Validations > Diagnostic Results Log

Purpose of the page

This page allows access to archived diagnostic data for both online and offline scenarios/periods. Only for data on online scenarios/periods is it possible to enter comments or authorise errors.

This page allows you to authorize the detected blocking errors in order to submit the entity

When the log is opened, some initial filters are applied:

- selected scenario/period, in case of submission process for scenario / period
- selected entities (entities or contributor nodes or consolidator nodes)
- selected step, if the log is opened from the cockpit with dimension by step or from one of the step tabs, in the standard cockpit
- only errors that have not yet been corrected are displayed

Once the log is open, filters can be modified or cancelled. The only implicit filters, which are always applied and cannot be modified by the user, are the filter on the selected process and the type of data displayed: if the log is opened from an offline scenario/period, it will only be possible to display offline data; if the log is opened from an online scenario/period, it will only be possible to display online data.

Actions on this page

Element	Description
	Show Control group information: Available for single selection and control group errors only. Opens a window showing the main information on the control group, including a list of its parent and child accounts. **Drill Down**: Available only for single selection and only for Control Group errors and only on online scenarios/periods. Opens a window displaying the amount and/or journal rows that generated the error, divided between parent and child accounts. **Show forms**: Available only for single selection and only for Control Group and Account errors (Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes). Opens a window showing the forms in which it should be possible to edit the control group or account, with the option to open the web/excel form. **Comment error**: available for single and multiple selection. In the case of a single selection, it opens a window where you can view and manage the comments present for the error. In the case of multiple selection, a window opens where you can write a comment that will be entered on all selected errors. In this case, the menu item is disabled if rows are selected on locked or display-only scenarios/periods. **Authorize blocking error**: Available for single and multiple selection and only on non-locked or display-only scenarios/periods and only if in the process submission rules, the information Enable blocking errors authorization = true and if at least one blocking, open and unauthorised error has been selected. Authorize selected blocking errors **Remove authorization for blocking error**: Available for single and multiple selection and only on non-locked or display-only scenarios/periods and only if in the process submission rules, the information Enable blocking errors authorization = true and if at least one blocking, open and authorised error has been selected. Removes the authorisation of selected blocking errors.
	**Comment on all records**: Only available on non-locked or display-only scenarios/periods. Opens a window where you can write a comment on all displayed errors. **Authorize all blocking errors**: Only available on non-blocked or display-only scenarios/periods and only if in the process submission rules, the information Enable blocking error authorization = true. Authorises all displayed blocking errors. **Remove authorization for all blocking errors**: Only available on non-blocked or display-only scenarios/periods and only if in the process submission rules, the information Enable blocking error authorization = true. Removes the authorisation of all displayed blocking errors. **Export to Excel:** Exports a report showing the details of all errors in the log, organized by scenario/period and step. The report only considers any filters applied to the fields Scenario/Period, Step, Entity, Entity Node, Entity Type, Successful.

Attributes on the page

Element	Description
Blocking	Indicates whether the error is blocking and therefore it will not be possible to submit the entity data. Non-filterable/sortable.
In progress	Only present if there is at least one category for the process for which the status 'Draft journal' can be managed. If the error relates to a draft journal, it displays the symbol .
Successful	Indicates whether the error has been fixed and is therefore no longer detected by the diagnostics checks.
Authorized	Indicates whether the error has been authorised.
Invalid	Indicates whether, after authorization, the error has been invalidated and will therefore have to be authorized again.

Element	Description
Scenarios	Code and description of the scenario.
Period	Code and description of the period.
Step	Step code.
Check	Present only on the 'All Checks' grid. The possible values are: • Gross control groups • IC control groups • Net control groups • Gross sign • IC sign • IC Capacity. • FX rate • Mandatory notes • Plausibility checks • Authorized blocking errors • IC matching • Journals submission • Draft Journals
Source control group	Code and description of the control group. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups.
Error detail	Notes in the control group list. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups.
Accounts	Account code and description. Filled only for Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks.
Account node	Code (hierarchy|node) and description of the accounts node. Filled only for Plausibility Check errors.
Entity	Entity code and description.
Entity node	Code (hierarchy|node) and description of the Entity node.
Entity type	The possible values are: • Entity • Contributor nodes • Consolidator Nodes
Category	Category code and description. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks, journal Submission, Draft journals.
Category node	Code (hierarchy|node) and description of the category node. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks.
Journal number	Journal code and description. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Plausibility Checks, Authorised Blocking Errors, Journal submission, Draft journals.
Custom dimension 1...5	Only present if the custom dimension is managed. Custom dimension code and description. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks. Custom dimension 2 is also defined for IC Matching errors.

Element	Description
Custom dimension 1...5 node	Only present if the custom dimension is managed. Code (hierarchy|node) and description of the custom dimension node. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks.
CTP entity	CTP Entity code and description. Filled only for errors in IC Control Groups, IC Sign, FX rate, Mandatory Notes, IC Matching, Plausibility Checks.
CTP Entity Node	Code (hierarchy|node) and description of the CTP Entity node. Filled only for errors in IC Control Groups, Plausibility Checks.
CTP Custom dimension 2	Only present if custom dimension 2 is managed. CTP custom dimension 2 code. Filled only for errors in IC Control Groups, IC Sign, FX rate, Mandatory Notes, IC Matching, Plausibility Checks.
Unit	Currency code. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Gross Sign, IC Sign, IC Capacity, FX rate, Mandatory Notes, Plausibility Checks.
Amount to detail	Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups.
Detail amount	Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups.
Difference	For errors on control groups, it is the difference between the detail amount and the amount to be detailed. For plausibility errors, it is the difference between the amount to be compared and the reference amount. Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups, Plausibility Checks. Non-filterable/sortable.
Amount	Filled only for Gross Sign, FX rate, Mandatory Notes errors.
Gross amount	Valued only for IC Sign, IC Capacity errors.
IC amount	Valued only for IC Sign, IC Capacity errors.
Invoice currency	Invoice currency code. Filled only for FX rate errors.
Invoice Currency amount	Filled only for FX rate errors.
Operator	Control group operator. The possible values are: • = • < • <= • > • >= • <> • Coherence • Bi-directional coherence • Coherence by element • Bi-directional coherence for element Filled only for errors in Gross Control Groups, IC Control Groups, Net Control Groups.

Element	Description
Sign	Check of sign specified for the account. The possible values are: • Debit • Credit Filled only for gross sign errors.
IC Capacity	IC capacity check specified for the account. The possible values are: • Check <= • Check = Filled only for IC Capacity errors.
IC sign	Check of IC sign specified for the account. The possible values are: • Debit • Credit • As Gross amount Filled only for IC Sign errors.
Cockpit	Cockpit code and description. Filled only for IC Matching errors.
Authorized Entity	Code and description of the authorized Entity. Filled only for authorized blocking errors.
Authorized Entity node	Code (hierarchy|node) and description of the authorized Entity node. Filled only for authorized blocking errors.
Rule	Code and description of the plausibility rule. Filled only for Plausibility Check errors.
Ctp entity for Segment	code and description of the CTP Entity for Segment. Filled only for Plausibility Check errors.
Ctp entity for Segment node	Code (hierarchy|node) and description of the Ctp entity for Segment node. Filled only for Plausibility Check errors.
Reference Scenario	Reference scenario code. Filled only for Plausibility Check errors.
Reference period	Reference period code. Filled only for Plausibility Check errors.
Comparison evaluation	Comparison currency code. Filled only for Plausibility Check errors.
Amount to compare	Filled only for Plausibility Check errors.
Reference amount	Filled only for Plausibility Check errors.
Expression	Result of the plausibility rule preliminary check. Filled only for Plausibility Check errors.
Materiality check expression	Plausibility check result. Filled only for Plausibility Check errors.

Element	Description
Row type	The possible values are: • Amount • Journal • Aggregated Filled only for Plausibility Check errors.
Data type	The possible values are: • Gross amount • IC amount • Gross Adjustment • IC adjustment • Gross aggregate • IC aggregate • Cause Filled only for Plausibility Check errors.
Elimination code	Elimination logic code. Filled only for Plausibility Check errors.
Error type	The possible values are: • Blocking: Entity data cannot be submitted • Non blocking • Warning • Blocking Below Threshold: Blocking error, but where the unbalanced amount is below the submission threshold set on the process validation rules
Authorized amount	Amount at the time of authorization. • For control group errors: difference between the amount of the child accounts and the amount of the parent accounts • For gross sign errors: amount • For IC sign errors: IC amount • For IC capacity errors: difference, in absolute value, between the IC amount and the gross amount
User authorization	User that performed the authorization.
Date authorization	Date and time when the error was authorized.
User Upd	User who has updated the content of the log.
Date Upd	Date and time at which the log content was updated.
Further information	Indicates whether there are any comments about the error.
Second last comment	Non-filterable/sortable.
Last comment	Non-filterable/sortable.
Last comment user	Non-filterable/sortable.
Last comment date	Non-filterable/sortable.

Element	Description
Note	The possible values are: • FX rate is not present in the FX rates table • FX rate is not present in the FX rates table for the previous scenario • Incorrect setup of the reference scenario/period Filled only for Plausibility Check errors.
Rule notes	Notes in the plausibility rule list. Filled only for Plausibility Check errors.

Ask AI for Diagnostics

Ask AI for Diagnostics acts as a dedicated assistant for the end user, with the goal of optimizing and simplifying the diagnostic data processing, especially when managing the various operational options available See Run Diagnostic Window - Process with dimensions by step.

AskAI Chat

The chat button is only available when the **0004270 Ask AI** functionality is enabled **on the process cockpit**.

This button, if enabled, is only available in the multidimensional process cockpit if the process has at least one step on Entity dimension.

The user's request can be made manually or by voice command. See Making a textual request in the Ask AI chat and Making a request in Ask AI chat with voice command.

If the request is recognised, interpreted and validated correctly by Ask AI, the chat will show the user the message 'Data processing started', indicating that the processing has begun.

If, however, Ask AI cannot understand or interpret the request, the chat will display an informative message indicating that the request could not be recognized: 'From your request it was not possible to understand what information you are referring to'.

In the event that the request does not pass validation, a detail window will open containing the specifications of the unfulfilled validations.

***IMPORTANT**: the quality of Ask AI answers improves with clear and specific questions. Avoid ambiguities, vague descriptions or overly general references.*

Execution mode

Through the Ask AI chat, the user can send requests to run the diagnostic.

The request formulated in natural language is interpreted by the Ask AI chat, which identifies the most appropriate matches between filters and execution options.

Ask AI executes the diagnostic in line with the selected process, respecting all process constraints and user-specific restrictions

Chat requests must refer to dimensions, dimensional elements and process parameters to be analysed. If essential information is missing, the Ask AI chat will retrieve it as follows:

If...	...then
It is not clear from the request whether it refers to diagnostics of the original scenarios or of the consolidation scenarios	Ask AI will diagnose the original scenarios
The request does not specify one or more scenarios/periods	Ask AI will diagnose the original scenarios and: • in case of submission process for scenario/period, it will run the diagnostic of the selected scenario/period • in case of single submission process, it will diagnose all valid scenarios/periods linked to the selected process
The request does not specify specific entities (Entities, contributor nodes, consolidator nodes)	Ask AI will perform diagnostics of all entities for which the user making the request is responsible
The request does not specify one or more controls	Ask AI will perform diagnostics of all valid controls for the selected process

Let's look at some examples of requests.

If...	...then
Run diagnostic	Ask AI will perform the diagnostic by calculating scenarios and entities as above
Run the diagnostic of my Entities	Ask AI will only run diagnostics on Entities excluding contributor and consolidator nodes
Perform diagnostics of consolidated data	Ask AI will run diagnostics on consolidation scenarios by analysing the data of the consolidable amount type
Run diagnostics on contributor nodes	Ask AI will only run diagnostics on contributor nodes, excluding Entities and consolidator nodes
Run the diagnostic on intercompany data	Ask AI will perform checks on intercompany data such as IC Control Groups, IC Capacity, IC Sign, IC Matching
Run diagnostic and update log	Ask AI will run the diagnostic and simultaneously update the log

Error Management in Ask AI for Diagnostic

When using Ask AI, there may be instances when a response cannot be obtained.

In such cases, error messages are displayed to help the user better understand the problem that occurred.

Condition	Error message displayed
Ask AI cannot understand or interpret the request	From your request it was not possible to understand what information you are referring to
Generic error occurred while processing the request, not due to a problem in understanding the request	A problem appears to have occurred. Please try again later or contact support for additional assistance.
The AskAI chat session expires after a period of inactivity and the page must be reloaded...	Your session has expired due to inactivity. Please reload the page.
There are too many requests to Ask AI	AskAi not available. Please try again later.

Calculation logics

Overview

Through the calculation logics users are able to evaluate an account as a formula based on other accounts storing the result into the Tagetik application database.

The calculation logics can be distinguished, according to the calculation type to perform, as follows:

- **standard logics**. Used to value an account as sum (or difference) of other accounts. They are typically used to calculate an account as the algebraic sum of its details (or variations), with basic possibility of acting on the dimension's detail, and on equal scenario / period;
- **custom multidimensional calculation logics**. Used when advanced non-linear operations have to be performed (e.g. multiplying or dividing two accounts, complex formulas), when factors and (or result) need to be read (written) in different scenarios / periods (e.g. reading from previous scenario) or on the dimensions (e.g. bundling on dimensions data to read or to write, reading from a fixed point or overriding the output element on given dimensions).

In both cases calculation logics need to be classified as belonging to one of the following Type:

- **Basic** calculation logics; all system-generated logics are created as *Basic*. System logics are the ones related to **result accounts**, **control groups**, plug accounts and those logics calculating the account gross amount as the sum of its inter-company declarations.
- **Other** calculation logics; user defined logics can be created both as *Basic* and *Other*.


The only difference between *Basic* and *Other* is the sequence with which they are calculated in the original scenario data processing: first the logics with type *Basic* are processed, then the type *Other*. Thus, running on Original Scenarios, calculation logics belonging to the *Other* type are able to read the results of *Basic* calculation logics, but not the other way around.

Running on Consolidation Scenarios the *Type* property is ignored: all logics are processed according to the sorting identified in the *Calculation Logic Graph* (in the *Process* settings), and they are able to read data generated from previous logics regardless of their *Type*.


System-generated logics are marked with a code starting with $ (control groups), $PL and $BS (result accounts), $L (plug accounts) and $IC (IC on gross amounts). System-generated logics cannot be modified in their configuration with the exception of the control group generated ones, where it is possible to add dimensional specifications (filters, fixed points and overrides).

From the **Data Processing** home page clicking the *Account Calculations* opens up a contextual menu with following elements:

- **List**, opening the calculation logics elements management screen;
- **Grouping**,opening to the calculation logics grouping structure management screen.

Calculation logics list

This screen lists all defined calculation logics, allowing to delete, modify or add new logics. In the right side pane, basing on the logic settings, are grouped the logic **Attributes**, **Filters and Overrides**, grouping **Relationships** and **Info**.

Attributes Pane

Each calculation logic is defined by following properties:

- **Code**; mandatory, this is the rule's unique ID of the rule. Only characters of ISO LATIN type are allowed. The maximum allowed length is 30 characters, it cannot start with $ (reserved for system-

generated logics).
- **Description** in all managed languages, mandatory the administration language**;**
- **Calculation type**. Mandatory, the possible types are:
  - **Sum**. The output account is calculated as the algebraic sum of other accounts (input accounts). It is possible to limit the set of data to run through the definition of suitable filters;
  - **Custom MD**. Allows you to obtain one or more accounts calculated through a custom script (TGKML Tagetik syntax). Unlike the "Sum" logics that performs simple sums of addenda with coefficient, custom multidimensional logics allow to perform calculations that combine input accounts in advanced formulas and logics, leveraging methods of dimensional manipulation adopted from the OLAP analysis (MDX). The calculation script has to be entered in the **Formula** attribute of the logic;
  - **IC on Gross amount**. Allows you to calculate the output account gross amount as the sum of IC data of one or more accounts (including the output account).

     Whether in the accounts table the **Gross amount equal to the IC sum** option is enabled, running the **Generate Gross amounts formulas from IC** deploy creates automatically the calculation logics which define an account's gross data from the IC data of the same account.

  - **Net on Gross amount**. Allows you to calculate the output account gross amount as the sum of net data (gross data - IC data) of one or more accounts (other than the output account).
- **Basic calculation logic** flag, to mark the logic as Basic logic.
- **Override sequence** field, allowing to specify the calculation logics run sequence. For each data collection process, enabled calculation logics are executed basing on their sequence number: the execution sequence of all the logics sharing the same sequence number is automatically settled by the system basing on their mutual dependencies (i.e. basing on input and output accounts of each of them). The resulting sequencing, based on sequence numbers and dependencies, is created automatically during the process deployment, and it is visible in the process configuration. Although overriding the calculation sequence it is usually not necessary and not advisable as it could affect performances, this could be useful when the input and output accounts identify fake circular dependencies. See the user manual for further details. For further details about how the system defines the dependencies graph, see **Enable calculation logic run.**
- **Run on original scenarios**. This property defines when to run the calculation logic on the Original Scenarios; possible values are:
  - **No**. The calculation logic won't run.
  - **Automatic and Submission**. The logic runs automatically together with Formulas (Basic or Other) data processing, and as part of the process step submission.
  - **Automatic**. The logics runs automatically together with Formulas (Basic or Other) data processing.
  - **On demand**. The logic runs only manually (see User Tools), via data entry form or ETL.
- **Run on consolidation scenarios**. This property defines when to run the calculation logic on the Conslidation Scenarios; possible values are:
  - **No**. The calculation logic won't run.
  - **Automatic**. The logics runs automatically together with Formulas (Basic or Other) data processing.
  - **On demand**. The logic runs only manually (see User Tools), via data entry form or ETL.

These execution options can be overridden by process in order to have different running policy per process (See Process configuration for further details).

When the calculation type is *Sum*, *IC on Gross amoun*t, *Net on Gross amount*, the Attributes pane shows the section **Calculation Details** where it is possible do define following properties:

- **Input details**
- **Output account**, defines the account where to write the calculation logic result.
- **Run on gross amounts**. When enabled the calculation logics reads and writes gross amounts. Enabling this option locks the output account for data entry on gross amounts: this can be overridden by process (See Process configuration for further details).
- **Run on IC amounts**.When enabled the calculation logics reads and writes IC amounts. Enabling this option locks the output account for data entry on IC amounts: this can be overridden by process (See Process configuration for further details). Running on IC amounts is possible only when gross amounts are calculated too.

- **Run on journals**. When enabled the calculation logics reads and writes journals. Enabling this option locks the output account for data entry on the journals. Additionally, when running on journals it is possible to specify the properties below:
  - **Compensate Entities on journals.** When this option is enabled, P&L result is calculated as the inverse of sum of all the other normal accounts, compensating it on the entities journals to avoid that in costs / revenues eliminations entry the system generates two rows of P&L net result with opposite sign (only per Basic logic types);
  - **Keep CTP on journals**: When enabled the detail of the input accounts counterparty is taken into consideration.

When the calculation type is *Custom MD*, the Attributes pane does not show the section Calculation Details section and in his place shows the **Formula** field is where to write the MD script. On saving the system checks the script syntax.

Filters and Overrides Pane

The **Filters and Overrides** pane is available only for calculation logics type **Sum** allowing to detail the logic with dimensional specifications such as:

- filtering on dimensions
- forcing to write the result to a given dimension element, regardless of the inputs

Whilst overrides are defined selecting a dimension element in the **Override** combo, filters are specified typing a CodeSet in the **Filter** text area. On saving the system checks the script syntax.

For further details on how to define Codesets and on how filters and overrides work refer to **Definition of a "Sum" calculation logic with dimensional specifications.**

 Entity dimension does not accept overrides.

 Category dimension does accept overrides only if the rule does not run on journals.

Relationships Pane

In the **Relationship** pane, it is possible to link the selected logic with the grouping nodes. It is possible to group the same logic to different nodes.

System logic info Pane

In case of selected logic is a system logic the **System logic info** pane is shown. This pane details the origin of the system logic.

Possible system logic are:

- **Control Group** generated logic; in this case the pane shows the source control group code and a link to direct access the control group management screen.
- **Plug account** generated logic; in this case the pane shows a link to direct access the IC elimination rules management screen, where plug account are defined.
- **IC on gross** amount generated logic; in this case the pane shows a link to direct access the Accounts management screen, where this setting is defined.
- **Result account** generated logic; in this case the pane shows the source result account set code and a link to direct access the result accounts management screen.

For further details on each of the above refer to the related section of the user manual.

Calculation logic, Input details

From the link in the attributes pane of the calculation logics elements management screen you can access the **Input details** management screen and define the input accounts for the selected calculation logic.

This screen lists all the input accounts, allowing to delete, modify or add new ones. In the right side pane are grouped the **Attributes** defining each input as follows:

- **Input Account**; it can be a single account or an hierarchy node: in this case all the accounts related to that node - at any level - are added as input accounts.
- **Markup**; it specifies a markup to be applied when summing the selected input account (or node).

Basing on the calculation logic settings, and only in case of logic type *Sum*, available dimension are grouped in two sections: **Filters** and **Fixed Points**.

- **Filters section** groups all dimensions where no overrides have been specified for the current logic: filtering an input account on a given dimension filters on reading that account only from the dimension elements identified by the filter.
- **Fixed Point section** groups all dimension having an override in place: defining a Fixed Point forces to read the inputs from the dimension element (or node) specified, regardless of the execution and header filters (i.e. reading from a fixed point can read outside the execution filters).

Filters and Fixed points are specified typing a *CodeSet* in the related text area. On saving the system checks the script syntax. For further details on how to define *CodeSets* refer to **Definition of a "Sum" calculation logic with dimensional specifications.**

Input details, definition results

On saving, input definition is automatically deployed and the full list of input and output accounts is available through the *Definition Results* hyperlink in the input details management screen.


Input and output accounts are listed together with the "cube" where they are read (or written): for further details about the available cube types see **Cubes and dimensions.**

Input and output accounts are used to identify dependencies between calculation logics.


Dependencies are automatically identified within rules sharing the same sequence number, just taking into consideration what the input and output accounts are for each calculation logic, regardless of dimensional specification. This could lead to fake dependencies that can be solved manually specifying the correct sequencing (**Override sequence** attribute).

Grouping

Calculation logics are managed through an aggregation structure of grouping type.

CCH Tagetik provides a predefined aggregation node (with code $), then the grouping structure can be customized with additional nodes according to any specific need. If needed several nodes can be marked as default relationships for new logics: this will automatically relate newly defined logics to those nodes.

A typical use of grouping nodes is to mark the different logics that need to be executed in different processes (e.g. planning logics, consolidation logics, etc.).

See **aggregation structures** for further details on managing aggregation structures.

Debug MD calculation logics

Tagetik allows the user to run in debug mode a calculation logic of "Multi-dimensional script" type. This feature allows to run a TGKML script both on original scenarios and on the consolidation scenarios, and then to analyze the script run steps by a graphical display in order to check the partial results and to analyze the performance of the various steps.

To run a TGKML script in debug mode, it is necessary to open the calculation logics elements table and, after selecting the "Multi-Dimensional Script" logic to run, click on the icon below the magnifier.

The system opens the **Debug MD calculation logics** window ; the selected MD appears on the right of the window, while on the left side it is necessary to select the run filters:

- the **process** on which the script must be run;
- the **scenario / period** on which the script must be run.
- the list of **entities** to run; if the field is left empty, all entities will be run
- a set of filters to limit the elements of the various dimensions on which the script must be run:
  - **Data type - Amounts**: gross data and IC entries are run
  - **Data type - Journals**: entity and consolidation journals are run
  - **Categories**: filters the data to run on the basis of the categories;
  - **Journal number**: filters the journals to run according to the selection;
  - **Custom dimension 1...5**: filters the data to run on the basis of the elements of the custom dimension 1...5.



The script is run only for those entities for which at least a not submitted step exists: therefore, if the selected list contains elements for which all steps are submitted, the data processing won't be run on such elements

When clicking on "**Debug**", the selected scrip will be run applying the set up filters.

By hovering over the button, the "**Debug without details**" item appears. With this item it is possible to run the debug without the execution details.

After running the script, the system opens a window showing the execution steps in the form of graph.

The tree structure describing the execution of the script is displayed on the left of the window. The root of this tree is represented by the calculation logic. Each node of the tree can relate to the account to calculate, to an input account, to an arithmetical operator or a numerical coefficient.



- The “Final operations” node is not a debug node but a node which allows monitoring the loading operation period and then evaluate the performance of the calculation logic;
- If the TGKML syntax of the calculation logic provides for a multicube assignment, several nodes with the same structure, one for each cube, will be created in the tree

By clicking on the node, it is possible to display on the bottom of the panel the arithmetical **Expression** evaluated in relation to that node.



On the nodes related to the input accounts and arithmetical operators (in bold), it is possible to display on the right of the window the partial results related to the selected input account / arithmetical operator.

Activate calculation logics run

After completing the parametrization of the calculation logics and defining the structure of their grouping, before running them it is necessary to specify in each process which node of the grouping contains the required logics.

See: Calculation logics.

Run a calculation logic from workflow and from job

Tagetik allows the user to run a calculation logic also from a workflow model task or from a job task. Below we describe how to define a task from which to run a calculation logic inside a workflow model.

If we assume that we are inside a task flowchart and we want to define a new task for the execution of the calculation logics, we will have to follow the steps below:

- define a new task by dragging the Calculation logics icon, assign a code and a description to the task;
- the following parameters can be set up:
  - **Calculation logic type.** Typology of the calculation logic to be run. Possible values:
    - Basic
    - Others
    - All
  - **Data type.** Typology of data on which the calculation logics has to be run. Possible values:
    - **Amounts.** The calculation logic is run on gross amounts and IC entries;
    - **Journals.** The calculation logic is run on the entity journals and on the consolidation journals;
    - **All** (default value).
  - **Category / Category node**. Filters data on which the calculations logic will be run by category;
  - **Journal number.** Filters the journals on which the calculations logic will be run by journal number;
  - **Calculation logic / Calculation logic node.** The system runs only the specified calculation logics;
  - **Sequence.** The system runs only the calculation logics with the specified sequence;
  - **Account / Account hierarchy node.** The system determines automatically the calculation logics that involve the specified accounts;
  - **Custom dimension 1...5 / Custom dimension 1...5 hierarchy node** Filters data on which the calculations logic will be run by the elements of the custom dimension 1...5;
  - **Filter calculation logics by data.** The system automatically determines the logics to be run based on the filter set up on the dimensions on which the calculation logics will be run;
  - **Consolidation scenario.**

 Let's see in detail the possibly set filters for partializing the calculation logics to be run.

- **Filter 1.** If we specify to filter the calculation logics depending on data, the system processes all the calculation logics having as input accounts the accounts present on the data tables after applying the dimension filters and all the "indirect" ones, namely those having as input accounts the calculated accounts from the previous logics. In presence of this filter, the system ignores possible other filters of 2,3,4 type;
- **Filter 2.** In presence of a filter on accounts and /or on an accounts hierarchy's, the system processes all the calculation logics having as input accounts those obtained be the union of the single accounts and the accounts related to the specified nodes. Also in this case, the system process all the "indirect" logics having as input accounts the calculated accounts from the from the previous logics. In presence of this filter the system ignores the possible other filters of 3,4 type;
- **Filter 3.** In presence of a filter on the calculation logics and/or on the grouping's nodes, the system processes only the calculation logics which belong to the union of the single logics and the logics related to the nodes. This filter is applied in intersection with the filter on the sequence;
- **Filter 4.** In presence of a filter on the sequence the system processes all the calculation logics with that sequence.

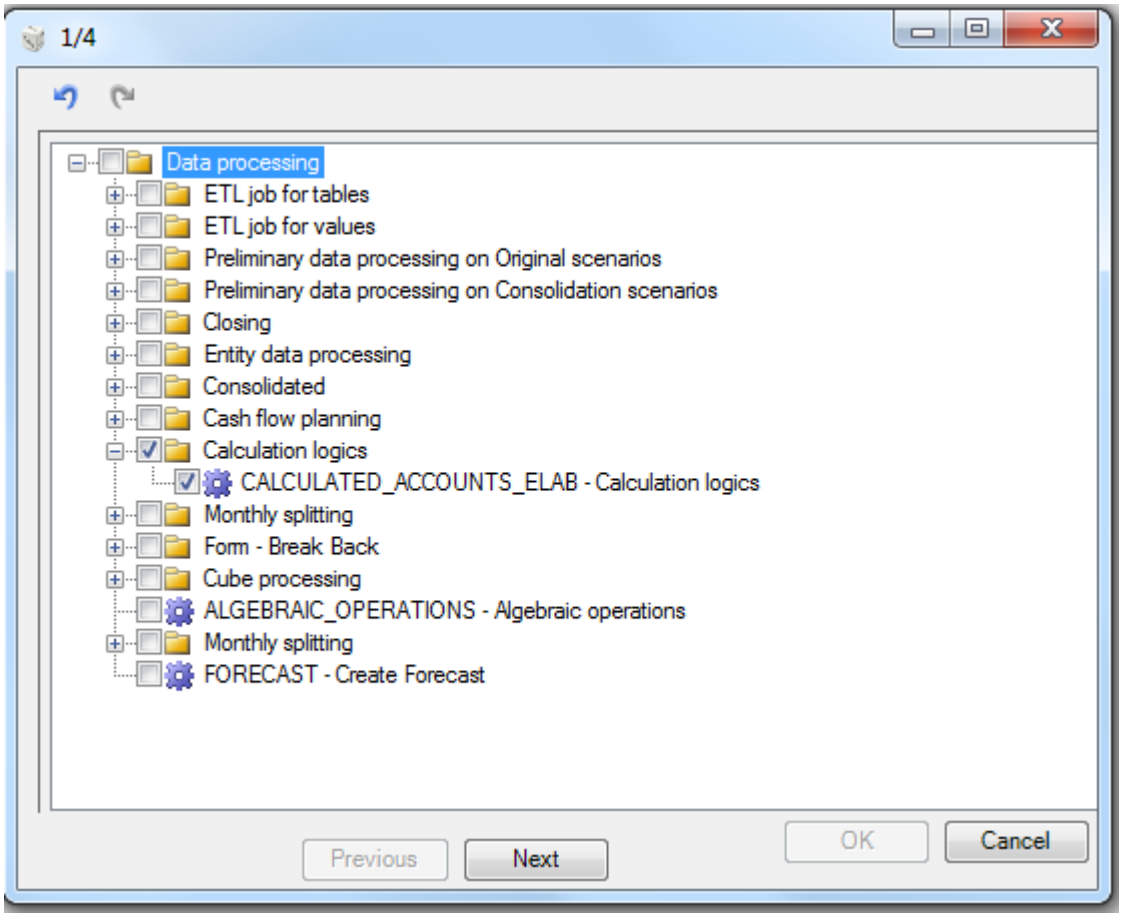
Run calculation logic from form

A calculation logic can be run also from a data entry form. For example, if you insert the sold quantities of a given product and the related prices in a form and you want to calculate the revenues from those two values, you can define a calculation logic (revenue = quantity * price). Such logic defined on the web interface can be included in a form and then run automatically when saving the data entry form or upon user's request.

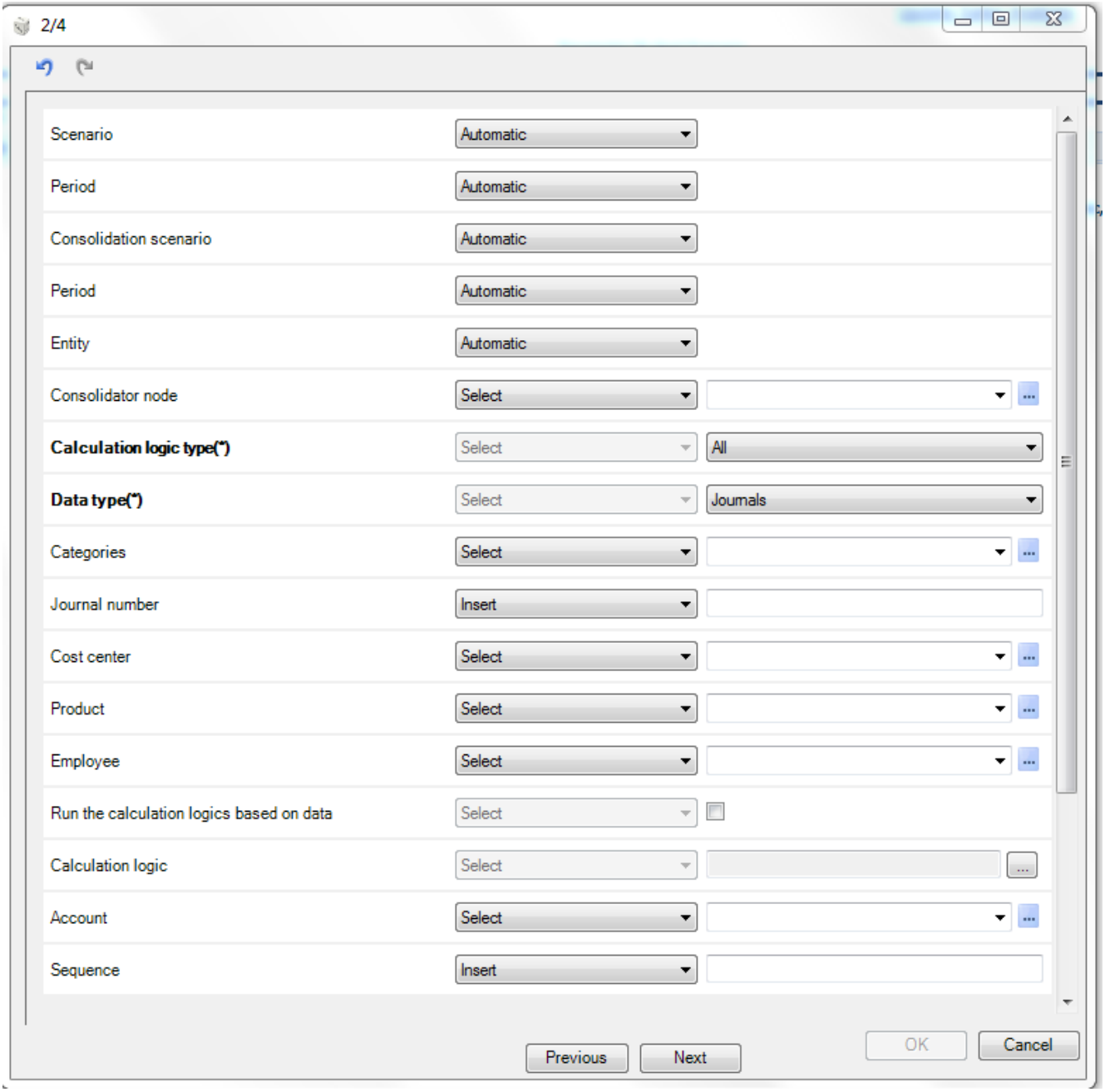
To add the execution of a calculation logic inside a form you have to:

- open the form in Design mode;
- click on the **Data processing** button in the Design ribbon bar or, in the Open Items left panel, select the row of the form in which the data processing has to be inserted and then right click and select *Run data processing > New data processing*.

The system displays a window in which the user has to select the **Calculation logics > CALCULATED_ACCOUNTS_ELAB - Calculation logics** check box and then click on **Next**.



Then the user has to define the required parameters for the data processing. The parameters in bold are mandatory.



When defining the parameters, it is necessary to **specify how the value of the parameter has to be retrieved**. Available methods:

- **Automatic**. This option can be selected only if the value to be used is chosen by the user when launching a form through the use of a parameter. The parameter can be configured as form filter, tab filter, matrix filter, row/column filter. In this way, the value used in the data processing will depend on the user's selection at runtime. Whereas, if the value is entered in the matrix, it must be specified in the run parameters for the data processing (Therefore, if the Automatic option is used, the value will not be taken into account). If a parameter is not mandatory or significant for the current data processing, its value can be retrieved automatically.
- **Excel Cell.** The system retrieves the parameter's value from an Excel cell. The user must specify the cell containing the value by using the ${H30} (cella H30) syntax.
- **Insert**. The user must insert the parameter's value inside the corresponding text box.
- **Select.** The user must select the parameter's value from a list of values
- **Cell field.** The system retrieves the parameter's value from a cell field.

The following parameters can be set up:

- **Scenario > Period**
- **Consolidation Scenario / Period**
- **Consolidator Entity / Node**
- **Calculation logic type.** Typology of the calculation logic to be run. Possible values:
  - Basic
  - Others
  - All
- **Data type.** Typology of data on which the calculation logics has to be run. Possible values:
  - **Amounts.** The calculation logic is run on gross amounts and IC entries;
  - **Journals.** The calculation logic is run on the entity journals and on the consolidation journals;
  - **All** (default value).
- **Category / Category node**. Filters data on which the calculations logic will be run by category;
- **Journal number.** Filters the journals on which the calculations logic will be run by journal number;
- **Custom dimension 1...5 / Custom dimension 1...5 hierarchy node** Filters data on which the calculations logic will be run by the elements of the custom dimension 1...5;
- **Run calculation logics based on data.** The system automatically determines the logics to be run based on the filter set up on the dimensions on which the calculation logics will be run; The required formula is identified based on the accounts that turn out to be changed upon saving. All formulas having the changed accounts will be marked for execution in the form; all dependent formulas will marked as well.
- **Calculation logic / Calculation logic node.** The system runs only the specified calculation logics;
- **Account / Account hierarchy node.** The system determines automatically the calculation logics that involve the specified accounts;
- **Sequence.** The system runs only the calculation logics with the specified sequence;

 Let's see in detail the possibly set filters for partializing the calculation logics to be run.

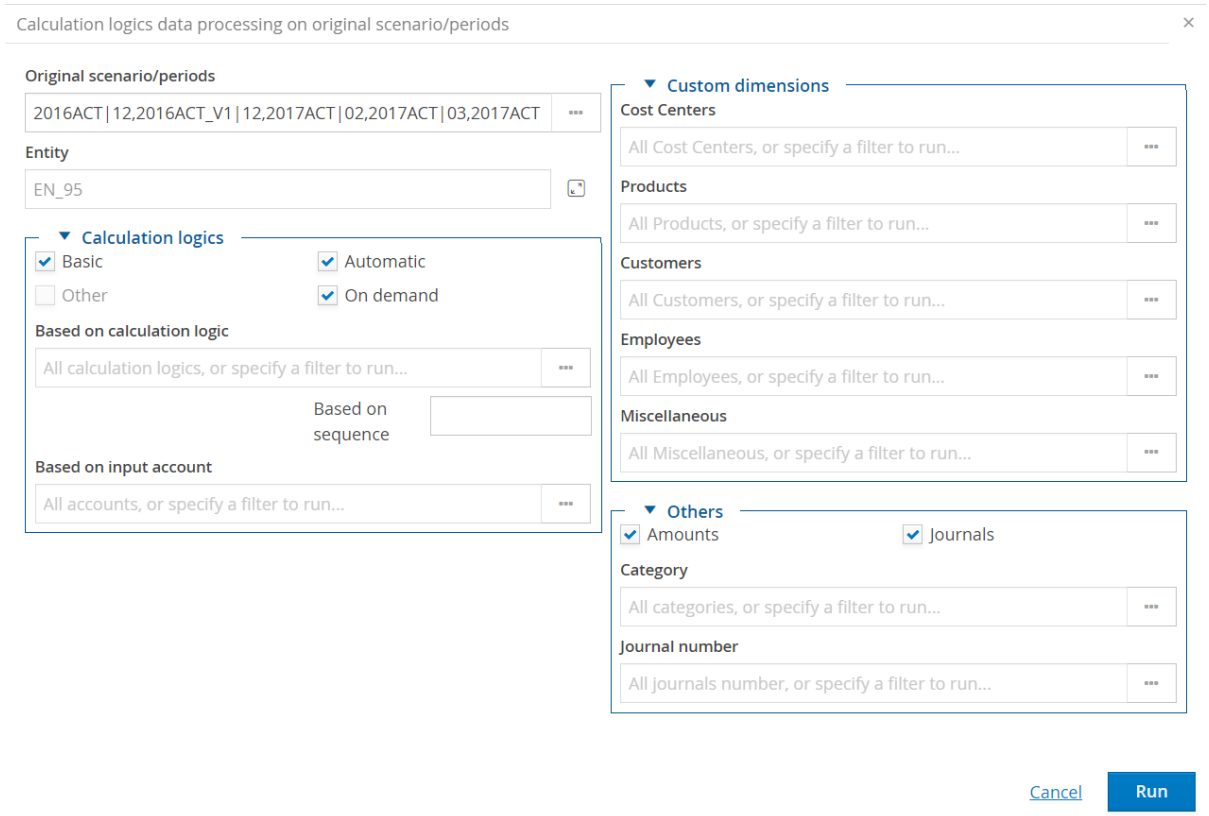
- **Filter 1.** If we specify to filter the calculation logics depending on data, the system processes all the calculation logics having as input accounts the accounts present on the data tables after applying the dimension filters and all the "indirect" ones, namely those having as input accounts the calculated accounts from the previous logics. In presence of this filter, the system ignores possible other filters of 2,3,4 type;
- **Filter 2.** In presence of a filter on accounts and /or on an accounts hierarchy's, the system processes all the calculation logics having as input accounts those obtained be the union of the single accounts and the accounts related to the specified nodes. Also in this case, the system process all the "indirect" logics having as input accounts the calculated accounts from the from the previous logics. In presence of this filter the system ignores the possible other filters of 3,4 type;
- **Filter 3.** In presence of a filter on the calculation logics and/or on the grouping's nodes, the system processes only the calculation logics which belong to the union of the single logics and the logics related to the nodes. This filter is applied in intersection with the filter on the sequence;
- **Filter 4.** In presence of a filter on the sequence the system processes all the calculation logics with that sequence.

Once you have specified the parameters, click on "Next" to specify a set of options telling the system how to run the data processing inside the form. The last step consists in assigning a code and a description to the data processing to be inserted into the form.

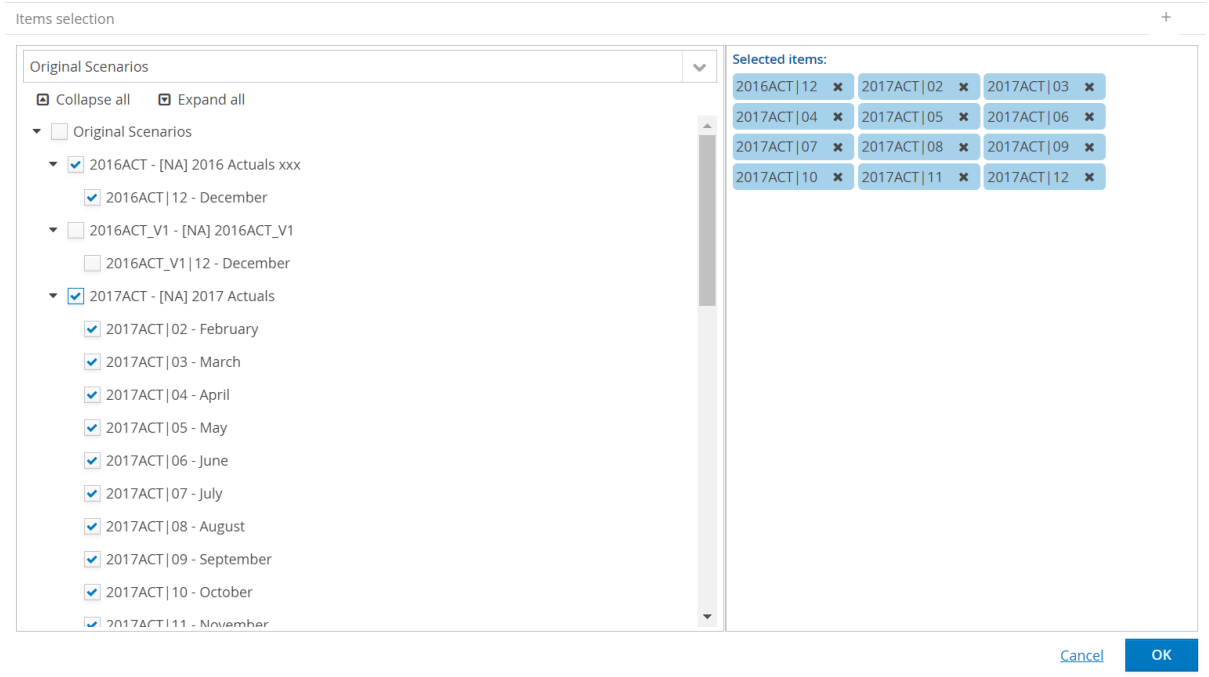
To run the data processing, just open the form in data entry mode, click on the **Data processing**  button in the Data entry ribbon bar and then click on the name of the data processing you want to run.

Processes cockpit - Contributor – Calculation logics

The **Calculation Logics** function allows you to run the calculation logics data processing for the selected process.



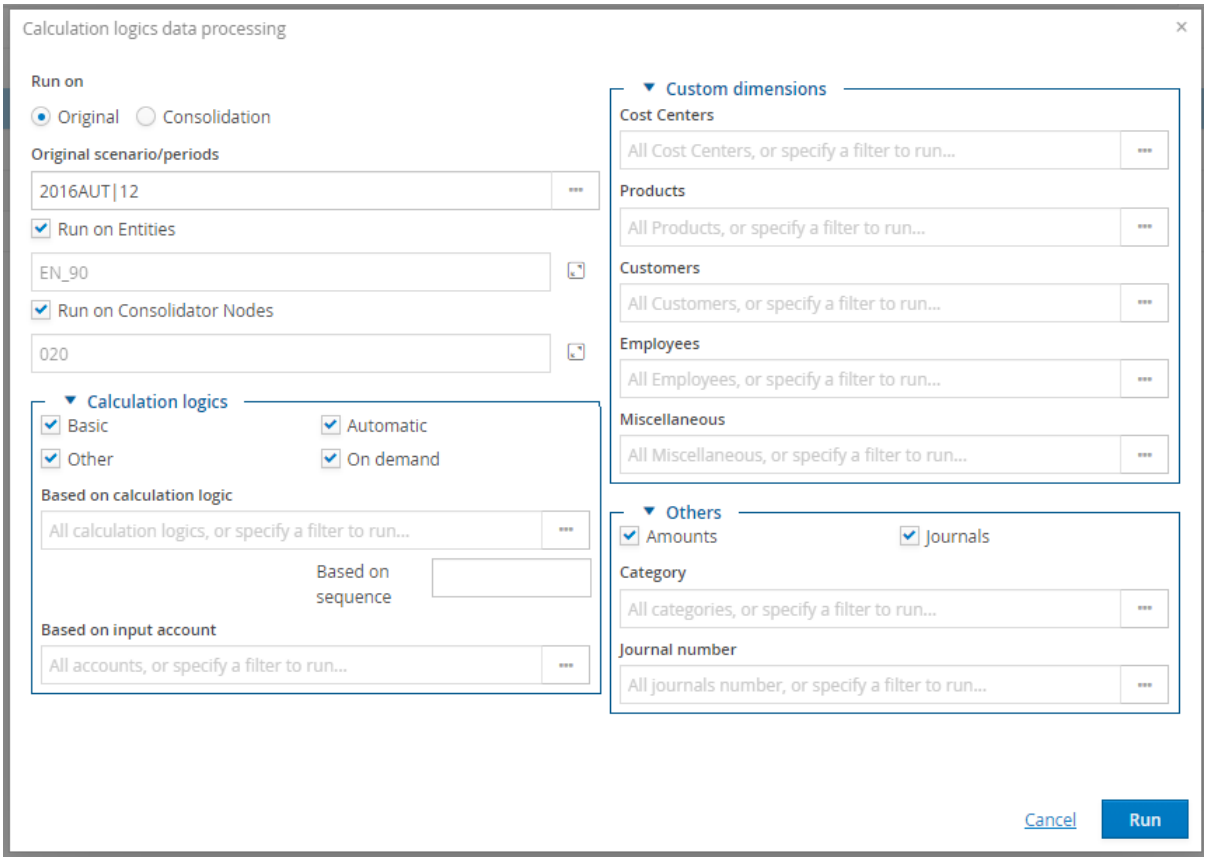
The user can run all calculation logics, or select and filter them based on the scenarios and periods linked to the process



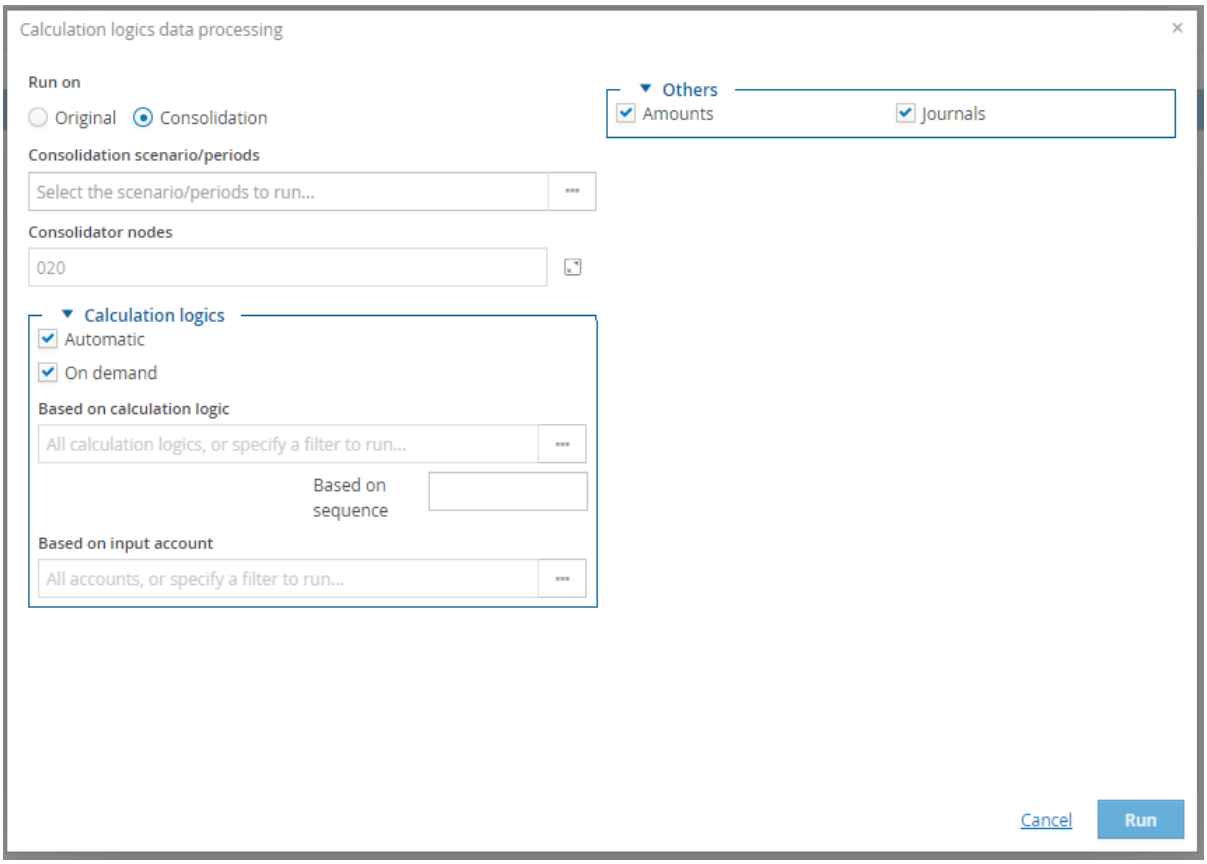
If the selection is not saved, then running the data processing or closing the popup resets the selection. Otherwise, it is possible to save sets of filters that a user can reuse later.

Processes cockpit - Consolidator – Calculation logics

The **Calculation Logics** function allows you to run the calculation logics data processing for the selected process, both for the original scenarios (Entity and/or Consolidator Nodes) and for the consolidation scenarios. You can access the option via "**Run on**".



For consolidation scenarios, the data processing will also be run on Entities that belong to the consolidation nodes selected from the cockpit.

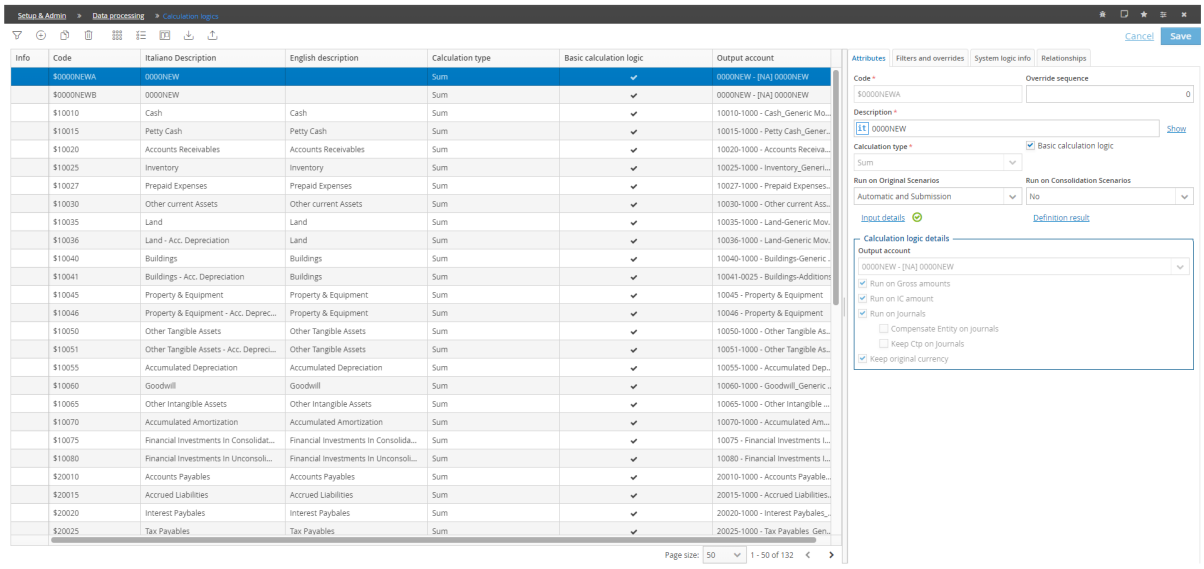

Calculation logics data processing
Run on
Original
Consolidation
Consolidation scenario/periods
Select the scenario/periods to run...
Consolidator nodes
020
Calculation logics
Automatic
On demand
Based on calculation logic
All calculation logics, or specify a filter to run...
Based on sequence
Based on input account
All accounts, or specify a filter to run...
Others
Amounts
Journals
Cancel
Run

Calculation logics of "Sum" type

A calculation logic of Sum type allows you to obtain an account (or calculated account) from the linear combination of other accounts (or input accounts).

Some dimensional specifications can be related to a logic of sum type in order to limit the set of data to be processes based on specific criteria or to assign a specific value for a specific dimension to the output data .

The calculation logics list can be accessed from the administrator interface, from *Data processing* > *Calculation logics* > *Elements*.



To add a calculation logic of Sum type, click on + and then define the following fields in the attributes tab:

- the **code**: 10010 (using a code equal to the code of the account to be calculated makes the calculation logic more explicative and allows you to immediately understand the task of that logic);
- the **description**: Gross intangible fixed assets;
- the **calculation type**: Sum;
- the **account to calculate**: 10010 - Intangible fixed assets;
- **enable** the "Run on journals" option.

After saving the header of the logic, you have to define the rows of the calculation logic, namely to define the accounts that have to be taken into account to determine the value of the account to calculate. To do that, click on the "Input details" link that popped up after saving the calculation logic in the attributes tab. When clicking on "Input details" the system displays a window in which you have to define the accounts for the calculation.

The definition can be done by account or by nodes of an accounts hierarchy. Moreover, you can specify a multiplicative coefficient (different for each account) to determine the calculation logic.

How the system deletes and saves the calculated data

Every time that a calculation logic of Sum type is run, the system saves the calculated data into the tables of the application DB as follows:

- it deletes any data on the target tables by filtering by:
  - calculation logic via the "notes" field;
  - origin (PROC_CALCULATED or PROC_CALCULATED_BASIC depending on whether the calculation logic is "Basic" type or "Other" type);

    - dimensions specified on the launch filter
- then it saves into the data tables the difference between the result of the sum and the existing data with input origin by filtering by output account, filtered dimensions or overridden dimensions specified in the calculation logic. Thus the system works in "compensation" mode.

Definition of a "Sum" calculation logic with dimensional specifications

Dimensional specifications can be defined in the Sum calculation logics

- **header**: from the *"Filter on Dimensions"* tab in the calculation logics window;
- **detail**: from a Sum calculation logic's input accounts definition window. In this case the specifications are defined for each input account.

Definition of a set of codes

Given a dimension, in order to define a set of elements, it is possible to use a sort of syntax to combine single elements or nodes of a hierarchy. The expression is given by a simple algebra. The atoms are:

- simple codes between square brackets (example: [C001])
- nodes of a hierarchy where, the hierarchy code is between square brackets and separated from the node's code by the symbol "|" (example: [$|N01])
- all the elements of a dimension (All)

The operators (that follow the usual order of precedence among operators) are:

- union <code set 1> + <code set 2> (example: [C001] + [$|N01])
- intersection <code set 1> * <code set 2> (example: [C001] * [$|N01])
- subtraction <code set 1> - <code set 2> (example: [G|S01] - [$|N01])
- brackets (example: ([C001] + [$|N01]) * [C002])

Filters

A filter allows you to "limit" the set of input data to run.

 The dimensions on which a filter can be applied are the category, the custom dimensions and the entity.

 In order to define a filter on a specific dimension, it is necessary to deselect the "override" field, in the "Filter on dimensions" tab of the calculation logics window. For further details, please see the following paragraphs about the management of overrides and reading from fixed point.

The filters can be defined at two levels:

- **header**: these filters can be defined by inserting a code set in the "Filter" field related to the chosen dimension. The filter will be applied to all the input accounts, limiting the reading of data to a specific subset of the filtered dimension. For example, a Sum calculation logic may read and write only on a subset of the cost centres.
- **single account**: these filters are specific for an input account or an accounts node. They can be defined from the detail window by inserting a code set into the "*Filter <dimension>*" column. The filter will limit the reading of an input account to a specific subset of the filtered dimension. For example, an input account among those that define the logic may have to be read only on a subset of the cost centres.

The definition of several filter levels (run filters, header filters, filters on single account) is interpreted on the input accounts as a filtering on the subset where the defined filters levels intersect. For example, when running a Sum calculation logic with a header filter on an A subset of the cost centres and then selecting a run filter on a B subset of the cost centres, the logic will be run on the A/B intersection subset.

Whereas, when running a Sum calculation logic with a header filter on an A subset of the cost centres and specifying for a single input account a filter on a B subset of the cost centres, such account will be read only on the A/B intersection subset.

The following examples show how to define a "Sum" type calculation logic with dimensional specifications.

Example 1

Let us assume:

- that the account 10110 is defined as the sum of the accounts 10110_010 and 10110_040;
- code set [D11]+[D13] has been defined as header filter on Custom Dim1;
- the filter has been defined on the custdim1 [D11] on the account 10110_010 ;
- the following set of input data is available:

Account	CustDim1	CustDim2	Amount	Origin
10110_010	D11	D21	150	INPUT_WEB
10010_010	D12	D22	80	INPUT_WEB
10110_010	D13	D22	65	INPUT_WEB
10010_040	D11	D21	30	INPUT_WEB
10010_040	D12	D21	10	INPUT_WEB
10010_040	D13	D22	50	INPUT_WEB

The system will read the following set of data:

Account	CustDim1	CustDim2	Amount	Origin
10110_010	D11	D21	150	INPUT_WEB
10010_040	D11	D21	30	INPUT_WEB
10010_040	D13	D22	50	INPUT_WEB

Following data will be written on the database after the data processing:

Acocunt	CustDim1	CustDim2	Amount	Origin
10110	D11	D21	150 + 30	PROC_CALCULATED
10010	D13	D22	50	PROC_CALCULATED

Override and reading from fixed point

Override

It enables you to define a specific value to be assigned to a given dimension when writing data. For example, you may need to attribute a specific category to the output account. The override can be defined from the "Override" field of the corresponding dimension.

 The dimensions on which it is possible to apply the override are the category and the custom dimensions.

 It is not possible to write data outside the run filter. If a Sum calculation logic tries to write outside the filter by defining an override, the system does not run the logic and records an error message on the audit.

 It is not possible to define the override on the category of a logic on which the "Run on journals" option is enabled.

Example 1

Let us assume that:

- the account 10110 is defined as the sum of the accounts 10110_010 and 10110_040;
- the element [D05] has been set as override on CustDim1;
- the following set of input data is available:

Account	Custdim1	Custdim2	Amount	Origin
10110_010	D11	D21	150	INPUT_WEB
10010_010	D12	D22	80	INPUT_WEB
10010_040	D11	D21	30	INPUT_WEB
10010_040	D12	D21	10	INPUT_WEB

Following data will be written on the database after the data processing:

Account	Custdim1	Custdim2	Amount	Origin
10110	D05	D21	150 + 30 + 10	PROC_CALCULATED
10010	D05	D22	80	PROC_CALCULATED

Reading from fixed point

If an override has been defined for a specific dimension, on such dimension it will be also possible to specify the subset of source elements.

The reading from fixed point can be defined:

- **on header**: it can be defined by inserting a code set in the "Filter" field related to the chosen dimension. The reading from fixed point will be applied to all the input accounts on which no specific subset has been defined, limiting the reading of data to a specific subset of the dimension. For example, you may read only on a subset of cost centres.
- **on account**: it can be defined from the detail window by inserting a code set into the " *<dimension> input*" column. This will limit the reading of an input account to a specific subset of the filtered dimension and it prevails over the one defined on header. For example, an input account may have to be read only on a subset of the cost centres.

Example 1

Let us assume that:

- the account 10110 is defined as the sum of the accounts 10110_010 and 10110_040;
- the code set [D05] has been defined as override on custdim1;
- the code set [D11]+[D13] has been defined as header reading from fixed point on custdim1;
- the reading from fixed point has been defined on [D12] cusdim1 on the account 10110_010;
- the following set of input data is available:

Account	Custdim1	Custdim2	Amount	Origin
10110_010	D11	D21	150	INPUT_WEB
10010_010	D12	D22	80	INPUT_WEB
10110_010	D13	D22	65	INPUT_WEB
10010_040	D11	D21	30	INPUT_WEB
10010_040	D12	D21	10	INPUT_WEB
10010_040	D13	D22	50	INPUT_WEB

the system will read the following set of data:

Account	Custdim1	Custdim2	Amount	Origin
10110_010	D12	D22	80	INPUT_WEB
10010_040	D11	D21	30	INPUT_WEB
10010_040	D13	D22	50	INPUT_WEB

Following data will be written on the database after the data processing:

Account	Custdim1	Custdim2	Amount	Origin
10110	D05	D21	30	PROC_CALCULATED
10010	D05	D22	80 + 50	PROC_CALCULATED

Calculation logics of "Multi-Dimensional Script" type

A Multi-dimensional calculation logic is a sophisticated script written using the TGKML Tagetik syntax derived from the MultiDimensional eXpressions (MDX) language for the OLAP analysis on SQL Server. As the MDX technology does, the TGKML scripts reads and write data applying the concept of "cube". A cube is a multi-dimensional representation of data. Each cube is defined by a set of dimensions which, in some cases, can be the same for different cubes.

With a single script the user can define several calculations which are processed by the system in the order they are in the script. The result of the script is saved into the Tagetik data tables.

 It is possible to set the synchronization level of Delete and Insert when running Multidimensional Script formulas in order to optimise the performance

The synchronization level must be set up in the general settings (*Setup & Admin > Data Model > Dimensions > Settings*) (*Navbar > Dimensions > Settings*) by opportunely setting the **MD engine synchronization level** option available in the **Advanced** section.

For further details, please see the BasicAdmin manual in Configuration section.

Cubes and dimensions

Each cube is identified by an acronym written between brackets and has a different meaning according as it is in the "input" or in the "output" part of the TGKML script. If it is:

- in the "input" part (input cube), it represents the "virtual" data table used to retrieve data from the database. Technically data are read from a view which puts together the records of one or several tables
- in the "output" part (output cube), it represents the physical tables in which the system saves the script's result. In a TGKML script only one output cube can be defined.

Dimensions

The Tagetik dimensions on which the cubes are based and which can be referred into the script are listed below:

- the scenario/period time dimension (**Time (Scenario,Period)**);
- the scenario (**Scenario**);
- the period (**Period**);
- the entity(**Entity**);
- the Ctp entity(**EntityCtp**);
- the Ctp custom dimension 2(**Dest2Ctp**);
- the category (**Category**);
- the custom dimensions (**DestN**);
- the journal number (**NumRett**);
- the Ctp entity for segment (**AziCtpSegment**);
- the Ctp custom dimension 2 for segment(**Dest2CtpSegment**);
- the currency (**Currency**);
- the currency 2 (**Currency2**).

Cubes

The cubes which can be managed in a script are listed below:

- **Cubes on original scenarios**.
  - **[LO]**. It represents the gross amounts plus the journals. The dimensions of these cube are:**Time (Scenario, Period), Entity, Category, Dest1, Dest2, Dest3, Dest4, Dest5, Currency**.
    - Data are read from the database view V_DATI_LORDI_RETT_RIGA which is the combination of the gross amounts tables (DATI_SALDI_LORDI) and the journals (DATI_RETT_RIGA) on child categories of the amount categories with not valued counterparty)
    - Data are written on the gross amounts table (DATI_SALDI_LORDI)
  - **[IC]**. I represents the intercompany amounts plus the journals. The dimensions of this cube are:**Time (Scenario, Period), Entity, Category, Dest1, Dest2, Dest3, Dest4, Dest5, Currency, EntityCtp, Dest2Ctp**.
    - Data are read form the database view V_DATI_IC_RETT_RIGA which is the combination of the IC amounts tables (DATI_SALDI_IC) and the adjustments (DATI_RETT_RIGA) on child categories of the amount categories with valued counterparty
    - Data are written into the IC amounts table (DATI_SALDI_IC)
  - **[RETT]**. Represents the entity and consolidation journals. The dimensions of this cube are: **Time (Scenario, Period), NumRett, Entity, Category, Dest1, Dest2, Dest3, Dest4, Dest5, Currency, EntityCtp, Dest2Ctp, AziCtpSegment, Dest2CtpSegment.**
    - Data are read and written from / into the DATI_RETT_RIGA table
  - **[SA]**. It represent the amounts in general. The dimensions of this cube are the same of the [IC] one but the counterparty dimension can be null:**Time (Scenario, Period), Entity, Category, Dest1, Dest2, Dest3, Dest4, Dest5, Currency, EntityCtp, Dest2Ctp.**
    - Read Data(gross and intercompany) are the combination of the [IC] and [LO] cubes and generates two sets: intercompany data (with counterparty) and the minorities quota (calculated by subtracting IC data from gross data)
    - Data with counterparty are written into the DATI_SALDI_IC table while the gross amounts (calculated by summing up the minorities and the intercompany) are written into the DATI_SALDI_LORDI table.

- **Cubes on consolidation scenarios**
  - **[SA_C]**. It represents the consolidable amounts. The dimensions of this cube are: **Time (Scenario, Period), Entity, Category, Dest1, Dest2, Dest3, Dest4, Dest5, Currency, EntityCtp, Dest2Ctp**.
    - Data are read / written from / into the DATI_SALDI_CONS table
  - **[RETT_C]**. It represents the consolidable journals the consolidable amount type. The dimensions of this cube are: **Time (Scenario, Period), NumRett, Entity, Category, Dest1, Dest2, Dest3, Dest4, Dest5, Currency, EntityCtp, Dest2Ctp, AziCtpSegment, Dest2CtpSegment**.
    - Data are read / written from / into the DATI_RETT_RIGA_CONS table
  - **[SA_RETT_C]**. It represents the amounts and consolidable journals. The dimensions of this cube are:**Time (Scenario, Period), Entity, Category, Dest1, Dest2, Dest3, Dest4, Dest5, Currency, EntityCtp, Dest2Ctp**.
    - Data are read from the DATI_SALDI_CONS e DATI_RETT_RIGA_CONS tables
    - Data are written into the amounts table (DATI_SALDI_CONS)

Amounts and currencies

Usually in the Tagetik data tables there are two different currencies and two amounts. All the available cubes use only one currency. The currency used for each data table is shown below:

- DATI_SALDI_LORDI: original currency
- DATI_SALDI_IC: Invoice currency
- DATI_RETT_RIGA: currency 2
- DATI_SALDI_CONS: currency
- DATI_RETT_RIGA_CONS: currency

When data are saved to the database, the system calculates "on the fly" the currency that is not used as dimension (thus that is not read from the input tables) in the cube according to the following rules:

- DATI_SALDI_LORDI, DATI_SALDI_IC: Entity Currency in the process
- DATI_RETT_RIGA: Journal currency
- DATI_SALDI_CONS, DATI_RETT_RIGA_CONS: Consolidation scenario currency.

Even though only one currency can be used as dimension in a cube the system, during the calculation step, uses both the amounts in the data tables. These amounts are:

- DATI_SALDI_LORDI.
  - Amount 1: original currency amount
  - Amount 2: Amount
- DATI_SALDI_IC.
  - Amount1: invoice currency amount
  - Amount 2: Amount
- DATI_RETT_RIGA.
  - Amount 1: Amount 2
  - Amount 2: Amount
- DATI_SALDI_CONS.
  - Amount 1: Amount
  - Amount 2: invoice currency amount
- DATI_RETT_RIGA_CONS.
  - Amount 1: Amount
  - Amount 2: Amount 2

The system calculates in parallel both the amounts. The only case when the system uses one amount only (amount 1) is in presence of a Boolean condition in the IF statement.

How does the system delete and store calculated data

When a MD calculation logic is run, in order to store calculated data to the application database tables the system performs the following steps:

- deletes data from the destination tables, filtering by:
  - output account, namely the account calculated by using the TGKML script
  - origin (PROC_CALCULATED or PROC_CALCULATED_BASIC depending whether the calculation logic is of "Base" or "Other" type);
  - filtered or overridden dimensions
- stores in the data tables the difference between the result of the TGKML script and already present data with origin different form PROC_CALCULATED or PROC_CALCULATED_BASIC filtering by output account, filtered or overridden dimensions. Thus it works in "compensation" mode. Let's see some examples to explain this concept.

EXAMPLE 1: Let us assume that we have a script which calculates the value of the account X00000, that the calculated value is 150 and that data on the database before running the script are the following:

Account	Amount	Origin
X00000	10	PROC_CALCULATED
X00000	80	INPUT_DEFORM

After the deletion, data on the database are the following:

Account	Amount	Origin
X00000	80	INPUT_DEFORM

Being the calculated value 150, data after the storing are the following:

Account	Amount	Origin
X00000	70	PROC_CALCULATED
X00000	80	INPUT_DEFORM

ForceDelete

When it is necessary to force a precise deletion even when a global deletion is planned, it is possible to use the ForceDelete syntax as output specifications for journal statement on physical accounts

For example, you can write:

[LO].[X00000](Entity.Filter([A00]), ForceDelete) := ...

Structure and definition of a TGKML script

A TGKML script:

- always starts with the keyword**"TGKML 1.0**" ;
- may contain one or more assignment instructions. Each assignment ends with the character **';'**;
- may contain one or more comments identified by the **//** symbol placed at the beginning of the line. Any character following the // symbol to the end of the line is ignored. Comments can be inserted at any point in the script.

TGKML 1.0

/ / Comment 1

/ / Comment 2

Assignment 1;

/ / Note 3

Assignment 2;

Assignment 3;

...

Assignment No;

An assignment can represent:

- a **syntactic substitution**;
- a **physical account**;
- a **temporary account**
- an item in a Financial Statement Template.

In general, the syntax of a TGKML script is always of the type:

<output statement> := <input statement>;

The left **Output** part of the assignment contains the indication of where to save the result of the TGKML script and its syntax is of the type [output_cube].[output_account] (Dimension.Filter([...]) , Dest#.[...] , ... , ...). For example, we can have [LO].[X00000](Entity.Filter([A00]), Dest1.[GEN]) to indicate that the result of the scritp is saved on the original gross amounts table (DATA_SALDI_LORDI), on the account X00000 , on the Entity A00 and on the element GEN1 of custom dimension 1. All other necessary dimensions are taken from the right-hand side Input of the assignment.

The right-hand side **Input** of the assignment basically contains the calculation algorithm and its syntax is of the type [input_cube].[input_account] (Dimension.Filter([...]) , Dest#.[...] , ... , ...). For example, we can have [LO].[C00000](Entity.Filter([A00]), Dest1.[GEN]).

Obviously, the above syntax is typical of the simplest situations, as it is possible to use several accounts (both physical and temporary) combined with algebraic or Boolean operators.

In the input specification, it is possible to refer not only to accounts or nodes of accounts, but also to 'Parent' items belonging to Financial Statement Templates that depend only on the 'Account' dimension.

 In the right Input part, the indication of the cube can be omitted if it is the same as the one specified in the left Output part.

Within the syntax of a TGKML script, it is possible to use:

- **comparison operators** <, >, <=, >=, <>, ==
- **boolean expressions** such as
  - Scenario like '%BDG'. Note that strings must appear between double quotes
  - Period == "12"
  - Cust Dim2 <> "GEN"
- **arithmetic operators** +, -, *, /, ^
- **numbers**. These must be written using the dot as a separator between the integer and decimal parts. It is possible to use exponential notation i.e. write 1e4 instead of 10000

Here are the conventions to be used to specify a single element of a dimension or a node of a hierarchical structure within the syntax:

- [element **code]** to indicate a single element of one dimension (e.g. [A00]);
- **[hierarchy code|node code]** to indicate a node of an aggregation structure (e.g. [01|$]);
- **All** to indicate all elements of a given dimension (e.g. Cust Dim1. All to indicate all elements of custom dimension 1).
- **{<FST>,<item>}**, **{<FST>, '<item>'}**, **{<FST>,'<item>'}**; **{<FST>,[<item>]}**: to indicate an item in a financial statement template
- There are two ways to define the input/output specification of an FST item:
  - Multi ({<FST>, <item>}, <specs>)
    e.g. Multi({<FST>, <item>}, Dimension.Filter([...])
  - {<FST>, <item>, <specs>}
    e.g. {<FST>, <item>, Dimension.Filter([...])}

Where the code of an FST item contains the '-' character (e.g. ITEM-001), it is **mandatory to** use the syntax {<fst>,[<item>]} (e.g. {001,[ITEM-001]})

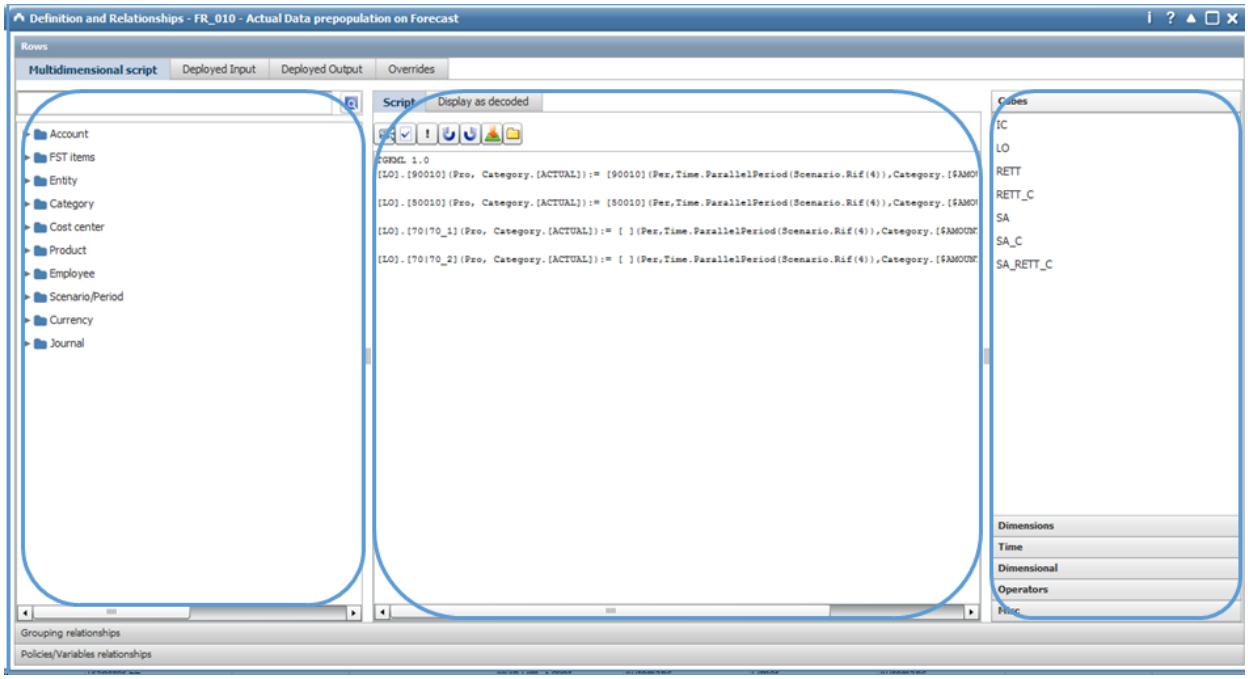
Within the Input part, it is possible to combine nodes and single elements of a dimension (e.g. write [01|$], [A00])

Definition of a TGKML script

To define a TGKML script, it is necessary to:

- create a 'Multi Dim. Script' type calculation logic (for further details see Calculation logics);
- double-click on the newly created logic.

The system opens the **Multidimensional Script** window.



The window is divided into three parts:

- **(1) - Selection area**. It is located on the left-hand side of the window and is intended for the selection (by double-clicking) of elements and/or nodes to be inserted within the script. In this area, the system displays all existing elements for a given dimension. In particular, the system displays a sort of tree whose nodes are:
  - a first dummy node 'Elements' containing all the elements of the dimension present in the elements list sorted by code;
  - as many nodes as there are hierarchies defined for the dimension, if the dimension includes hierarchy type aggregation structures, otherwise a single Grouping node. Under each of these nodes, the system replicates the structure of the hierarchy and/or grouping.

    The various nodes are identified by the icon , and the single elements by .

    - When the number of elements to be displayed is very high (generally more than 50), the system, in order to improve the display and selection of elements, creates dummy nodes identified by the icon . Such nodes cannot be used in the definition step;
    - Selection can be made by navigating the tree in the selection area or by using the text search functionality on codes and/or descriptions of elements/nodes. The search can be performed on a given tree node using the icon (except for dummy nodes used by the system for paging tree elements). The search result is displayed in a grid from which one or more items can be selected.
- **(1) - Script**. It contains the area in which the syntax of the TGKML script is to be entered and a series of buttons that allow you to:
  - insert a TGKML script by importing it from a text file;
  - save the script to a text file on the user's computer;
  - restore the previous situation in case of an error;
  - repeat the previous action;
  - perform a check to verify the correctness of the script syntax. The system displays the result of this check at the bottom;
  - save the script and at the same time generate the deployed tables (for more details on the content of these tables, see **Calculation Logic Deployment**).

  This area includes an additional **'Display as decoded'** tab that displays the script alongside each dimensional element with its description



- **(3)** Contains all dimensions / functions / cubes / objects that can be used within a TGKML script.

Filters

The concept of **filter** is very important for designing a TGKML, script since it allows to **"limit"**:

- the set of input data;
- data to delete;
- data to save to the output cube

The only mandatory filter in the syntax of a TGKML script is the filter on the Time dimension.

The possible filters are:

- **prompt filters**. These are applied to all the MD calculation logics run from the application's web interface, from the data entry form, from the workflow model related to an application process, from the process cockpit etc. A typical example is the run of a set of MD calculation logics only for the A00 Entity;
- **filters on account dimension**. These are applied to the single input account present in the assignment of a given TGKML script;
- **filters on an assignment**. These are applied to a single "assignment" part of the TGKML script and can be considered as an additional restriction to the prompt filters. For example an assignment might read and write only from/on a subset of cost centers (managed on the custom dimension 1).

The syntax to use to define an input or output filter is the following:

Dimension.Filter([...])

The syntax to use in the case of filter on FST item (input) is:

{<FST>,<item>,**Dimension.Filter([...])**}

For example it is possible to write

- Entity.Filter([A00]) to filter on the entity A00;
- Entity.Filter([A00],[A01]) to filter on the entity A00 and on the entity A01;
- Entity.Filter([01|$]) to filter on all the entities related to the node $ of the hierarchy 01.

Input filters

Let's see some examples of input filters definition.

Example 1

Let us assume that we have the following set of input data:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	150
A00	X00000	2013BDG	$AMOUNT	D11	D21	80
A00	X00000	2013BDG	$AMOUNT	D11	D22	30

and to define the input part of the script as follows:

... := [X00000](Dest1.Filter([D11]), Dest2.Filter([D22]))

Then the system reads the following set of data:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount
A00	X00000	2013BDG	$AMOUNT	D11	D22	30

Example 2

Let us assume that we have the following set of input data:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount	Origin
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	150	PROC_CALCULATED
A01	X00000	2013BDG	$AMOUNT	D11	D21	80	PROC_CALCULATED
A02	X00000	2013BDG	$AMOUNT	D11	D22	30	PROC_CALCULATED
C00	X00000	2013BDG	$AMOUNT	D11	D22	30	PROC_CALCULATED

and to define the input part of the script as follows:

... := [LO].[X00000](Entity.Filter([A00],[A01],[A02]))

and that we have defined, at the script runtime, a prompt filter on the elements A00, A01 of the entity dimension.

Thus the system reads the following set of data:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount	Origin
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	150	PROC_CALCULATED
A01	X00000	2013BDG	$AMOUNT	D11	D21	80	PROC_CALCULATED

Output filters

Let's see some examples of output filters definition.

Example1

Let us assume that we have the following syntax

[LO].[X00000](Dest1.Filter([D11], [01|$])) :=...

in this case the system deletes and then saves the original gross amounts related to the account X00000 on the element D11 of the custom dimension 1 and on all the elements of the custom dimension 1 related to the node $ of the hierarchy 01.

On the other hand, if we have

[LO].[X00000](Entity.Filter([A00])) :=...

the system deletes and then saves original gross amounts related to the account X00000 only for the Entity A00.

Example 2

Let us assume that:

- we have defined the Output part of the script as follows [LO].[X00000](Entity.Filter([A00],[A01],[A02])) := ...
- we have a prompt filter on the elements A00, A01,A02 of the entity dimension;
- we have the following set of data before the deletion:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount	Origin
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	150	PROC_CALCULATED
A01	X00000	2013BDG	$AMOUNT	D11	D21	80	PROC_CALCULATED
A02	X00000	2013BDG	$AMOUNT	D11	D22	30	PROC_CALCULATED
C00	X00000	2013BDG	$AMOUNT	D11	D22	30	PROC_CALCULATED

The system deletes and saved the gross amounts for the account X00000 and the entities A00,A01,A02.

After the deletion, following data are on the database:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount	Origin
C00	X00000	2013BDG	$AMOUNT	D11	D22	30	PROC_CALCULATED

Example 3

Let us assume that:

- we have defined the Output part of the script as follows [LO].[X00000](Entity.Filter([A00],[A01],[A02])) := ...
- we have a prompt filter on the element A00 of the entity dimension;
- we have always the same set of data before the deletion of the former example

The system deletes and saves gross data for the account X00000 and the entity A00.

Following data are present after the deletion:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount	Origin
A01	X00000	2013BDG	$AMOUNT	D11	D21	80	PROC_CALCULATED
A02	X00000	2013BDG	$AMOUNT	D11	D22	30	PROC_CALCULATED
C00	X00000	2013BDG	$AMOUNT	D11	D22	30	PROC_CALCULATED

Example 4

Let us assume that:

- we have defined the following script [LO].[X00000] := [C00000](Entity.Filter([A00]));
- we have a prompt filter on the elements A00, A01,A02 of the entity dimension;
- that the value calculated by the script for the entity A00 is equal to 50;

Moreover, let us assume that data on the database before running the script are the following:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount	Origin
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	150	INPUT_DEFORM
A01	X00000	2013BDG	$AMOUNT	D11	D21	80	INPUT_DEFORM
A02	X00000	2013BDG	$AMOUNT	D11	D22	30	INPUT_DEFORM
C00	X00000	2013BDG	$AMOUNT	D11	D22	30	INPUT_DEFORM

Data present after the script run are the following:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount	Origin
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	150	INPUT_DEFORM
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	-100	PROC_CALCULATED
A01	X00000	2013BDG	$AMOUNT	D11	D21	80	INPUT_DEFORM
A01	X00000	2013BDG	$AMOUNT	D11	D21	-80	PROC_CALCULATED
A02	X00000	2013BDG	$AMOUNT	D11	D22	30	INPUT_DEFORM
A02	X00000	2013BDG	$AMOUNT	D11	D22	-30	PROC_CALCULATED

 When a filter is applied only to the input part of the script, depending on how the prompt filters are set, the system might reset data which are out of the filter scope (resulting from the intersection of the prompt filters with the filters of the single assignment).

Reading from fixed point

In some cases it is necessary to define a TGKML script where a given amount has to be read from a dimension's specific element; for example the products' price might be saved to a specific XY element of the custom dimension 2.

This requires the concept of **reading from fixed point** which allows to specify the element from which a given amount has to be read.

The syntax to use for defining the reading from fixed point is:

Dimension.[...]

The syntax to use in case of reading from fixed point on FST item (input) is:

{<FST>,<item>,**Dimension.[...]**}

For example it possible to write

- [20132](Dest2.[XY]) to specify that the value of the account 20132 is read only from the XY element of the custom dimension2;
- [20132](Time.[2009|12]) to specify that the value of the account 20132 is read only from the 2009 / 12 scenario / period;
- [20132](Dest2.[01|XY]) to specify that the value to read for the account 20132 is the sum of all the custom dimension 2 elements which are related to the sub-tree of the hierarchy 01 whose root is the XY node;
- [20132](Dest2.Null) to specify that the value to read for the account 20132 is the sum of all elements with custom dimension 2 equal to null;
- [20132](Dest2.NotNull) to specify that the value to read for the account 20132 is the sum of all element with custom dimension 2 different from null.

 When the reading from fixed point function is used, the system "removes" the X dimension (with constant value) from the input data set. Namely, if we assume we have the following input data set:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	150

and that we define the Input part of the script as follows:

... := [X00000](Dest1.[GEN], Dest2.[GEN])

Thus the system reads the following set of data:

Entity	Account	Scenario	Category	Amount
A00	X00000	2013BDG	$AMOUNT	150

This does not happen if we use an input filter. Namely, if we assume we define the Input part of the script as follows:

... := [X00000](Dest1.Filter([GEN]), Dest2.Filter([GEN]))

Thus the system reads the following set of data:

Entity	Account	Scenario	Category	Dest1	Dest2	Amount
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	150

In this way, if we assume we have

[LO].[D00000]:= [X00000](Dest1.[GEN], Dest2.[GEN]) * [C00000]

where [C00000] has on the database the following values

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	01	10
A00	C00000	2013BDG	$AMOUNT	D11	D22	01	20
A00	C00000	2013BDG	$AMOUNT	D12	D22	01	5

the result is:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	D00000	2013BDG	$AMOUNT	GEN	GEN	01	150*10
A00	D00000	2013BDG	$AMOUNT	D11	D22	01	150*20
A00	D00000	2013BDG	$AMOUNT	D12	D22	01	150*5

while, in presence of an input filter

[LO].[D00000]:= [X00000](Dest1.Filter([GEN]), Dest2.Filter([GEN])) * [C00000]

the result is:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	D00000	2013BDG	$AMOUNT	GEN	GEN	01	150*10

Overrides

In some cases it is necessary to define a TGKML script where the output account has a specific value on a given dimension. This requires the concept of **override** which allows to specify a fixed value (also NULL) to assign to a given dimension. In this way the system firstly deletes data on the database filtering by this overridden value and then saves data setting the dimension where the override has been defined with the fixed value specified in the output part of the script.

 The scenario and the period are the only dimensions where an override cannot be defined.

Tagetik allows to

- **override a dimension's value**. [LO].[X00000](Dest2.[D11]) :=...

  In this case the system writes the script's result on gross data of the account X00000 and of the element D11 of the custom dimension 2;

- **Override a dimension's value to null**. [LO].[X00000](Dest2.Null) :=...

  In this case the system writes the result of the script on gross data of the account X00000 and of the element <null> of the custom dimension 2;

- **override the value on a hierarchy's node**. [LO].[X00000](Dest2.[01|$]) :=...

  In this case the system writes the script's result on gross data of the account X00000 and on all the elements of the custom dimension 2 which are related to the overridden node, namely the node $ of the hierarchy whose code is 01;

- **override the same element defined in the input filters.** [LO].[X00000](Entity.ForceOnFilter) :=...
- **define an override using** the **Ancestor** and **Parent** utilities.
  - [LO].[X00000](Dest2.Parent([01]) :=...

    In this case the system, after calculating the input element by element, sums up the results of the elements related to the same parent node in the hierarchy 01 and writes the result on the reference element defined for such node

  - [LO].[X00000](Dest2.Ancestor([01],3) :=...

    In this case the system, after calculating the input element by element, sums up the results of the elements related to the same parent node of level 3 in the hierarchy 01 and writes the result on the reference element defined for such node.

Physical and temporary accounts

When it is necessary to define a very complex TGKML script, subdividing its definition in several steps is considered to be a best practice. This "Divide et Impera" approach is realized using the temporary accounts. The temporary account, unlike the **physical account**, present in the Tagetik accounts table, is available only in the script in which it is defined.

The syntax to use for defining a temporary account is the following:

<output cube>.@<name to assign to the temporary account> := <expression>;

For instance, it is possible to write

- [LO].@TEMP_ACCOUNT := [C00000](Entity.Filter([A00]));

After defining a temporary account , it can be used with the following syntax

@<name assigned to the temporary account>

for example:

....:=@TEMP_ACCOUNT;

 When in a TGKML script it is necessary to refer several times the same physical account, it is recommended to create a temporary account. Indeed, in this way the script is more efficient.

 Generally, a temporary account does not have to be used in a script applying a temporary shift such as

[LO].@TEMP_ACCOUNT := [C00000](Entity.Filter([A00]));

... := @TEMP_ACCOUNT(Time.Scenario.Prev);

since the account might not exist on the previous scenario.

 In a TGKML script, wherever an integer parameter has to be inserted in a function, it is possible to use a temporary account. The system will read the amount of the "first record" of this account by rounding it to the closest integer value and it will use it as a parameter for the function. Which is the "first record" of a temporary account is not predictable, so the correct way to use this feature is to define a single record temporary account. Please note that, in this case, the system uploads only the information about the amount of the first record and not the information of the entire record. This is why the parameter should be considered non-dimensional, not with the same dimension as source data.

Syntactical substitution

To simplify the TGKML script reading and maintenance , it is possible to use **syntactical substitutions**, namely to define a sort of "alias" of a given expression. Once a syntactical substitution is defined, it can be called several times in the script by using the operator "@".

The syntax to use for defining a syntactical substitution is the following:

<name to assign to the syntactical substitution> : {list of elements separated by comma}

For example, it is possible to write

- MyDest2Filter : {Dest2.Filter([XY], [01|03])};
- MyFilter : {Dest1.Filter ([D11],[D12]), Entity.Filter([A00]), Dest2.[GEN]};
- NA : {Entity.[NA], Dest1.[NA], Dest2.[NA], Dest3.[NA],Dest4.[NA]};

 Generally, the syntactical substitutions definition is performed on the top of the script so as that they can be found easily.

Once a syntactical substitution is defined, it can be called using as syntax

@<name assigned to the syntactical substitution>

for example:

....:=[C00000](@MyFilter);
[LO].[X00000](@NA) := ... ;

SET expression

To simplify the definition of a given dimension's elements set and to allow to reuse this set several times in a TGKML script without defining the set each time from scratch, it is possible to use the SET expression, namely to define a sort of "alias" of a given dimension's elements set. Once the elements set is defined, the SET expression allows to call it several times in the script using the operator "@".

The syntax to use is the following:

<name_dimension>.@<name to assign to the set> := {elements list}

For example, it is possible to write

- Dest1.@MY_SET_1 := [D11] + [D12] + [GEN];

  In this case the defined set is MY_SET_1 which is made up of the D11, D12 and GEN elements of the custom dimension 1;

- Entity.@MY_SET_2 := [01|$];

  In this case the defined set is MY_SET_2 which is made up of all the entities related to the node $ of the entity hierarchy 01;

- Dest2.@MY_SET_3 := All;

  In this case the defined set is MY_SET_3 which is made up of all the elements of the custom dimension 2 table.

For the account dimension, it is possible to define a set of elements by referring to a FST item. The syntax to use is: {<fst>,<item>} . It is possible to use only Parent items of financial statement templates that depend only on the "Account" dimension.

For example, it is possible to write

- Account.@MY_SET_3 := ([C00000] + {S01,V0001});

In this case, the defined set is MY_SET_3 which is made up of the account C00000 and all the accounts related to the FST item V0001 of the financial statement template S01;

- [LO].@MY_SET_4:= Multi([X00000] + {S01,V0001} + {S02,V0003});

In this case the defined set is MY_SET_4 which is made up of the account X00000, and all the accounts related to the FST item V0001 of the financial statement template S01 and all the accounts related to the FST item V0003 of the FST S02;

When defining the elements list it is possible to use the following algebraic operators:

- **+.** elements union. For example, writing Entity.@MY_SET_4 := @MY_SET_2 + [A01]; means defining a set of entities made up of the entities of the set MY_SET_2 plus the A01 entity.
- **-**. Eliminate some elements from a list;
- ***.** Elements intersection

Once a set of elements is defined, by using the SET expression it is possible to call it using the following syntax

@<name assigned to the set>

The SET expression can be used also

- **in the reading from fixed point.**

  Dest1.@MY_SET_1 := [D11] + [D12] + [GEN]; //definition

  ... := [C00000](Dest1. @MY_SET_1);

to specify that the value of the account C00000 is read only from the elements D11, D12 and GEN of the custom dimension 1;

- **in the overridden values definition.**

  Dest1.@MY_SET_1 := [D11] + [D12] + [GEN]; //definition

  [C00000](Dest1.@MY_SET_1) :=...;

  to specify that the script's result is written on the account X00000 and on the elements D11, D12 and GEN of the custom dimension 1;

- **in the filters definition.**
- **in the definition of temporary accounts existing only for some determined elements** (those belonging to the set defined by the SET expression)

  Dest1.@MY_SET_5(@TEMP_ACCOUNT,15) := [D11] + [D12] + [GEN]; // 15 is the value of the temporary account for the elements D11,D12 and GEN of the custom dimension 1. The result of this assignment is the following

Account	Dest1	Amount
@TEMP_ACCOUNT	D11	15
@TEMP_ACCOUNT	D12	15
@TEMP_ACCOUNT	GEN	15

- **for the definition of more complex elements sets.**

  Dest1.@MY_SET_6(@TEMP_ACCOUNT_1,5) := [D11] + [D12]; // 5 is the value of the temporary account for the elements D11 and D12 of the custom dimension 1. The result of this assignment is the following

Account	Dest1	Amount
@TEMP_ACCOUNT_1	D11	5
@TEMP_ACCOUNT_1	D12	5

Dest2.@MY_SET_7(@TEMP_ACCOUNT_2,3) := [D21] + [D22]; // 3 is the value of the temporary account for the elements D21 and D22 of the custom dimension 2. The result of this assignment is the following:

Account	Dest1	Amount
@TEMP_ACCOUNT_2	D21	3
@TEMP_ACCOUNT_2	D22	3

@TEMP_RESULT := @TEMP_ACCOUNT_1 * @TEMP_ACCOUNT_2

Account	Dest1	Dest2	Amount
@TEMP_RESULT	D11	D21	5*3=15
@TEMP_RESULT	D11	D22	5*3=15
@TEMP_RESULT	D12	D21	5*3=15
@TEMP_RESULT	D12	D22	5*3=15

IF Construct

The IF instruction allows performing different calculations according to a logical condition. In this case, the syntax to use is

... := if <logical_condition> then <then_expression> else <else_expression> end;

For example, we can write:

[LO].[X00000] := if [C00000] == 0 then [D00000] else [E00000] end;

Generally, the result of the IF instruction is:

- <then_expression>when the <logical_condition> is true;
- <else_expression> when the <logical_condition> is false.

Technically, in order to evaluate the IF expression, the system runs the following operation

<logical_condition> *<then_expression> + <logical_condition>*<else_expression>

that, in some cases, may give wrong results. For further details, please see **Advanced usage of the BOUND function**.


The syntax <logical_expression> *<account> is an alternative way to get all combinations between <account> and the values for which the <logical_expression> is true.


The IF instruction can be nested and the condition ELSE can be omitted.

Example 1

[LO].@CONDITION := [C00000] == 10;

[LO].[X00000] := @CONDITION * [D00000](Dest1.[D16]);

Additionally, let us assume the following data are on the database.

ACCOUNT C00000

Entity	Account	Scenario	Category	CustDim1	CustDim2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	D11	D21	01	10
A00	C00000	2013BDG	$AMOUNT	D12	D21	01	10
A00	C00000	2013BDG	$AMOUNT	D13	D21	01	30
A00	C00000	2013BDG	$AMOUNT	D14	D21	01	50
A00	C00000	2013BDG	$AMOUNT	D15	D21	01	10

ACCOUNT D00000

Entity	Account	Scenario	Category	CustDim1	CustDim2	Period	Amount
A00	D00000	2013BDG	$AMOUNT	D16	D21	01	20

When evaluating @CONDITION, the system generates as many records as existing records for the account C00000 by assigning the value 1 where the condition is verified, otherwise 0. We will get the following values:

Entity	CustDim1	CustDim2	Amount
A00	D11	D21	1
A00	D12	D21	1
A00	D13	D21	0
A00	D14	D21	0
A00	D15	D21	1

The result of our script will be:

Entity	Account	Scenario	Category	CustDim1	CustDim2	Period	Amount
A00	X00000	2013BDG	$AMOUNT	D11	D21	01	20*1
A00	X00000	2013BDG	$AMOUNT	D12	D21	01	20 *1
A00	X00000	2013BDG	$AMOUNT	D15	D21	01	20*1

Example 2

[LO].@CONDITION := Dest1 <> “xxx”;

[LO].[X00000] := @CONDITION * [D00000](Dest1.[GEN]);

This is another way to use the IF condition in order to deploy an account on a given dimension (in this case, the custom dimension 1). the system evaluates the @CONDITION by creating a list of records for each element of the custom dimension 1 with code different from xxx. If the record with code xxx doesn't exist, it means that we shall open as may records with amount 1 as elements of the custom dimension 1.

Monthly and cumulated values

A THKML script can read and store both monthly and cumulated values. Let's see in detail the syntaxes to use:

- **Per** - Monthly data;
- **Pro** - Cumulated data;
- **Per(N)** with N integer - Custom period length. This syntax allows to read data from a scenario which is not the current one. For example if we use the syntax Per(24) from the period 06 of the scenario N the system calculates periodical data as follows: Scenario N | Period 06 (cumulated) + Scenario N-1 | Period 12 (cumulated) + Scenario N-2 | Period 12 (cumulated) - Scenario N-2 | Period 06 (cumulated).

Example: [LO].[X00000](Pro,Dest1.Filter([D11])) := [C00000](Per);


Inserting the period length information before the filters or the reading from fixed point is considered to be a best practice.

Data reading

By default, the system reads data as they are stored in the database, namely it read cumulated data. If the user needs to read monthly data he has to use the **Per** syntax as shown below.

Let us assume that data on the database are the following:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	01	50
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	02	80
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	03	110
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	04	125
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	05	160
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	06	215

and that we read periodical data

... := [C00000](Per);

Read data are the following:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	01	50
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	02	30
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	03	30
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	04	15
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	05	35
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	06	55

 Periodical data makes sense only for P&L or Other Variations account types.

Data storing

By default the system stores the calculated value for a given period only on that period. If the user needs to store cumulated data he has to use the Pro syntax as shown below.

Let us assume that data on the database are the following:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	01	50
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	02	80
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	03	110
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	04	125
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	05	160
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	06	215

and that we have the following script [LO].[X00000](Pro) := [C00000](Per);

Read data are the following:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	01	50
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	02	30
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	03	30
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	04	15
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	05	35
A00	C00000	2013BDG	$AMOUNT	GEN	GEN	06	55

Data stored on the database are the following:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	01	50
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	02	50+30
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	03	50+30+30
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	04	50+30+30+15
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	05	50+30+30+15+35
A00	X00000	2013BDG	$AMOUNT	GEN	GEN	06	50+30+30+15+35+55

Sum operator

When two accounts have to be summed up, two cases have to be distinguished:

- the dimensions that identify the value of the two accounts are homogeneous, namely the accounts operate on the same number of dimensions; in this case we have **homogeneous operands**. For example, if we assume to mange the custom dimensions 1 and 2 we can have [A] + [B]. In this case, the two operands are homogeneous since both the value of the A account and the value of the B one are read taking into consideration all the elements of the custom dimensions 1 and 2;
- the dimensions that identify the value of the two accounts are not homogeneous, namely the accounts do not operate on the same number of dimensions; in this case we have **non-homogeneous operands**. For example, if we assume to manage the custom dimensions 1 and 2 we can have [A] + [B] (Dest2.[GEN]). In this case, the two operands are not homogeneous since the value of the A account is read taking into consideration all the elements of the custom dimensions 1 and 2 while the value of the B account is read taking into consideration all the elements off the custom dimension 1 while the custom dimension 2 is fixed at GEN.

In presence of homogeneous operands the sum has a "standard" behavior and the TGKML script does not require a specific syntax, unlike the non-homogeneous operands that require the **BOUND** syntax.

Example 1

If we assume to perform the data entry on three custom dimensions: Cost centers, Products and Departments and that we want to calculate the value of the total revenues as sum of the revenues given by the products sale and of the other revenues

We can write the following TGKML script:

[LO].[50099] := [50010] + [50020]

where:

- [LO] = target cube (Gross amounts)
- [50099] = account to calculate(Total revenues)
- [50010] =input account containing the products revenue
- [50020]= input account containing the other revenues

INPUT ACCOUNT 50010

Account	CustDim1	CustDim2	CustDim3	Amount
50010	D11	D21	D31	5
50010	D11	D22	D32	6

INPUT ACCOUNT 50020

Account	CustDim1	CustDim2	CustDim3	Amount
50020	D11	D21	D31	7
50020	D11	D22	D32	8
50020	D12	D21	D31	9

OUTPUT ACCOUNT 50099

Account	CustDim1	CustDim2	CustDim3	Amount
50020	D11	D21	D31	5 + 7 = **12**
50020	D11	D22	D32	6 + 8 = **14**
50020	D12	D21	D31	0 + 9 = **9**

Thus, the system sums up the records of the two accounts which match perfectly on all the dimensions and just rewrites those that do not match.

Let's see some examples to better understand how the sum operator works.

Let us assume to manage the custom dimensions 1 and 2 and that we want to sum up the values of the C00000 e D00000 revenue accounts. Moreover, let us assume to have following data on the database.

CONTO INPUT C0000

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	D11	D21	01	50
A00	C00000	2013BDG	$AMOUNT	D11	D22	01	30

CONTO INPUT D00000

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	D00000	2013BDG	$AMOUNT	D11	D21	01	10
A00	D00000	2013BDG	$AMOUNT	D11	D22	01	20
A00	D00000	2013BDG	$AMOUNT	D12	D23	01	5

Example 2. Sum with NON homogeneous operands.

[LO].[X00000] := [C00000](Dest1.[D11]) + [D00000];

In this case the value of the single operands is:

[C00000](Dest1.[D11])

Entity	Account	Scenario	Category	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	D21	01	50
A00	C00000	2013BDG	$AMOUNT	D22	01	30

[D00000]

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	D00000	2013BDG	$AMOUNT	D11	D21	01	10
A00	D00000	2013BDG	$AMOUNT	D11	D22	01	20
A00	D00000	2013BDG	$AMOUNT	D12	D23	01	5

The system sums up the records that match on all the involved dimensions except the one where the reading from fixed point has been performed.

Thus the result is:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	X00000	2013BDG	$AMOUNT	D11	D21	01	50 + 10
A00	X00000	2013BDG	$AMOUNT	D11	D22	01	30 + 20

the third row of the D00000 account disappears because the D23 element of the custom dimension 2 does not have any equivalence in the C00000 account.

Thus, generally data might be lost when non homogeneous operands are summed up

Example 3. Sum with NON homogeneous operands changing order.


The previous example concerns the sum between two non homogeneous operands. If the operands are more than two, the order in which the operations are performed is important.

Let us assume to manage the two custom dimensions 1 and 2 and that we want to sum up the values of the C00000, D00000 and F00000 revenue accounts. Moreover, assume that we have following data on the database.

CONTO INPUT C0000

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	D11	D21	01	10
A00	C00000	2013BDG	$AMOUNT	D11	D22	01	20

CONTO INPUT D00000

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	D00000	2013BDG	$AMOUNT	D11	GEN	01	30
A00	D00000	2013BDG	$AMOUNT	D12	GEN	01	40

CONTO INPUT F00000

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	F00000	2013BDG	$AMOUNT	D11	D21	01	50
A00	F00000	2013BDG	$AMOUNT	D11	D22	01	60
A00	F00000	2013BDG	$AMOUNT	D12	D21	01	70
A00	F00000	2013BDG	$AMOUNT	D12	D22	01	80

The result of the script

[LO].[X00000] := [C00000] + [D00000](Dest2.[GEN]) +[F00000]

is

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	X00000	2013BDG	$AMOUNT	D11	D21	01	10+30+50
A00	X00000	2013BDG	$AMOUNT	D11	D22	01	20+30+60
A00	X00000	2013BDG	$AMOUNT	D12	D21	01	70
A00	X00000	2013BDG	$AMOUNT	D12	D22	01	80

While, if we invert the operands' order, namely we consider the script

[LO].[X00000] := [F00000] + [D00000](Dest2.[GEN]) + [C00000]

the result is:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	X00000	2013BDG	$AMOUNT	D11	D21	01	10+30+50
A00	X00000	2013BDG	$AMOUNT	D11	D22	01	20+30+60
A00	X00000	2013BDG	$AMOUNT	D12	D21	01	70+40
A00	X00000	2013BDG	$AMOUNT	D12	D22	01	80+40

and thus the result of a sum with non homogeneous operands changes reverting the operands' order.

Product operator

When the product of two accounts has to be calculated, it is necessary to distinguish two cases:

- the dimensions which identifies the value of the two accounts are homogeneous, namely the accounts operate on the same number of dimensions, in this case we talk about **homogeneous operator**. For example, if we assume we manage on the custom dimension 1 the cost center and on the custom dim 2 the products, we can calculate the value of the products' revenues as the product of the price [A] and the sold quantity [B]. In this cases the two operators are homogeneous since both the value of the account A and the one of the account B are read considering all the elements of the custom dimensions 1 and 2;
- the dimensions which identifies the value of the two accounts are not homogeneous, namely the accounts do not operate on the same number of dimensions, in this case we talk about **not homogeneous operators**. For example, if we assume we manage on the custom dimension 1 the cost center and on the custom dim 2 the products, we can calculate the value of the products' revenues as the product of a generic price (price of the generic product GEN) [A](Dest2[GEN]) and the sold quantity [B]. In this case the two operators are not homogeneous since the value of the account A is read considering all the elements of the custom dimension 1 while the custom dimension 2 is fixed on GEN and the value of the account B is read considering all the elements of the custom dimension 1 and 2.

In presence of homogeneous operators, the product has a "standard" behavior namely the product is performed between the values whose dimensions correspond exactly to each other.

Example 1

Let us assume we collect data on three custom dimensions: Costs center, Products and Department and we want to calculate the value of the products sale's revenues as the product between price and the sold quantity of each product.

We can write the following TGKML script:

[LO].[50010] := [90020] * [90010]

where:

- [LO] = target cube (Gross amounts)
- [50010] = account to calculate(Revenues - Products)
- [90020] = Input account containing the average price of the product
- [90010]= Input product containing the sold quantities

INPUT ACCOUNT 90020

Account	Custdi1	Custdi2	Custdi3	Amount
90020	D11	D21	D31	20
90020	D11	D22	D32	30

INPUT ACCOUNT 90010

Account	custdi1	Custdi2	Custdi3	Amount
90010	D11	D21	D31	4
90010	D11	D21	D32	5
90010	D12	D22	D31	6

OUTPUT ACCOUNT 50010

Account	Custdi1	Custdi2	Custdi3	Amount
50010	D11	D21	D31	20 * 4 = **80**

Example 2

Let us assume we have to calculate

[LO].[50010] := [90020](Dest2.[GEN]) * [90010]

where:

- [LO] = target cube (Gross amounts)
- [50010] = account to calculate (Revenues - Products)
- [90020] = input account containing the average price of the product
- Dest2.[GEN] = element of the custom dimension 2 where the product's average price is stored
- [90010]= input account containing the sold quantities

INPUT ACCOUNT 90020

Account	Custdi1	Custdi2	Amount
90020	D11	GEN	2
90020	D12	GEN	3
90020	D12	D22	4

INPUT ACCOUNT 90010

Account	Custdi1	Custdi2	Amount
90010	D11	D21	6
90010	D11	D22	7
90010	D12	D21	8
90010	D12	D22	9
90010	D13	D21	10

OUTPUT ACCOUNT 50010

Account	Custdi1	Custdi2	Amount
50010	D11	D21	2 * 6 = **12**
50010	D11	D22	2 * 7 = **14**
50010	D12	D21	3 * 8 = **24**
50010	D12	D22	3 * 9 = **27**

Example 3

Let us assume we have to calculate

[LO].[50010](Dest2.Filter([D21],[D22])) := [90020] *[90010]

where:

- [LO] = target cube (Gross amounts)
- [50010] = account to calculate (Revenues - Products)
- [90020] = input accounts containing the product's average price
- Dest2.Filter([D21],[D22]) = filter on the elements D21 and D22 of the custom dimension 2
- [90010]= input account containing the sold quantities

INPUT ACCOUNT 90010

Account	custdi1	Custdi2	Custdi3	Amount
90010	D11	D21	D31	20
90010	D11	D22	D32	30
90010	D12	D22	D31	40
90010	D11	D23	D32	50
90010	D12	D23	D31	60

INPUT AMOUNT 90020

Account	custdi1	custdi2	custdi3	Amount
90020	D11	D21	D31	4
90020	D11	D22	D32	5
90020	D12	D22	D31	6
90020	D11	D23	D32	5
90020	D12	D23	D31	6

OUTPUT ACCOUNT 50010

Account	custdi1	custdi2	custdi3	Amount
50010	D11	D21	D31	20 * 4 = **80**
90020	D11	D22	D32	30 * 5 = **150**
90020	D12	D22	D31	40 * 6 = **240**

Example 4

Let us assume that we must calculate

[LO].[X0000] := [C0000](Dest1.[D12]) *[D0000]

INPUT ACCOUNT C0000

Account	custdi1	custdi2	amount
C0000	D11	D21	50
C0000	D12	D22	30

INPUT ACCOUNT D0000

Account	custdi1	custdi2	amount
D0000	D11	D21	10
D0000	D11	D22	20
D0000	D12	D22	5

OUTPUT ACCOUNT X0000

Account	custdi1	custdi2	amount
X0000	D11	D22	30 * 20 = **600**
X0000	D12	D22	30 * 5 = **150**

Example 5

Let us assume that we must calculate

[LO].[X0000] := [C0000](Dest1.[D12]) *[D0000](Dest2.[D22])

CONTO INPUT C0000

Account	custdi1	custdi2	amount
C0000	D11	D21	50
C0000	D12	D21	40
C0000	D12	D22	30

INPUT ACCOUNT D0000

Account	custdi1	custdi2	amount
D0000	D11	D21	10
D0000	D11	D22	20
D0000	D12	D22	5

OUTPUT ACCOUNT X0000

Account	custdi1	custdi2	amount
X0000	D11	D21	40 * 20 = **800**
X0000	D12	D21	40 * 5 = **200**
X0000	D11	D22	30 * 20 = **600**
X0000	D12	D22	30 * 5 = **150**

Example 5

Let us assume that we must calculate

[LO].[X0000] := [C0000](Dest1.[D11]) *[D0000](Dest1.[D12])

INPUT ACCOUNT C0000

Account	custdi1	custdi2	amount
C0000	D11	D21	50
C0000	D12	D21	40
C0000	D12	D22	30

INPUT ACCOUNT D0000

Account	custdi1	custdi2	amount
D0000	D11	D21	10
D0000	D11	D22	20
D0000	D12	D22	5

OUTPUT ACCOUNT X0000: This account is empty because the products doesn't give any result.

Parallel operators

When running an assignment of sum, subtraction, product, division type where both branches of the operation have very long calculation times, it is possible to use the parallel operators |+|, |-|, |*|, |/|.

Let us assume we have the following calculation type

[LO].[50010](Dest1. [D13]) := [90020](Dest1. [D11]) **<op>** [90010](Dest1. [D12]);

where <op> is an operator +, -, *, /, and we know that both the first operand ([90020](Dest1. [D11])) and the second operand ([90010](Dest1. [D12])) have very long calculation time.

When using the parallel operators, namely

[LO].[50010](Dest1. [D13]) := [90020](Dest1. [D11]) **|<op>|** [90010](Dest1. [D12]);

the system tries to run the two branches of the operation in parallel, decreasing the total run time of the assignment.

 If only one of the two branches of the operation requires very long calculation time, the use of parallel operators does not make significant improvements to the runtime.

BOUND function in a sum

The "Bound" function is used when it is necessary to sum up one or more elements, some of which have the reading from fixed point option so as to get the correct result and to avoid losing data like in the standard sum in presence of non-homogeneous operands.

The syntax of this function is:

Bound(<bounding expression>, “name”, <sum>)

Technically, when this function is used, the system deploys the domain of each account belonging to the sum (the domain is the combination of the various dimensions for which the account is set) in accordance with the specified domain in the Bound utility.

Let us assume that we have

[LO].@temp := [D00000];

[LO].[X00000] := Bound(@temp, “bound”, @temp + [C00000](Dest1.[D11]));

namely, the Bound expression is the D00000 account and that data present on the database are:
CONTO INPUT C00000

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	D11	D21	01	50
A00	C00000	2013BDG	$AMOUNT	D11	D22	01	30

CONTO INPUT D00000

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	D00000	2013BDG	$AMOUNT	D13	D21	01	10
A00	D00000	2013BDG	$AMOUNT	D11	D22	01	20
A00	D00000	2013BDG	$AMOUNT	D12	D23	01	5

the domain of the C00000 account is deployed in accordance with the domain of the BOUND deployment; the result is

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	D13	D21	01	50
A00	C00000	2013BDG	$AMOUNT	D11	D22	01	30
A00	C00000	2013BDG	$AMOUNT	D12	D23	01	0

At this point, we have a standard sum of homogeneous operands whose result is:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	X00000	2013BDG	$AMOUNT	D13	D21	01	50 +10
A00	X00000	2013BDG	$AMOUNT	D11	D22	01	30 + 20
A00	X00000	2013BDG	$AMOUNT	D12	D23	01	0 + 5

Advanced use of the BOUND function

Let's see some advanced uses of the BOUND functions.

As stated in the paragraph about the **IF Construct**, the IF expression assessment system runs the following operation

<logical_condition> *<then_expression> + <logical_condiztion>*<else_expression>.

but, for several reasons, this expression might not give the correct result, for example when the <logical_condition> cannot be evaluated (for example, an account is compared to a constant) or the <logical_condition> , the <then_expression> and the<else_expression> have not homogeneous operands.

In these cases, in order to have a correct result, it is necessary to use the BOUND function and write

[LO].[X00000] := Bound ([D00000] + [E00000], @bound, if [C00000] == 0 then [D00000] else [E00000] end);

- **[C00000](1-[D00000])**

  By using the BOUND function it is possible to calculate the correct result for all the combinations in which D00000 is not defined. For example, if we write

  [LO].[X00000] := Bound([C00000], “bound”, @bound * (1 – [D00000]));

  the result will be C00000 for all the combinations in which D00000 does not exist. This result is got only using the BOUND function which allows to deploy the domain of D00000 in accordance with the one of C00000.

   In this example we have used the C00000 account as BOUND expression . Using an account belonging to an assignment as BOUND expression is not mandatory, but you can use any defined account.

Functions and examples by topic

Time management

The Time dimension is a mandatory dimension for each TGKML script. Tagetik provides the user with a set of functions which allow to move along the "time" axis (scenario/period) while working on data present on different scenarios/periods.

The Time dimension can be navigated as follows:

Syntax	Description
Time.Prev	Previous period
Time.Next	Following period
Time.Lag(N)	N periods backward. N can be also a negative number and, in this case, the move is forward Example: Time.Lag(3) means third period before the current one
Time.Lead(N)	N periods forward. N can be also a negative number and, in this case, the move is backward Example: Time.Lead(3) means third period after the current one
Time.Scenario	sum of all the Periods of the current Scenario (the one the current period belongs to)
Time.PN	Where N is an integer. Sum of a group of periods (P3=three months; P2=two months; P4=four month) Example: Time.P3 sum of the amounts of the three months related to the current scenario
Time.Scenario.Prev	Previous Scenario
Time.Scenario.Next	Following Scenario
Time.Scenario.First	First period of the current scenario
Time.Scenario.Last	Last period of the current scenario
Time.Scenario.Prev.Last	Last period of the scenario before the current scenario
Time.Scenario.Last(P3)	Last four months on the current scenario
Time.Scenario.Period(N)	With N = 1,..,maxPeriod Period N of the current scenario
Time.Scenario.First(P3).Last	Last period of the first four months of the current scenario
Time.Ref(N)	With N = 1,..,5 Reference scenario N related to the current scenario
Time.[scenario_code]	Sum of all the periods of a specific scenario Example: Time.[2013BDG] = sum of all the periods of the scenario 2013BDG
Time.[scenario_code|period_code]	Reads data from a specific scenario/period. Example: Time.[2013BDG|12] = period 12 of the scenario 2013BDG

Moreover it is possible to specify a time range using the following syntaxes

Syntax	Description
Time.ToDate	All the periods <= of the current period
Time.ToDate(P3)	All the periods<= of the current period of the current four month
Time.P3.ToDate	All the four months of the current scenario<= of the current four month
Time.Span(1)	Current period and previous period
Time.P3.Span(1)	Current and previous for month

 The above mentioned features for the Time dimension have to be used in an assignment as follows:

[LO].[X0000] := [C00000](Dest1.Filter([01|D11]), Time.Scenario.Prev);

[LO].[X0000] := [C00000](Dest1.Filter([01|D12]), Time.Scenario.Ref(3));

When it is needed to read, within an assignment, the current period from a scenario which is not the current one, the "**ParallelPeriod**" function has to be used.

For example, if the current period is 3, using the syntax

... := [C00000](Dest1.Filter(01|D11), Time.ParallelPeriod(Scenario.Prev));

the system reads from the same period (3) of the previous scenario.

The system reads from the same period (3) of the previous scenario, whereas by using the syntax

... := [C00000](Dest1.Filter(01|D11), Time.ParallelPeriod(Scenario.[2015ACT]));

the system reads data from the same period (3) of the scenario 2015ACT.

Dimension management

For each dimension (except period, scenario and account) Tagetik provides the user with the following utilities:

- **All**. It behaves like the reading from fixed point. It allows to read data summing up on all the elements belonging to the dimension.

  For example, if data on the database are:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	C00000	2013BDG	$AMOUNT	D11	D21	01	50
A00	C00000	2013BDG	$AMOUNT	D11	D22	01	30
A00	C00000	2013BDG	$AMOUNT	D11	D23	01	20

  and we write ...:=[C00000](Dest2.All) then read data are:

Entity	Account	Scenario	Category	Dest1	Period	Amount
A00	C00000	2013BDG	$AMOUNT	D11	01	100

- **Ancestor**. Allows to read data summing up on alla the elements related to the parent node of N level
- **Parent**. Allows to read data summing up on all the elements related to the parent node

- **RefLeaf(Parent([hierarchy_code]))**. Allows to read the reference element from a given hierarchy.

  For example if we write ... := [C00000](Entity.RefLeaf(Parent([01]))); the system read, for each entity, data from the reference element of the parent node of the 01 hierarchy's entity

- **RefLeaf(Ancestor([hierarchy_code],level))**. If we write ... := [C00000](Entity.RefLeaf(Ancestor([01],2)); the system reads, for each entity, data from the reference element of the second level parent node of the 01 hierarchy's entity.

Write on an account node/set

The TGKML syntax allows to write the script's result not only on a single account but also on an accounts node or on a set of nodes.

Write on node

The system writes the same result on all the accounts which are related to the specified node. The syntax to use is the following:

[LO].[account hierarchy code|node code] := ...;

For example, we can write [LO].[GG|03] to indicate to write the result on all the accounts which are related to the node 03 of the hierarchy GG.

In this kind of assignments, it is possible to specify as input accounts the same output accounts (typically read from another part of the cube). This is the syntax:

[LO].[GG|03] := [ ](Time.ParallelPeriod(Time.Scenario.Prev));

 Please, note that there is a space between square brackets.

It is possible to combine these operands with normal operands (single input accounts). I this case the amounts read from single input accounts are combined with all the output accounts. For example:

[LO].[GG|03] := [20132] * [ ](Time.ParallelPeriod(Time.Scenario.Prev));

Write on a set of accounts

It is possible to define a set of accounts (for further details see SET expression) and then write the result on them only.

For example, we can define the following set of accounts:

Account.@tmp := ([account_1] + [account_2] + [account_3]);

to use in the script syntax as follows:

[LO].@tmp :=...;

MULTI function

It is possible to have a result similar to the previous ones using the **Multi** function:

[LO].Multi([account_1] + [account_2] + [account_3] +..., Dimension.Filter([]), Dest#.[...],...,...) := ...;

For example we might have

[LO].Multi([X00000] + [C00000], Dest1[D11], Entity.Filter([01|$])) := [D00000] *[P00000];

In this case the product's result is written on each account that belongs to the set defined by the Multi syntax.

The Multi syntax can be used in the input part of the script as well. In this case the system calculates the sum of the accounts that belong to the set defined by the Multi syntax.

Examples:

[LO].@TMP1 := Multi([C00000] + [D00000] + [H00000], Per, Dest1[D11]);

[LO].@TMP2 := Multi([E00000] + [F00000], Per);

[LO].[X00000] := Bound(@TMP2, "P", @TMP1 - @TMP2);

 Even if the Multi function and the Set function are similar to each other, the first one is more efficient since the system processes it one time only and not, like for the Set function, every time it is called .

LoopOnTime

Generally, when a TGKML script is run on several scenarios / periods at the same time, the system calculates data and then stores them all at once to the database and not one scenario / period per time.

Whether the user needed to manage the same account as output and as input within the same script, for example to value data on several months using, per each month, previous month's data and wrote an assignment of the following type:

[LO].[X00000] := [X00000](Time.Prev)

this assignment wouldn't give any result.

Indeed, in these cases it is necessary to use a specific syntax: the **LoopOnTime** and then write

[LO].[X00000](LoopOnTime) := [X00000](Time.Prev)

 When the LoonOnTime is used, technically the system calculates and writes data one scenario / period per time.

When the accounts to value are more than one and they refer to each other and thus it is necessary to run a set of assignment n times, one for each scenario/period specified in the filter, then it is necessary to use the **LoopOnTime** syntax and write

<< "LoopOnTimeOn";

assignment 1

assignment 2

assignment 3

<< "LoopOnTimeOff";

In this way the system runs the assignment 1 on the first scenario/period, then the assignment 2 on the first scenario/ period, then the assignment 3 on the first scenario period and then the same assignments on the scenario/period 2 and so on.

 The LoopOnTimeOn syntax has to be used only when it is really necessary since it can delay the script runtime.

LoopOnTime(Scenario)

If the user needs to use, within the same script, the same account as output and as input, for example to build data using data of the previous scenario and not of the previous scenario/period, it he can use the syntax LoopOnTime(Scenario)and then write, for example

[LO].[X00000](LoopOnTime(Scenario)) := [X00000](Time.Prev)

 Technically, when using the syntax LoopOnTime(Scenario), the system calculates and writes data scenario by scenario.

When the accounts to build are more than one and they are related to each other and therefore it is necessary to run a set of assignments n times, one for each scenario specified in the filter, the syntax to use is **LoopOnTimeOn(Scenario)** and then write

<< "LoopOnTimeOn(Scenario)";

assignment 1

assignment 2

assignment 3

<< "LoopOnTimeOff";

In this way, the system runs the assignment 1 on the first scenario, then the assignment 2 on the first scenario, then the assignment 3 on the first scenario and then the same assignments on the scenario 2 and so on.

Exponentiation (^)

Exponentiations can be performed using the ^ operator.

The syntax is

base^exponent

Where base and exponent can be numeric values or any complex expression.

Depending on the value of base and exp, the function behaves as follows:

- if the base > 0 the operation is performed just like in mathematics;
- if the base < 0 and the exponent is integer (positive/negative or 0) the operation is performed just like in mathematics;
- if the base < 0 and the exponent is not integer the result is 0;
- if the base = 0 the result is 0.

Example 1

Let's assume that we want to calculate

[LO].[X00000] := [C0000] ^ 2;

INPUT ACCOUNT C0000

Account	CusDim1	CusDim2	Amount
C0000	D11	D21	10
C0000	D11	D22	20
C0000	D12	D22	-5

OUTPUT ACCOUNT X0000

Account	CusDim1	CusDim2	Amount
X0000	D11	D21	10 ^ 2 = **100**
X0000	D11	D22	20 ^ 2= **400**
X0000	D12	D22	(-5) ^ 2 = **25**

Example 2

Let's assume that we want to calculate

[LO].[X00000] := [C00000] ^ [D00000];

INPUT ACCOUNT C0000

Account	CusDim1	CusDim2	Amount
C0000	D11	D21	9
C0000	D12	D21	3
C0000	D11	D22	-2
C0000	D12	D22	-1

INPUT ACCOUNT D0000

Account	CusDim1	CusDim2	Amount
D0000	D11	D21	2
D0000	D11	D22	3
D0000	D12	D22	3
D0000	D12	D22	0.5

OUTPUT ACCOUNT X0000

Account	CusDim1	CusDim2	Amount
X0000	D11	D21	9 ^ 2= **81**
X0000	D11	D22	3 ^ 3 = **27**
X0000	D12	D22	(-2) * 3 = **-8**
X0000	D12	D22	(-1) * 0.5 =**0**

PeriodIndex

The **PeriodIndex** function allows to obtain an integer based on the current period which the script is evaluated for. Thus the following syntax

[LO].[X00000] := [C00000] / PeriodIndex;

allows to divide the value of the account C00000 by:

- 1 when the assignment is evaluated for period 01;
- 2 when the assignment is evaluated for period 02;
- 3 when the assignment is evaluated for period 03;
- ...
- 12when the assignment is evaluated for period 12.

The PeriodIndex function can be used also in a parametric way, namely it is possible to write, for example, PeriodIndex(P3), where P3 (quarter) specifies the time range to consider, so as to have the following results:

- 1 when the assignment is evaluated for periods 01, 04, 07, 10
- 2 when the assignment is evaluated for periods 02, 05, 08, 11
- 3 when the assignment is evaluated for periods 03, 06, 09, 12

Copy one dimension to another

Through a TGKML script it is possible to copy one dimension to another. Let's see some examples to better understand which syntax has to be used.

[X00000](EntityCtp.Entity) - sets the ctp entity equal to the entity;

[X00000](EntityCtp.Entity, Entity.EntityCtp) - exchanges the entity with the ctp entity;

[X00000](Currency.Currency2) - sets the original currency equal to the currency

[X00000](Amount.Amount2) - sets the amount in original currency equal to the amount (it works in the same way for the currency even though it is not specified).

Moreover, Tagetik allows to read the amount rather than the amount in original currency (for further details about the amounts see **Amounts and currencies**) without changing the information on the currency. For example, if we write:

[20132](Amount2OnAmount) - sets the amount in original currency equal to the amount (but it DOES NOT work this way on the currency fields).

Rounding and average

In a TGKML script it is possible to:

- round the value of an account to a given number of significant digits;
- calculate the average of a given expression.

Rounding

The function to use to round the value of an account to a given number of significant digits (number of decimals) is

Round(n)

where n is an integer that specifies the number of decimal figures to take into account.

Thus, if we write

... := [C00000](Round(2));

the value of the C00000 account is read rounded at two decimal figures.

On the other hand, if we write

[LO].[X00000](Round(2)) := ...;

the value of the output account X00000 is rounded to two decimal digits.


When using the Round(n) function in output, the rounding applies to the final result that includes the negative amounts with not calculated origin. For example, if we round to zero decimal digits the result of a calculation that gives 1 and the database already includes a data with amount 0.1 and other origin, we will get a record with amount 1 (1 - 0.1 = 0.9 rounded to 1) with calculation origin. So the total on the database will be 1.1.

Whereas, when using the Round(n)function in input, the rounding is applied only to the data reading and, in the previous example, we would get a record with amount 0.9 (Round(1,0) - 0.1 = 1 - 0.1) with calculation origin. The total on the database will be 1.

Average

Tagetik provides the user with two different utilities

- **Avg**
- **AvgDomain**

for calculating the average values.

The **Avg** utillity

Avg(<expression>, <dimensional specifications>)

divides the obtained value considering the specified dimensions by the number of set tuples that contributed to the value.

The AvgDomain utility

AvgDomain(<expression>, <dimensional specifications>)

divides the obtained value considering the specified dimensions by the number of set tuples and which have not contributed to the value.


The following can be entered as input dimensional specifications for the functions Avg and AvgDomain:

- one or more specifications of Dim.Parent(<hierarchy code>) type: in this case, each lowest level element related to a node of the specified hierarchy according to the Parent logic, will be associated with an amount equal to the average amounts present on the lowest level elements related to the same node according to the Parent logic;
- one or more specifications of Dim.Ancestor(<hierarchy code>, <level>) type: in this case, each lowest level element related to a node of the specified hierarchy according to the Ancestor logic, will be associated with an amount equal to the average amounts present on the lowest level elements related to the same node according to the Ancestor logic;
- Time.PN: in this case, the average value of the n-tuple is associated to each n-tuple on the PN time interval;
- Time.Scenario: the average value of the n-tuple is associated to each n-tuple on the current scenario;

Example 1

Let us assume we have we have the node 01 of a hierarchy defined on the custom dimension 1 to which the elements D01, D02, D03, D04 are related and of a hierarchy defined on the custom dimension 1 to which the elements D01, D02, D03, D04 are related and that on such elements we have the following values

Dest1	Amount
D01	10
D02	20
D03	30
D04	

Then if we write

... := Avg ([C00000], Dest1.Parent([01]));

the result will be

Dest1	Amount
D01	60/3 = 20
D02	60/3 = 20
D03	60/3 = 20

while, if we write

... := AvgDomain ([C00000], Dest1.Parent([01]));

the result will be

Dest1	Amount
D01	60/4 = 15
D02	60/4 = 15
D03	60/4 = 15
D04	60/4 = 15

.

Currency conversion

When a TGKML script reads data from an entity and writes the result on another entity which has a different currency, it is necessary to perform a conversion; indeed in these cases the normal value the system would write on the amount 2 (for further details see "Amounts and currencies" on page 83) doesn't make any sense.

Tagetik provides the users with the **ConvertOriginal**utility where the amount 2 is calculated by converting the amount into currency 2.

 The ConvertOriginal function can be used with the temporary accounts only if all needed information for the conversion is available in the temporary account, that is:

- scenario
- period
- account
- entity or journal number (information not needed for the consolidated cubes)
- currency

 From Tagetik 5.0.2, the conversion utility **ConvertInput** is no longer supported.

Category pool

Within a TGKML script it is possible to use the categories grouping's nodes labeled as "pool" (for further details see Editable categories pool) using the following syntax

Category.Pool ([node code|grouping code])

Let's see some example about the behavior of the system when the abovementioned function is used within the script.

Let us assume we have the following script:

[LO].[X00000](Category.Pool([01|$])) := ...

where 01 is a categories grouping's node labeled as pool. Let's assume that:

- CAT_1 and CAT_2 are two categories related to he node 01;
- USER_1 is authorized to write on the category CAT_1;
- USER_2 is authorized to write on the category CAT_2;

and that data on the database are:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount	Origin
A00	X00000	2013BDG	CAT_1	D11	D21	01	10	INPUT_DEFORM
A00	X00000	2013BDG	CAT_2	D11	D22	01	20	INPUT_DEFORM

The user USER_1 runs the script and has as result 100. In this case the system writes on the database following data

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount	Origin
A00	X00000	2013BDG	CAT_1	D11	D21	01	10	INPUT_DEFORM
A00	X00000	2013BDG	CAT_2	D11	D22	01	20	INPUT_DEFORM
A00	X00000	2013BDG	CAT_1	D11	D21	01	70	PROC_CALCULATED

The user USER_2 runs the script and has as result 110. Inthis case the system writes on the database following data

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount	Origin
A00	X00000	2013BDG	CAT_1	D11	D21	01	10	INPUT_DEFORM
A00	X00000	2013BDG	CAT_2	D11	D22	01	20	INPUT_DEFORM
A00	X00000	2013BDG	CAT_1	D11	D21	01	70	PROC_CALCULATED
A00	X00000	2013BDG	CAT_2	D11	D22	01	10	PROC_CALCULATED

 If in the Input part of the script the currency is not specified, while reading data the system performs a reading from fixed point on the category node specified in the output part.

 The system, at the deletion step, deletes data by user.

Swap

This function allows to swap two dimensions and can be used both in the input and output part of an assignment.

The syntax is:

Swap(<dim1>,<dim2>)

When used in the input expression of an assignment, the system swaps the codes of the two dimensions having the same effect of the Copy one dimension to another function performed twice.

 If a filter is present on one (or both) of the involved dimensions, it is applied on the swapped dimensions (e.g. if the user swaps Entity and EntityCtp, the filter on entity is applied on the counterpart).

When Swap is used in the output expression of the assignment, the system:

- Deletes every row on the target table (the dimensions are swapped even when applying a filter);
- During the compensation amount calculation all existing records are read swapping the dimensions.

New Journal creation

In TGKML scripts that produce Journals (run on journal cubes RETT and RETT_C), the system checks if all the required Journal Headers exist on the Scenario/Period being processed.

It is possible to create new Journal Headers to contain the resulting Journals based on other existing Headers that have the same Journal Number field.

There are two functions that allow doing so: **NewRett()** and **OpenAdj()**.

Both functions can only be used in the output expression of an assignment.

NewRett

Its syntax is:

NewRett(<Time Jump>)

where <Time Jump > is a Time Movement function (refer to "Time management" on page 112 for details) whose result will be used to determine the Journal Header used as reference for the creation of the new ones.

Assume having the following assignment:

[RETT].[OUT](NewRett(Time.Scenario.Prev.Last)) := [IN1]+[IN2];

the system checks if all the Journals that are returned as result of the input expression have a corresponding Journal Header in the Scenario/Period being processed. If the Header does not exist, the system will create a new one with the following fields:

- Scenario and Period - same as the ones being processed.
- Journal Number - same as the one resulting from the input expression.
- All the other fields - same as the Header with the following specifics:
  - Scenario and Period obtained by applying <Time Jump> to the Scenario/Period being processed, in this case Last Period of the Previous Scenario
  - Journal Number resulting from the input expression of the assignment

 If no Journal Header with the same Number exists for the Scenario/Period obtained by the Time Jump, data will be inconsistent.

In case the cube used in the assignment is RETT_C the new Header will have the following fields:

- Scenario - same as the Original Scenario of the Consolidation one being processed.
- Period - same as the one being processed.
- Journal Number - same as the one resulting from the input expression.
- All the other fields - same as the Header with the following specifics:
  - Scenario and Period obtained by applying <Time Jump> to the Consolidation Scenario/Period being processed and then shifting to its Original Scenario.
  - Journal Number resulting from the input expression of the assignment.

OpenAdj

In this case no time jump is performed. The system will check all input records and retrieve the Journal Headers for all input Scenario/Period combinations.

Assume having the following assignment:

[RETT].[OUT](OpenAdj) := [IN1](Time.Scenario.Prev.Last)+[IN2];

the system checks if all the Journals that are returned as result of the input expression have a corresponding Journal Header in the Scenario/Period being processed. If the Header does not exist, the system will create a new one with the following fields:

- Scenario and Period - same as the ones being processed.
- Journal Number - same as the one resulting from the input expression.
- All the other fields - the system searches all Headers having the above Journal Number for the Scenario/Periods of the Input records (in this case Current Scenario/All Periods and Previous Scenario/Last Period) and acts as follows:
  - If the Journal Number is present on exactly one Scenario/Period, the system will use the fields of the corresponding Journal Header;
  - If the Journal Number is present on more than one Scenario/Period, the system checks if the corresponding Journal Headers are identical. If all fields match, they will be used for the new Header being created, otherwise, the system signals an error.
  - If there is no match, the system signals an error.

Aggregate functions: Max/Min/Sum

Aggregate functions that are available are: Min, Max, Sum. They return the minimum, maximum or average for the set of records taken into account.

The syntax is:

<function>(<expression>, <dimension1>, ...)

where:

- <function> can be Max, Min or Sum
- <expression> is any TGKML expression
- <dimension1> is any dimension on which the expression is not constant. It's possible to specify more than one dimension (comma separated)

Here an example:

[LO].[90010](Dest2.[GEN]):= Max([50210], Dest2);

The result is obtained starting from the given expression, grouping on all dimension except the one(s) specified. For each obtained row, the amounts are generated calculating the specified aggregation function (maximum, minimum or sum), on all the rows which generated current row.

If the original set of records are

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	50210	2013BDG	$AMOUNT	D11	D21	01	20
A00	50210	2013BDG	$AMOUNT	D11	D22	01	45
A00	50210	2013BDG	$AMOUNT	D12	D22	01	30
A00	50210	2013BDG	$AMOUNT	D12	D27	01	25

the result is:

Entity	Account	Scenario	Category	Dest1	Dest2	Period	Amount
A00	90010	2013BDG	$AMOUNT	D11	GEN	01	45
A00	90010	2013BDG	$AMOUNT	D12	GEN	01	30

DaysNumber function

The **DaysNumber** function allows you to retrieve the number of days set up on the period. For the consolidation scenarios, the value is read from the corresponding original period. Therefore, the following syntax

[LO].[X00000] := [C00000] / DaysNumber;

allows you to divide the value of the account C00000 by the number of days between the beginning of the scenario and the end of the run period.

The DaysNumber function can indicate a time jump and / or a periodic reading of the value:

- DaysNumber(Per) reads the number of days of the current period
- DaysNumber(Per(3)) reads the number of days of the last 3 months (rolling)
- DaysNumber(Time.P3.Last, Per(3)) reads the number of days of the current quarter.

Countervalue calculation

Abstract

This data processing is aimed at providing an instrument to calculate the counter value of the amounts (gross and IC) in a massive way from the transaction currency (original currency) to the entity currency.

Description

Generally the countervalue calculation is not necessary since in Tagteik data are automatically defined in dual currency. Nevertheless, in some cases it is necessary to update the amount expressed in the entity currency from the amount expressed in the transaction currency in a massive way. This is the case, for instance, of the Currency interest rates update or of the import of only the amount in the transaction currency from external source .

Setup

In order to make the countervalue calculation work properly, it is necessary to define the following information:

- Currency interest rates
- Period length (for the conversion at period average FX rate)

Currency interest rates

Accounts of P&L and Other Variation nature or accounts of Variation type

The currency interest rate used for the countervalue calculation of these accounts is the one defined in the accounts table, with some exceptions:

1. When in the process Data entry rules (*Setup & Admin > Processes > Select Process > Process Definition > Data Entry & Submission Rules > Gross Amount & IC > Override rules*) (*Navigation panel> Processes & Workflows > Process > Select Process > Process Definition > Data Entry & Submission Rules > Gross Amount & IC > Override rules*) the "Override Period average FX rate on P&L or on Variations accounts" value has been selected as **P&L: Countervalue calculation method** :

   In this case we use the **period average** interest rate applied to the periodic amount;

2. When a P&L FX rate or a Previous P&L FX rate has been specified on the account table:

In this case we use the interest rate defined in the **P&L: Countervalue calculation method** of the process;

3. When one of the following FX rates has been specified on the account table: Previous Final, Historical, Initial Balance, Year-to-date average.

   In this case we use the **final** interest rate

Normal or Detail accounts of BS, Guar. and Comm. and Other stocks nature

These accounts are converted only if the "Run Countervalue calculation on BS, Guar. and Comm. and Other stocks accounts" option is enabled in the data entry advanced rules of the process.

The currency interest rate used for the countervalue calculation of these accounts is the one defined in the accounts table, with some exceptions:

1. When a P&L FX rate or a Previous P&L FX rate has been specified on the account table.

   In this case we use the interest rate defined in the **P&L: Countervalue calculation method** of the process;

2. When one of the following FX rates has been specified on the account table: **Previous Final**, **Historical**, **Initial Balance**, **Year-to-date average**

   In this case we use the **final** interest rate

Period length

The period length defined in the Scenario/Period table is used to identify the previous period when it is necessary to calculate the periodic amount.

Stored procedure

Tagetik allows data processing via stored procedures.

Stored procedures are necessary in the following cases:

- the functionalities required by the user are not feasible using standard or multidimensional account calculation logic, built-in logic, or ETL processes;
- when there is a performance issue such that using an ETL process would be less efficient;
- when the use of an ETL would not produce the desired results, or when parameterisation would be too complex.

 Whenever using a stored procedure, it is essential to remember that

- as with ETLs, special attention should be paid to the sequence in which they are executed (a sequence not structurally guaranteed by Tagetik);
- The execution of a stored procedure does not use the Tagetik technological framework; that is, it is executed without any controls related to user restrictions, Entity / custom dimension and account / custom dimension restrictions, or scenario / period locking (unless, of course, the stored procedure itself is explicitly developed to take these aspects into account).

A stored procedure should always be built with particular care to avoid destructive effects on the rest of the application. It is advisable to always seek the advice of Tagetik technicians.

A stored procedure can be:

- *scheduled* by inserting it within a job via a 'Data processing - Stored procedure' type task;
- *inserted as a task within a workflow* via a 'Stored Procedures' type task;
- *executed within a data entry form;*
- *executed by ETL* via a Stored Procedure type 'Command'.

Stored procedure definition

The definition of a stored procedure on the application database can be accessed from the administrator user web interface, by following the path:

- *Setup & Admin > Processes tile > Database objects*. In this case, users can access the full database objects list

For further details on the stored procedure definition window see **Database objects**.

In both cases, Tagetik always records within the audit trail the operations performed through these functionalities so that you always have full control over 'who, when and how' operated on the data

 The system places a condition on the name of the stored procedures that can be run directly by Tagetik: they must have a name (independently of any existing schema) with CPM_SP_ prefix

Parameters

Stored procedures can define input, output, or input/output parameters, depending on the type of database on which they are created and used.

Parameters can be of any type except binary.

SQL Server

@INPUT_PARAMETER VARCHAR(30)
@OUTPUT_PARAMETER VARCHAR(100) **OUTPUT**

Oracle

INPUT_PARAMETER **IN** VARCHAR2
OUTPUT_PARAMETER **OUT** VARCHAR2
INOUT_PARAMETER **IN OUT** NUMBER

Postgresql

IN INPUT_PARAMETER VARCHAR(30)
OUT OUTPUT_PARAMETER VARCHAR(30)
INOUT INOUT_PARAMETER INTEGER

SAP Hana

IN INPUT_PARAMETER VARCHAR(30)
OUT OUTPUT_PARAMETER VARCHAR(100)
INOUT INOUT_PARAMETER NUMERIC(2)

When selecting the stored procedure pre-execution parameters, only Input and Input/Output type parameters will be displayed.

If the stored procedure has been added as a task within a workflow, parameters can be assigned values inherited from the context—by using specific parameter naming—such as the data collection process, entity, source scenario, period, and step, thus avoiding the need to manually set them again.

For each parameter, values can be selected from a custom editor; the type of editor depends on the name assigned to the individual parameter (see the rules listed in the tables below).

Dimension	Prefix Name	Prefix Name with value inherited from workflow
Data collection process	RACCOLTA	RACCOLTA
Period	DIM_PER	WFDIM_PER
Scenario (both original and consolidation)	DIM_SCE	
Original scenario	DIM_SCE_O	WFDIM_SCE_O
Consolidation scenario	DIM_SCE_C	
Entity	DIM_AZI	WFDIM_AZI
Step	DIM_FASE	WFDIM_FASE
Custom dimension 1	DIM_DEST1	
Custom dimension 2	DIM_DEST2	
Custom dimension 3	DIM_DEST3	
Custom dimension 4	DIM_DEST4	
Custom dimension 5	DIM_DEST5	
Category	DIM_CAT	
Accounts	DIM_CONTO	
Cause	DIM_CAUSALE	
Unit	DIM_VAL	
Form	DIM_PROSP	

For all these parameters related to Tagetik dimensions, the assigned editor is an elements list lookup.

Other types	Prefix Name	Editor
Session user	SESSION_ USER	This parameter is never manually defined by the user, but is always defined automatically by the system with the code of the session who requested the launch of the stored procedure.  For Postgresql the parameter to use is SESSION_USR
Flag	FLAG_	Check box
Tagetik Enumerated Values List	COMBO_ <nome enumerato>	Drop-down list

Below are the names to be used for the most common enumerations applicable to “List of values from Tagetik enumerations” type parameters:

- NaturaConto;
- TipoConto;
- TipoCategoria;
- TipoScenario.



- if a parameter does not have a valid name or type, or if the stored procedure has not been deployed, then it will not appear in the list of user-executable procedures;
- It is possible to have multiple parameters that reference the same Tagetik dimension. for example RACCOLTA1 and RACCOLTA2;
- if a parameter has the name DIM_SCE_C1 or DIM_SCE_O1, it will be considered by the system as a consolidation scenario and an original scenario, respectively;
- if the Scenario and Period parameter is used, the lists proposed for the selection of values will not be related to each other; i.e. it will be possible to select a value for the period parameter regardless of whether such a period exists for the given scenario;
- in the lists proposed for the selection of parameter values, user restrictions are never applied.

When the stored procedures are executed, all Input, Output and Input/Output parameters will be reported in the audit logs.

EXAMPLES

Sql Server

```
CREATE PROCEDURE CPM_SP_TEST (
@RACCOLTA VARCHAR(30),
@OUTPUT1 VARCHAR(100) OUTPUT,
@DIM_SCE_O VARCHAR(15),
@OUTPUT2 VARCHAR(1000) OUTPUT,
@DIM_PER VARCHAR(2),
@NUM1 NUMERIC(2) OUTPUT,
@WFDIM_AZI VARCHAR(30),
@NUM2 NUMERIC(2) OUTPUT,
@COMBO_NATURACONTO VARCHAR(1),
@FLAG_TEST NUMERIC(1),
@SESSION_USER VARCHAR(90)
```

```
)
AS
BEGIN
SET @NUM1 = @NUM2 + 1
SET @OUTPUT1 = 'Test output1'
SET @OUTPUT2 = @RACCOLTA + '-' + @DIM_SCE_O + '-' + @DIM_PER + '-' + @WFDIM_AZI + '-' + @SESSION_
USER
RETURN (1)
END
```

Oracle

```
CREATE OR REPLACE PROCEDURE CPM_SP_TEST (
RACCOLTA IN VARCHAR2,
OUTPUT1 OUT VARCHAR2,
DIM_SCE_O IN VARCHAR2,
OUTPUT2 OUT VARCHAR2,
DIM_PER IN VARCHAR2,
NUM1 IN OUT NUMBER,
WFDIM_AZI IN VARCHAR2,
NUM2 IN OUT NUMBER,
COMBO_NATURACONTO IN VARCHAR2,
FLAG_TEST IN NUMBER,
SESSION_USER IN VARCHAR2
)
IS
BEGIN
NUM1 := NUM2 + 1;
OUTPUT1 := 'Test output1';
OUTPUT2 := RACCOLTA || '-' || DIM_SCE_O || '-' || DIM_PER || '-' || WFDIM_AZI || '-' || SESSION_USER || '-' ||
NUM1 || '-' || NUM2;
END CPM_SP_TEST;
```

Postgresql

```
CREATE OR REPLACE FUNCTION CPM_SP_TEST (
IN RACCOLTA VARCHAR(30),
OUT OUTPUT1 VARCHAR(30),
IN DIM_SCE_O VARCHAR(15),
OUT OUTPUT2 VARCHAR(30),
IN DIM_PER VARCHAR(2),
INOUT NUM1 INTEGER,
IN WFDIM_AZI VARCHAR(30),
INOUT NUM2 INTEGER,
IN COMBO_NATURACONTO VARCHAR(1),
IN FLAG_TEST INTEGER
)
RETURNS void AS
$$
BEGIN
NUM1 := NUM2 + 1;
```

```
OUTPUT1 := 'Test output1';
OUTPUT2 := RACCOLTA || '-' || DIM_SCE_O || '-' || DIM_PER || '-' || WFDIM_AZI || '-' || SESSION_USER || '-' || NUM1 || '-' || NUM2;
END;
$$ LANGUAGE plpgsql;
```

SAP Hana

```
CREATE PROCEDURE "CPM_SP_TEST" (
IN RACCOLTA VARCHAR(30),
IN DIM_SCE_O VARCHAR(15),
OUT OUTPUT2 VARCHAR(1000),
IN DIM_PER VARCHAR(2),
INOUT NUM1 NUMERIC(2),
IN WFDIM_AZI VARCHAR(30),
INOUT NUM2 NUMERIC(2),
IN COMBO_NATURACONTO VARCHAR(1),
IN FLAG_TEST NUMERIC(1),
IN "SESSION_USER" VARCHAR(255)
)
LANGUAGE SQLSCRIPT SQL SECURITY INVOKER AS
BEGIN
OUTPUT2 = :RACCOLTA || '-' || :DIM_SCE_O || '-' || :DIM_PER || '-' || :WFDIM_AZI || '-' || :SESSION_USER || '-' || :NUM1 || '-' || :NUM2;
END;
```

Example: The following stored procedure reads the data from the cogs table, assigns them progressive values, and then loads them into the gross balances table based on the account in which they must be stored.

```
ALTER PROCEDURE [dbo].[CPM_SP_COGS_TO_DSL] as
declare @provenienza varchar(50)
declare @SCE_O VARCHAR(10)

begin
set @SCE_O = '2011MACT'
set @provenienza = 'CPM_SP_COGS_TO_DSL'

select
cod_azienda,
cod_scenario,
cod_periodo,
'G01' Dest1,
ProdCategoryCode Dest2,
ClusterCustomerCode Dest3,
CompanyCurrencyCode CompanyCurrencyCode,
DocCurrencyCode DocCurrencyCode,
-sum( isnull(Qty_L , 0 ) ) QTY_L,
-sum( isnull(Qty_A , 0 ) ) QTY_A,
-sum( isnull(Qty, 0 ) ) AV_P1_M2,
-sum( - isnull(SalesmanCommision, 0)
```

```
- isnull(DistributorCommision, 0)) P25100_M2,
-sum( -isnull(Cost , 0 ) ) P15110_M2,
-sum( -isnull(CostCorporate_S,0 )) CPR_S,
-sum( -isnull(YearEndBonus,0)) B00001,
-sum( isnull(Value_F_A ,0 )) VAL_F_A,
sum( isnull(Value_F_S ,0 )) VAL_F_S, -sum( isnull(Value_F_A ,0 )+ isnull(Value_F_S ,0 )) VAL_F,
-sum( isnull(Value_L_S ,0 )) VAL_L_S,
-sum( -isnull(Royalties , 0 )) P25200_M2
/* Start: temporary table */
into #COGS_TO_DSL
from cogs
where cod_scenario = @SCE_O
and cod_azienda in (select cod_azienda from azienda )
group by
cod_azienda,
cod_scenario,
cod_periodo,
ProdCategoryCode,
ClusterCustomerCode,
DocCurrencyCode,
CompanyCurrencyCode
/* END temporary table */

select
cod_azienda,
cod_scenario,
p.cod_periodo,
Dest1,
Dest2,
Dest3,
CompanyCurrencyCode CompanyCurrencyCode,
DocCurrencyCode DocCurrencyCode,
sum( QTY_L ) QTY_L,
sum( QTY_A ) QTY_A,
sum( AV_P1_M2) AV_P1_M2,
sum( P25100_M2 ) P25100_M2,
sum( P15110_M2 ) P15110_M2,
sum( CPR_S ) CPR_S,
sum( B00001 ) B00001,
sum( VAL_F_A ) VAL_F_A,
sum( VAL_F_S ) VAL_F_S,
sum( VAL_F ) VAL_F,
SUM( VAL_L_S) VAL_L_S,
sum( P25200_M2 ) P25200_M2
into #COGS_TO_DSL_YTD
from #COGS_TO_DSL t, periodo p
where t.cod_periodo <= p.cod_periodo
and cod_scenario = @SCE_O
group by
```

```
cod_azienda,
cod_scenario,
p.cod_periodo,
dest1,
dest2,
dest3,
DocCurrencyCode,
CompanyCurrencyCode
--/* FINE PROGRESSIVIZZAZIONE */

delete from dati_saldi_lordi
where cod_scenario = @SCE_O
and provenienza = @provenienza

/* START: insert into dati_Saldi_lordi */
--50225 SalesPeople Costs
insert into dati_saldi_lordi(OID_DATI_SALDI_LORDI, COD_SCENARIO, COD_PERIODO, COD_AZIENDA, COD_
CONTO, COD_DEST1, COD_DEST2, COD_DEST3, COD_CATEGORIA, IMPORTO, COD_VALUTA, IMPORTO_VALUTA_
ORIGINARIA, COD_VALUTA_ORIGINARIA, PROVENIENZA, USERUPD, DATEUPD, NOTE)
select
newid() OID_DATI_SALDI_LORDI,
t.cod_scenario COD_SCENARIO,
t.cod_periodo COD_PERIODO,
cod_azienda COD_AZIENDA,
'50225' COD_CONTO,
Dest1,
Dest2,
Dest3,
'S'  COD_CATEGORIA,
P25100_M2 IMPORTO,
COMPANYCURRENCYCODE COD_VALUTA,
P25100_M2 IMPORTO_VALUTA_ORIGINARIA,
COMPANYCURRENCYCODE COD_VALUTA_ORIGINARIA,
@PROVENIENZA PROVENIENZA,
'Young' USERUPD,
getdate() DATEUPD,
null NOTE
from #COGS_TO_DSL_YTD t
where P25100_M2 <> 0
and cod_scenario = @SCE_o
and Dest2 <> 'POP'

-- 50220 - Advertising and Promotion
insert into dati_saldi_lordi(OID_DATI_SALDI_LORDI, COD_SCENARIO, COD_PERIODO, COD_AZIENDA, COD_
CONTO, COD_DEST1, COD_DEST2, COD_DEST3, COD_CATEGORIA, IMPORTO, COD_VALUTA, IMPORTO_VALUTA_
ORIGINARIA, COD_VALUTA_ORIGINARIA, PROVENIENZA, USERUPD, DATEUPD, NOTE)
select
newid() OID_DATI_SALDI_LORDI,
t.cod_scenario COD_SCENARIO,
t.cod_periodo COD_PERIODO,
```

```
cod_azienda COD_AZIENDA,
'50220' COD_CONTO,
Dest1,
Dest2,
Dest3,
'S' COD_CATEGORIA,
P15110_M2 IMPORTO,
COMPANYCURRENCYCODE COD_VALUTA,
P15110_M2 IMPORTO_VALUTA_ORIGINARIA,
COMPANYCURRENCYCODE COD_VALUTA_ORIGINARIA,
@PROVENIENZA PROVENIENZA,
'Young' USERUPD,
getdate() DATEUPD,
null NOTE
from #COGS_TO_DSL_YTD t
where P15110_M2 <> 0
AND Dest2 <> 'POP'
AND T.COD_SCENARIO = @SCE_O
end
```

Database objects

What is it for

In CCH TagetikDatabase Objects are used to:

- Create non-standard database objects on the application database to implement particular customer requirements, such as:
  - custom views to perform import via ETL;
  - stored procedures and functions to implement specific data processes that cannot be accomplished otherwise;
  - customised queries to carry out, via Tagetik's query analyzer functionalities, direct queries on the database or for the creation of staging tables to be used in a stored procedure or for the creation of queries to be used as 'form queries';
- Display the archive of all non-standard objects that can be transferred from one database to another, together with the rest of the parameterisation, when importing/exporting the database.

Operations performed on Database Objects are always recorded within the audit trail.

***IMPORTANT**: Since these functions can operate directly on the database, they are not available to all users by default. Only users with the 'Entity data processing: calculation logics etc.' module enabled in their role can display them*

Deploy

Actions (⋮) > Deploy

Necessary to physically create a non- standard database object on the database. It is only visible if the body of the database object has been defined for the database type linked to the application. This operation creates the database object; if an object with the same name already exists, it is first deleted and then recreated.

If errors occur during deployment:

- When using an MSSQL, PostgreSQL, or SAP HANA® database, the system detects errors and reports them to the user;
- In the case of an ORACLE database, the system cannot detect them, so they are only reported to the user at the time the object is used.

***IMPORTANT**: Database objects of Query type can only be deployed. In this case, the system executes the query statements.*

***IMPORTANT**: Database objects of Query type containing Select statements are not executed by the system. Failure to execute Select queries is tracked in the audit .*

Execution

Utility > Run

If a database object of procedure or function type has already been deployed, Tagetik allows you to run it directly from the application.

In order for a procedure or function to be run directly from the application, it must have a name written in capital letters beginning with CPM_SP_. For the requirements that such an object must meet in order to be considered a Tagetik data processing and thus be executed directly by the application, see **Stored procedure**

Database Objects page

Admin | <application database>

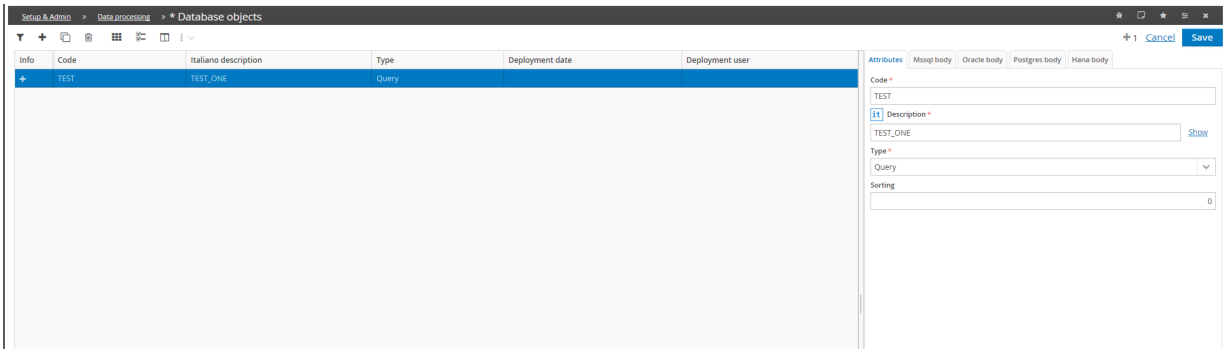
Data processing > Database Objects

Data processing > Database Objects

Purpose of the page

From this page you can display and create Database Objects.

What it looks like



Element	Description
To	Action buttons
B	Database Objects list
C	Configuration tab

Actions on this page

Element	Description
	This filters the data displayed on the page.
	Adds a row to insert a new Database Object.
	Duplicates a Database Object with the same characteristics as the selected one, except for the code.
	Deletes the element contained in the selected row.
	Enables/Disables the grid modify function. This allows you to change the characteristics of the selected elements directly in the table, and to hide the configuration tabs.
	Enables multi-row selection to allow mass changes.
	Selects the columns to be shown/hidden in the table and those to be locked when scrolling the table.
	• **Deployment**: enables the object deployment • (Only with Procedure and Function objects) **Run**: executes the operation.

Definition of a database object

Data processing > Database Objects

Data processing > Database Objects

Attributes tab

Please find below the most important fields.

Field	Description
Code	Identifies the object. For view, function, and procedure objects, the code also corresponds to the name used to create the object in the database
Description	Shows the description of the object in all languages managed by CCH Tagetik.
type	Enables the management of the following database object types: ◦ ***Functions*** ◦ ***Procedure*** ◦ ***Query*** (The definition of a query object may comprise several statements) ◦ ***View***
Order	Indicates the order in which the objects must be executed when the user has selected more than one.

Syntax Tabs

These tabs contain the Mssql, Oracle, Postgres Hana, Other Body syntax management settings, which can be accessed by double-clicking on the database object.

Field	Description
MSSQL body	It contains the instructions defined with a Mssql syntax that will be used in the case of a primary MsSql connection.
Oracle body	It contains the instructions defined with an Oracle syntax that will be used in the case of a primary Oracle connection.
Postgres body	It contains the instructions defined with a Postgres syntax that will be used in the case of a primary connection of Postgres type. If a database object of procedure or function type is being created, both the object name and the parameters must be enclosed in double quotes.
Hana body	It contains the instructions defined with a Hana syntax which will be used in the case of a primary connection of Hana type.
Other Body	Contains the instructions that will be used in case of a connection to a secondary database, using the syntax of the secondary database. ***IMPORTANT**: Instructions can only be defined in the **Other Body** field for database objects of Query type. If instructions are defined for database objects of type procedure, view, or function, the system returns a validation error upon saving.*

***IMPORTANT**: When creating a database object of type procedure, view, or function, the system checks during saving that the name used in the syntax matches the identifier code of the database object.*

***IMPORTANT**: The instruction is encoded according to the CCH Tagetikstandard before being saved to ensure portability to different databases; chr(13)+chr(10) character combinations are removed from the instruction because they cause malfunctions during deployment. For example, they prevent the use of blank lines or '--' characters when entering comments.*

Technical notes on connections

Type of connection	Description
Primary connection	Connection defined as primary in the configuration file of the application server standalone.xml.
Secondary connection	Connection defined as secondary in the configuration file of the application server standalone.xml.
Tertiary connection	Schemaless connection defined as primary or secondary in the configuration file of the standalone.xml application server.

***IMPORTANT**: Secondary connections are marked with prefix (S) and tertiary connections with prefix (T).*

Balancing

The Balancing is a data processing that allows the system to automatically balance the unbalanced amounts below the threshold.

The analysed unbalanced amounts are those detected by the diagnostic of equality control groups (that is, with “=” operator) and by IC capacity check (for details on the diagnostic, see the relevant section Data processing > Diagnostic).

The balancing is enabled on the process, in the Data processing section:

- on Entities. The balancing operates on data inserted on original scenarios related to the Entities: amounts and Entity journals (both contributor and consolidator)
- on Group. The balancing includes group consolidation data present on original scenarios: consolidation journals on original scenarios
- on Consolidation Area. The balancing operates on all data present on consolidation scenarios: consolidation journals on consolidation scenarios (consolidable amount type)

The balancing is not performed at the contributor node level.

The threshold taken into consideration when deciding whether or not to perform the balancing is the one set up in the validation rules for the equality control groups and for the accounts: Original scenario balancing threshold and Consolidation scenario balancing threshold (the latter is only available for control groups).

If the Currencies for thresholds field is also defined, then the threshold amount is understood to be expressed in the specified currency and for the comparison it must be converted into the currency of the Entities / journals to which it is applied. The Fx rate used for the conversion is the Final Fx rate. If it is not defined, then the amount of the threshold is understood to be expressed in the currency of the entities / journals to which it is applied.

If the thresholds are not defined, then the system will not perform any balancing.

The balancing can be run:

- from the contributor cockpit. Once you have selected one or more Entities or one or more nodes to be processed, it is necessary to select the "Entity data processing" button from the menu on the right and then select "Balancing" in the data processing tree available under the "Entity data processing" node. This data processing is visible only if the Balancing data processing has been enabled for the selected process. Moreover, if you want to perform only the balancing of the fiscal year's result, you can select the "Balancing only the result" option

- from Consolidator Cockpit. Once one or more nodes to be processed have been selected, it is necessary to select the "Consolidation data processing", Standard or Custom, button from the menu on the right. If selected, the Standard data processes will be performed automatically, including the Balancing, if they are enabled on the process. If you select the Custom data processes, a window will pop up allowing you to run the Entity, group and area data processes, including the Balancing, if enabled on the process
- from Workflow. Inside the workflow of a contributor or mixed step, it is possible to insert a "Data processing by Entity" task having the Balancing among the data processes to be run. Whereas, inside the consolidator or mixed steps, it is possible to insert a "Consolidation" task, by choosing to run the Entity and / or group and / or area data processing
- from Job. Inside à Job, it is possible to insert a "Data processing by Entity" task having the Balancing among the data processes to be run. Moreover, it is possible to insert a "Consolidation" task, by choosing to run the Entity and / or group and / or area data processing

Control groups balancing

First, the system determines the order with which the various control groups have to be balanced. This is necessary as this data processing balances the child accounts. Thus, if an account is child of a control group and parent of another control group, then the control group in which it is a child must be balanced first.

The control groups that are balanced are those with operator "=" and having the "Balance automatically" option enabled. As already mentioned, they also need to have been correctly parameterized in the validation rules for the equality control groups, by defining the balancing threshold.

 The control groups with dependence on dimensions for account / custom dimension are not balanced.

First, the data processing removes the old balancing rows (namely, those with PROC_BALANCING origin on the original scenarios and CONS_BALANCING on the consolidation scenarios) and then inserts the new ones, for each "level" of calculated control groups.

To determine the rows on which to balance the amounts, it runs the diagnostic of the control groups that, when run by the Balancing, returns only errors below the balancing threshold.

The balancing is performed for all data types (gross, intercompany, net).

Balancing rows are inserted to balance the errors diagnosed at the level of category and category node, custom dimension, and custom dimension node and, for the IC control groups, Ctp entity and Ctp entity node.

 Neither errors diagnosed at category level on journal categories, nor errors diagnosed at level of category node containing only journal categories will be balanced.

When the row to be balanced is determined (for example, if the control group reports an unbalanced amount at the category node level, the balanced amount may be inserted on any category belonging to the node), at comparable account, the one with the highest amount is taken. If no row is found, the system will generate one.

IC capacity balancing

The balancing of the IC capacity errors is performed for the accounts for which the IC capacity check is equal to "Check=". As already mentioned, they also need to have been correctly parametrized in the validation rules for the accounts, by defining the balancing threshold.

First, the data processing deletes the old balancing rows (those with PROC_BALANCING origin) and then inserts the new ones.

To determine the rows on which to balance the amounts, it runs the diagnostic of the IC capacity that, when run by the Balancing, returns only errors below the balancing threshold.

Balancing rows are inserted to balance the errors diagnosed at the level of category and category node, custom dimension and custom dimension node.

 Neither errors diagnosed at category level on journal categories, nor errors diagnosed at level of category node containing only journal categories will be balanced.

When the row to be balanced is determined (for example, if the IC capacity check reports an unbalanced amount at the category node level, the balanced amount may be inserted on any category belonging to the node), at comparable account, the one with the highest amount is taken. If no row is found, the system will generate one.

 To distinguish the balancing rows on control groups from those of IC capacity, the “Notes” field is filled in inside the data tables by specifying the control group for which the balancing was performed or by specifying the capacity balancing.

Monthly splitting

When it is necessary to define periodic data of one or more accounts from the cumulated data of a specific period, it is possible to use CCH Tagetik's **Monthly splitting** data processing or the **Custom Splitting on Periods** data processing .

The first one (Monthly splitting) is defined for each single account and allows you to split per month from the last period of a scenario, on all previous periods of the Scenario, while the second one (Custom Splitting on Periods) allows you to define in much more detail the data to split, the Scenarios /Periods on which to split the data and the driver to use

In order to use the **Monthly splitting**, it is necessary to define some rules which are not linked to a specific process nor to an entity, but to the account to which the amount to be split belongs.

The **monthly splitting** rules allow the user to generate entries on the target periods.

Monthly splitting rules

The monthly splitting rules can be defined for the "P&L", "Other variations" accounts or for the "Variations" accounts of "BS" or "Other stocks" nature.

For the other account types / natures it is not possible to define the monthly splitting rules since they are always treated as cumulated accounts.

In order to define a monthly splitting rule, it is necessary to:

- access the accounts elements table (accessible from the administrator user web interface by following the path *Setup & Admin* > *Data Model* > *Dimensions* > *Account* > *Elements*) *Navigation panel*> *Dimensions* > *Account* > *Elements*) and select the account for which the the monthly splitting rule has to be defined;
- place the cursor on the "**Monthly splitting rules**" tab

The **Monthly splitting type** property specifies how the cumulated amount has to be split. Possible splitting types:

- *No*. The system does not run any monthly splitting
- *Flat*. The account's value is shared equally on each period (e.g. the amount of December is equally split on 12 months)
- *Coefficient 445*. The account's value is distributed according to a coefficient which follows the 4,4,5 progression of the weeks during the months. Considering that in one year there are 52 weeks and that in each quarter there are 13 weeks, the "weights" of the weeks in the quarter will be 4/52 8/52 13/52 17/52 .... 47/52 52/52

- *Coefficient 454*. The value of the account is distributed according to a coefficient which follows the 4, 5, 4 progression of the weeks during the months. Considering that in one year there are 52 weeks and that in each quarter there are 13 weeks, the "weights" of the weeks in the quarter will be 4/52 9/52 13/52 17/52 .... 48/52 52/52
- *Coefficient 544*. The value of the account is distributed according to a coefficient which follows the 5,4,4 progression of the weeks during the months. Considering that in one year there are 52 weeks and that in each quarter there are 13 weeks, the "weights" of the weeks in the quarter will be 5/52 9/52 13/52 18/52 .... 48/52 52/52

   The rules using the coefficients 445, 454 and 544 can be applied only if the monthly splitting is done on 12 periods.

- *Adv. management* The system splits per month the value of an account using a driver. The driver to use is defined according to the following information

Reference:

the **Reference account**. Account used to calculate the monthly splitting driver. If none is specified, the system uses the account for which the monthly splitting rule is being defined.

As an alternative to a single account, it is possible to indicate an entire hierarchy of accounts (**Reference accounts hierarchy**)

the **reference scenario**. The definition of a reference scenario is required when the scenario to use for the calculation is different from the data processing scenario. The reference scenario for the driver is parametric, in other words determined by the system during the rule execution. The available parametric values are:

- PREV. The reference scenario for the driver is the one before the processed scenario
- NEXT. The reference scenario for the driver is the one after the processed scenario
- REF1...REF5. The reference scenario for the driver is the 1...5 reference scenario related to the processed scenario

a possible filter on the managed **custom dimensions 1...5**, on the **category**, on the**Ctp entity** and on the **Ctp custom dimension 2** to apply when defining the monthly splitting driver

Process setup for the "Monthly splitting" management

In order to enable the "Monthly splitting" data processing for a given process the user needs to:

- open the process elements table accessible from the admin user interface from *Setup & Admin > Processes > select the process to parametrize* and then *Actions (⋮) > Define process.* ; *Navigation panel> Processes& Workflows > Process > select the process to parametrize* and then *Actions (⋮) > Define process.* ;
- in the **Data Processing** area, enable the **Others** > **Monthly splitting by Account** data processing
- save

“Monthly splitting” data processing

Once the parametrization of the monthly splitting rules is complete, it is possible to run the monthly splitting data processing. This data processing can be run only if its execution has been enabled for the selected process (for further details see chapter **Other** data processing of the manual on the process parametrization).

The system calculates data to be split per month according to the information specified in the initial filter:

- the **last period** of the scenario to run. In order to run the monthly splitting it is necessary to "launch" the data processing on the last open period of a scenario: usually it is run on the period 12 but, if the periods represent the weeks, it is run on the period 52. If several scenarios/periods are specified in the initial filter, the system puts together the monthly splitting periods: for example, scenarios 2010C/12 and 2010B/12 will be run together if they have the same target periods
- the **entities**
- other possible **optional filters**
- the **accounts** to which a monthly splitting rule is linked

The data processing "operates" only on the gross amounts and on the IC amounts and takes into account all origins but those generated:

- by a previous monthly splitting (PROC_MONTHLY_SPLIT_NEW)
- by the preconsolidation reclassifications (PROC_RECLASSIFICATION)
- by the depreciation calculation (PROC_DEPRECIATION)
- by the basic calculations (PROC_CALCULATED_BASIC)
- by the deferred taxes calculation (PROC_DEFERRED_TAXES)
- by the balancing (PROC_BALANCING)
- by the equity reclassification (PROC_NETEQUITY_RECL)
- by the application of calculation logics (PROC_CALCULATED)
- by the deconsolidation (PROC_DECONSOLIDATION3)

The system applies the monthly splitting rule to each data to be split per month, as follows:

- if the "Monthly splitting type" defined for the account is "NO", then data is not split per month
- if the "Monthly splitting type" defined for the account is "simple" type, then data is split per month by applying the constant coefficient specified in the rule
- if the "Monthly splitting type" defined for the account is “advanced”, then the system calculates the monthly splitting drivers according to the information specified in the rule. The calculated drivers are used by the system to calculate the weights to apply to data to be split per month. Once the weights have been calculated, they can be applied to data to split per month.

Before entering the monthly splitting entries, the system removes the existing ones.

Run "Monthly splitting"

The monthly splitting can be run:

- from the contributor user's **process cockpit**. After selecting one or more entities or one or more nodes, it is necessary to select the **Entity data processing** button from the *Actions* menu and then select **Monthly splitting** in the data processing tree under the "Entity data processing" node. This data processing is visible only if the Monthly splitting data processing is enabled for the selected process.
- **from a data entry form**
- **from a workflow task**
- **from a job task**

From form

The "Monthly splitting" can be run also from a data entry form as a single data processing as a part of the Entity data processes enabled for a given process.

Single data processing

Open the form in Design mode, click on  **Data processing** on the Design ribbon bar (or right click on the Open Items panel).

The system displays a window in which the user will select the data processing to run: select **Monthly splitting** and click on **Next** to set up the parameters and define the data processing execution area on the Entity, Scenario, Period and Category dimension and on the Custom dimensions. The parameters in bold are mandatory.

The parameters definition modes are:

- **Automatic**. The system retrieves automatically the parameter's value from a form filter, a tab filter, a matrix filter, a row/column filter. If the filter is defined by a parameter (therefore its value is chosen by the user at run-time) the choice of the data processing value will be up to the user
- **Excel cell**. The system retrieves the parameter's value from an Excel cell. The user must specify the cell containing the value by using the syntax ${H30} (cell H30).
- **Insert**. The user must enter the parameter's value into the corresponding text box
- **Select**. The user must select the parameter's value from a list of possible values
- **Cell field**. The system retrieves the parameter's value from a cell field.

After setting up the parameters, click on "Next" to configure a series of options which allow the system to understand how the data processing has to be run within the form (e.g. if it has to be run upon saving or manually, for further details see **Data processing**).

Eventually it is necessary to assign a code and a description to the data processing that will be inserted in the form.

Entity data processing

The monthly splitting belongs to the Entity data processing and can be run also as a part of such data processing just by selecting it from the *Entity data processing* selection window.

In this case, the following parameters have to be set up:

- the original **scenario / period** on which the data processing has to be run;
- the **entities** to which data to run are allocated;
- **Disable impacted**. By default this option is disabled.

 In this case, it is not possible either to select the accounts to split per month or specify filters on the other dimensions..

From workflow

Tagetik allows the user to run the monthly splitting also from a workflow model task. Also in this case, it can be run in two ways, namely either as a part of the entity data processing or individually.

 In this case, the process, the period and the entities to run are pre configured in the process cockpit.

Single data processing

Open the Workflow definition page in edit mode, define a new Task by dragging the *Monthly splitting* icon, *Other data processing* tab into the workflow area.

Besides the regular options of any Workflow Task, in the Parameters area it is necessary to specify the data processing execution area on the Account, Category and Custom dimensions.

Entity data processing

Define a new Task by selecting it from the top bar, in the *Core data processing* tab, *Entity Data processing*.

Select **Monthly Splitting** in the parameters.

 In this case, it is not possible either to select the accounts to split per month or to specify filters on the other dimensions.

From job

Tagetik allows the user to run the monthly splitting also from a job task. Also in this case, it can be run in two ways, namely either as a part of the entity data processing or individually.

Single data processing

Open the Job definition page in edit mode, define a new Task by dragging the *Monthly splitting* icon, *Other data processing* tab;

Besides the regular options of any Job Task, in the Parameters area it is necessary to specify the Process and the data processing execution area on the Account, Entity, Scenario, Period, Category and Custom dimensions.

By selecting the **All** option (Entity and Scenario/Period) the values of the corresponding parameter will be determined by the system when running the job and they will coincide with all the available values, at that time, for that specific parameter.

Entity data processing

Define a new Task by selecting it from the top bar, in the Core data processing tab, *Entity data processing*.

In the parameters area, it is necessary to specify the process, the Scenarios / Periods and the Entities to run and then select the **Monthly splitting** data processing.

In this case, it is not possible either to select the accounts to be split per month or specify filter s on the other dimensions.

By selecting the **All** option (Entity and Scenario/Period) the values of the corresponding parameter will be determined by the system when running the job and they will coincide with all the available values, at that time, for that specific parameter.

Job

A job is a set of activities, which allows several different types of data processing to be performed in a single launch.

The different activities are represented as tasks in a workflow and will be executed by the system according to the scheme by which they were designed. Depending on the objectives, data processes may be specified sequentially, in parallel or in a mixed manner.

All the most common data processes handled by Tagetik can be entered into a job.

Creating a job can be useful when there is a need to implement multiple activities, even of a different nature, to be launched automatically with fixed parameters.

Define Job

The job list is accessible from the administrator user's application home page by following the path *Data processing > Jobs.*

Each job is identified by:

- a **code**
- a **description**.

Moreover, it is possible to set up the automatic sending of an e-mail at the end of the execution. When the "send mail" option is enabled, the subject of the message, its text and the list of recipients can be defined for each job.

The *Advanced* section includes the *DTP Task: ask runtime parameters* option. This option allows you to decide whether or not to show the runtime parameter request window for the DTP task when running the task.

Once the header has been created, the graph configuration window can be opened from the *Actions* menu, where the initial node is shown by default.

To edit the graph, click on the *Edit* button.

The node types available in a job can be grouped into:

- **start** The task of this type is labelled 'Start' and indicates the beginning of the graph. The system creates it automatically.
- **one or more Data processing tasks**. The Data rocessing-type tasks can be run sequentially or in parallel and have a code label to identify the data processing the task represents
- **join**. Tasks of this type always indicate the end of a data processing group. Users can enter tasks of this type manually.

To add nodes to the graph it is necessary to choose the category and drag & drop the elements from the top of the view to the graph area.

To delete a node instead, you can select the node and press the *Delete* key or right-click on it and then select *Delete*.

The data processes are grouped together:

- **Data Integration**. Job from application server for loading elements lists or values:
  - Run ETL
  - Loading to Analytical Workspace: loading via the Quick Data Loader
  - Update Financial Workspace from Analytical Workspace
  - DTP: allows running a Data Transformation operation on Analytical Workspace
- **Core Data Processing:**
  - Preliminary of Entity and Group
  - Preliminary on Consolidation area

    - Entity Data Processing
    - Consolidation
    - Calculation logics
    - Allocations and Closing: launches the Allocations and Closing data processing
    - Cash Flow Planning
- **Validations:**
    - Diagnostic log update
- **Other data processing**: data processing not included in other groups including the execution of stored procedures and the processing of already defined cubes:
    - *Monthly splitting* : launches the *Monthly splitting* data processing
    - Custom Splitting on Periods
    - Create Forecast
    - Algebraic Operations: see *algebraic operations*
    - Stored Procedure
    - Process Cube
    - Form - Report list
    - Tower Watson integration protocol
    - Towers Watson integration: data cleaner
    - Banking IC Spreading
    - Job
- **Other activities**
    - Deployment: see Deployment
    - Update status based on data (Process Cockpit)
    - Lock/Unlock editability: Locks/Unlocks editability on process cockpit.
    - Purge Closing entries: allows deleting the content of the used tables from the allocation and closing module
    - Send mail
    - Copy Consolidation
    - Copy amounts and journals
    - C.O. - Replace attachments : Allows the automated replacement of files attached to a document without the end user having to manually re-attach files that have changed.
    - C.O. - Create Snapshots automatically : makes it possible to make a copy of a document or parts of it
    - Export Application DB
    - ML Training: allows you to add and configure a task for a Machine Learning Train data processing to be launched via Job
    - ML Execute: allows you to add and configure a task for a Machine Learning Execute data processing to be launched via Job

To add a branch between two nodes, it is necessary to drag & drop from the start node to the end node.

This allows the creation of a workflow of tasks to be executed automatically. The graph can be structured and modified through commands that add, remove or join branches and nodes.

For each task, it is necessary to specify a description for it, pre-populated with the description of the type of task chosen; the launch parameters are then configured. This is a basic activity as each data processing will be performed using the specified filters. To set the parameters, simply click on the desired task. The system populates the detail panel that suggests all possible parameterisable dimensions. The elements suggested vary according to the type of data processing chosen.

In edit mode, it is also possible to ask the system to adjust the graph layout automatically, apply a zoom, undo changes or save progress.

It is possible to save a job even if all of its task parameters have not been set. This mechanism makes a Job a kind of template from which specific instances can be created during execution or scheduling.

Defining a Job with Parameters

When defining a Job, it is possible to use I parametri to define the Process and the Scenario/Period, instead of setting an explicit value.

This functionality allows you to create a job with several tasks all using the same set of *Parameters*. For example, it is possible to create a Job consisting of an ETL Launch task and a Consolidation task that use the same Process parameter.

The tasks that currently allow the use of *Parameters* are as follows:

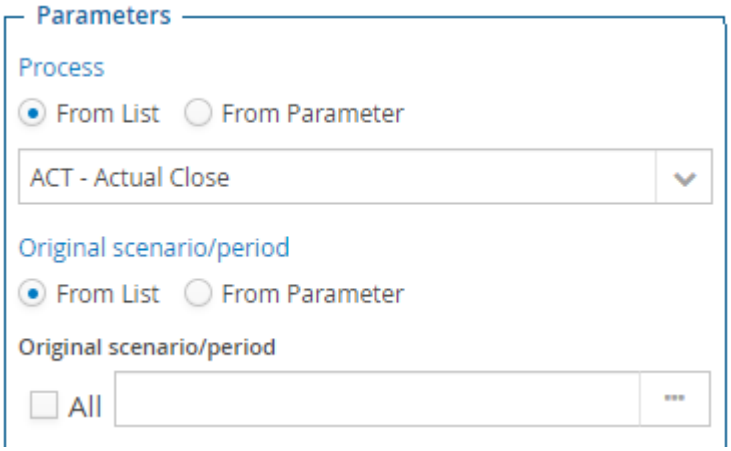
- Run ETL
- Preliminary on Consolidation area
- Entity Data Processing
- Consolidation
- Diagnostic log update
- Preliminary data processing on Original scenarios
- Calculation logics
- Banking IC Spreading
- Stored Procedure

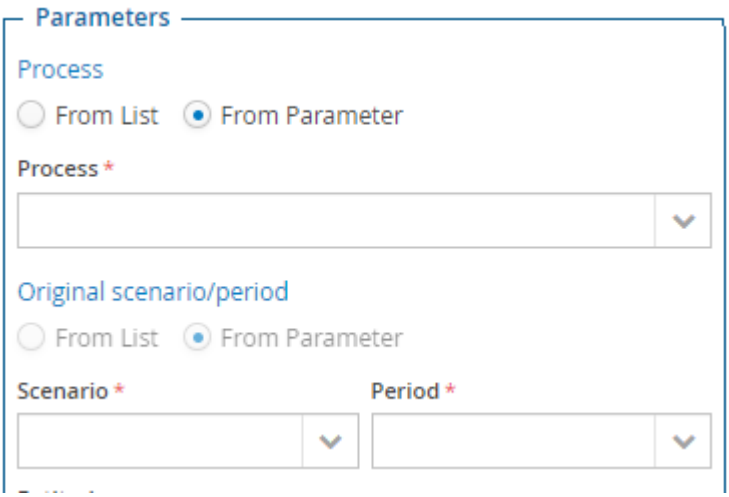
For these tasks, it is possible to specify the use of a *Parameter* for:

- Process
  - "From Elements List". Possible values are all the processes allowed for the task.
  - "From Parameter". Possible values are all the *Parameters* of Process type.
- Scenario / Period
  - "From Elements List". Possible values are all scenarios/periods allowed for the task (scenarios/periods are filtered by the previously chosen process).
  - "From Parameter". The possible values are all dimensional *Parameters* of type Scenario and all dimensional *Parameters* of type Period.

When defining the Process 'From Parameter', the system forces the user to also define the Scenario / Period 'From Parameter' in case the parameter is mandatory for the launch.

For each job, not more than one parametric Process can be used. As of now, the check is only carried out at runtime as the job may consist of nested jobs that change over time.





Running and scheduling a job

To run or schedule a job, it is necessary to click on the *Review and Launch* button from the read-only view of its graph.

In the next step, the user must set all the parameters of each task and finally click on *Launch* or *Schedule*, depending on the operation to be performed.

Running a Job with Parameters

When you click on the *Review and Launch* action menu and then select a job with *Parameters*, the system displays a window for the definition of the *Parameters* contained in the job.

Once the *Parameter* values have been defined, the system opens the window for launching or scheduling the Job. From this window all task launch values can be rechecked and the Job can be scheduled. When switching from the *Parameter* Definition window to this window, all null values are automatically deleted. In this window, it is possible to display the values used during the launch of the job for each task.

If two different Process *Parameters* were used in the Job, the system will display an error when opening the launch window; in this case the user will have to change the job in order to be able to run it.

Export to Excel

In the *Show Graph* window and in the *Review and Launch* window, it is possible to run the ***Export to Excel*** function by clicking the icon 

.

Loading task on Analytical Workspace

The Data Integration tab contains the loading task on Analytical Workspace that can be drag-and-dropped to the chart area when you are in edit mode.

The task configuration parameters are:

- **process**. This field is mandatory
- **scenario**. This field is not editable with the single submission process; it is mandatory and single selection with the scenario / period submission process
- **period**. This field is not editable with the single submission process; it is mandatory and single selection with the scenario / period submission process
- **entity**. This field is mandatory and multi-selection; the lookup allows you to select entities or nodes and in this case the field keeps the list of the related entities
- **Analytical workspace**. This field is mandatory and single selection
- **Dataset**. This field is mandatory and single selection; you can choose a code related to the selected workflow

- **Datasource**. This field is mandatory and single selection; you can choose a code related to the selected workflow and dataset. Only the database and analytical workspace datasources can be selected

Data Transformation Package Task

The Data Integration tab contains the Data Transformation Package run task that can be dragged to the graph area when the Edit mode is on.

The task configuration parameters are:

- **process.** the field is mandatory; if it is set with an Multidimesional process, it will only be possible to launch the task by selecting Entity or node Entity (launching from jobs with a dimension other than Entity is currently not supported) and will take place in 'Context independent run' mode.
- **scenario**. the field is not editable in case of single submission process; on the contrary, it is mandatory and single-selection in case of scenario / period submission processes
- **period**. the field is not editable in case of single submission process; on the contrary, it is mandatory and single-selection in case of scenario / period submission processes
- **entity**. the field is single-selection; it can only be filled alternatively to the hierarchy Entity and node fields
- **Entity hierarchy**. the field is in single selection; it can only be filled alternatively to the Entity field
- **node Entity**. the field is in single selection; it can only be filled alternatively to the Entity field; it is possible to choose a code associated with the chosen hierarchy
- **Analytical workspace**. the field is mandatory and single-selection
- **Data Transformation Package**. the field is mandatory and single-selection; you can choose a code for the selected workflow
- **Context independent run.** the option is selected and cannot be changed in the case of Muldimensional processes.

Tasks of this type support the use of parameters required at runtime. The parameter request window will only be displayed if the option *DTP Task: ask runtime parameters* is selected on the job.


The option *DTP Task: ask runtime parameters* is not taken into account when the job is run from cockpit and API.

ML Training and ML Execute tasks

The Other Tasks tab contains tasks for the execution of Machine Learning Training and Execute data processing. Tasks can be dragged into the graph area when editing.

The ML task definition parameters are:

- **description**. this field is mandatory and requires a description of the task being added to the graph.
- **process.** Mandatory field
- **scenario**. in the case of a single submission process, the field is not editable; whereas for scenario/period submission processes, it is mandatory and in single selection
- **period**. in the case of a single submission process, the field is not editable; whereas for scenario/period submission processes, it is mandatory and in single selection
- **Analytical workspace**. the field is mandatory and in single selection
- **entity.** the field is single-selection; it can only be filled alternatively to the hierarchy Entity and node fields
- **Entity hierarchy**. the field is in single selection; it can only be filled alternatively to the Entity field
- **node Entity**. the field is in single selection; it can only be filled alternatively to the Entity field; it is possible to choose a code associated with the chosen hierarchy
- **Analytical workspace**. the field is mandatory and single-selection
- **Advanced Analytics Task**. the field is mandatory and in single selection. It is possible to choose an Advanced Analytics associated with the chosen Analytical workspace

- **Training**. the field is mandatory. It is possible to choose a training associated with the chosen Advanced Analytics Task and the model used in the Advanced Analytics
- The '**Clear output data before the data processing**' option is not mandatory. It is possible to select it if you want the output data to be deleted before the execution of a Machine Learning data processing.

Analytical Workspace loading task on Financial Workspace

The Data integration tab contains the Analytical Workspace loading task on Financial Workspace that can be dragged to the chart area when the Edit mode is on.

The task configuration parameters are:

- **process.** Mandatory field
- **scenario**. the field is not editable in case of single submission process; on the contrary, it is mandatory and single-selection in case of scenario / period submission processes
- **period**. the field is not editable in case of single submission process; on the contrary, it is mandatory and single-selection in case of scenario / period submission processes
- **entity.** the field is mandatory and multi-selection; the lookup allows you to select either entities or nodes and, in this case, the field keeps the list of linked entities.
- **Analytical workspace**. the field is mandatory and single-selection
- **Dataset**. the field is mandatory and single-selection; you can choose a code for the selected workflow

.

IC Spreading Data Processing Task

The Other Data Processing tab contains the IC Spreading Data Processing Run Task that can be dragged to the chart area when the Edit mode is on.

The task configuration parameters are:

- **Process**. the field is mandatory.
- **Original Scenario / Period** The field is mandatory and multi-selection; it is possible to run them all by the specific check-box.
- **Entity** the field is optional and multi-selection; it is possible to run them all by the specific check box.
- **IC Spreading Logic.** The field is optional and multi-selection; it is possible to run them all by the specific check box.

 The IC Spreading function is significant only for the processes with IC matching type including the custom dimension 2.

ETL Run Task

The Data integration tab contains the ETL Run Task that can be dragged to the chart area when the Edit mode is on.

The task configuration parameters are:

- **job**. Mandatory field
- **process.** Mandatory field
- **scenario / period**. Optional field
    - The ETL data processing uses the value of this parameter only if ETL parameters linked to ETL dimensions with Scenario and Period Identity are used in the ETL; for those parameters it is

possible to enable the "Define automatically" option so that they are defined with the input values. If several sets of Scenario / Period are selected in the application Job (E.g.: SCE1|01,SCE2|02), the ETL parameters will be defined one by one (Scenario parameter: SCE1,SCE2 - Period parameter: 01,02).

- **step** the field is optional and multi-selection; it is possible to run them all by the specific check box.
- **account hierarchy.** the field is optional and single-selection.
- **account node.** the field depends on the value of the account hierarchy field. the field is optional and multi-selection.

Export from AW - File format Task

The Data Integration tab contains the Export from AW - File Format run task, which can be dragged into the graph area when editing.

The task configuration parameters are:

- **process**. field is mandatory; it will not be possible to select multiorganizational Processes.
- **scenario**. the field is not editable in case of single submission process; on the contrary, it is mandatory and single-selection in case of scenario / period submission processes
- **period**. the field is not editable in case of single submission process; on the contrary, it is mandatory and single-selection in case of scenario / period submission processes
- **entity**. the field is single-selection; it can only be filled alternatively to the hierarchy Entity and node fields
- **Entity hierarchy**. the field is in single selection; it can only be filled alternatively to the Entity field
- **node Entity**. the field is in single selection; it can only be filled alternatively to the Entity field; it is possible to choose a code associated with the chosen hierarchy
- **Analytical workspace**. the field is mandatory and single-selection
- **Dataset**. the field is mandatory and single-selection; you can choose a code for the selected workflow
- **Typology**. the field is mandatory and single selection; you can pick a typology associated with the chosen dataset
- **Data Target Mapping File**. the field is mandatory and single-selection; you can choose a code for the selected dataset. Selectable DTMs will only be those that do not have a File Location equal to *Download*.

Diagnostic log update Task

The "Validation" tab contains the "Diagnostic log update" task that can be dragged to the chart area when the Edit mode is on.

The task configuration parameters are:

- **Process**. This can be selected from list or from parameter. When it is selected from list, the system displays all active processes, namely those having at least one original scenario / period not stored offline and visible to the user; when it is selected from parameter, then the system displays all parameters of Process type. If you select a process on which the diagnostic is disabled or which does not manage, in the validation rules, the traceability of errors detected by the diagnostic, then the system will return a validation error.
- **Original Scenario/ Period**. This can be selected from list or from parameter. When you select it from list, you can select one or more original scenarios / periods not stored offline and belonging to the selected process.
- **Entity**. The system displays all Entities that belong to the process and to the selected scenarios / periods, which are not Ctp only or consolidator only and are visible to the user.
- **Contributor nodes**. The system displays all nodes to be submitted in the data collection process, which belong to the selected process and are visible to the user.
- **Consolidator nodes**. The system displays all nodes to be consolidated in the consolidation process, which belong to the selected process and are visible to the user.

When creating / editing the job's chart, all parameters are optional.
On the contrary, the following parameters are mandatory at runtime:

- Process
- Original Scenario/Period
- at least one among: Entity, Contributor Nodes and Consolidator Nodes

Send Mail Task

The Other Activities tab contains the Send Mail task that can be dragged to the chart area when the Edit mode is on.
The task configuration parameters are:

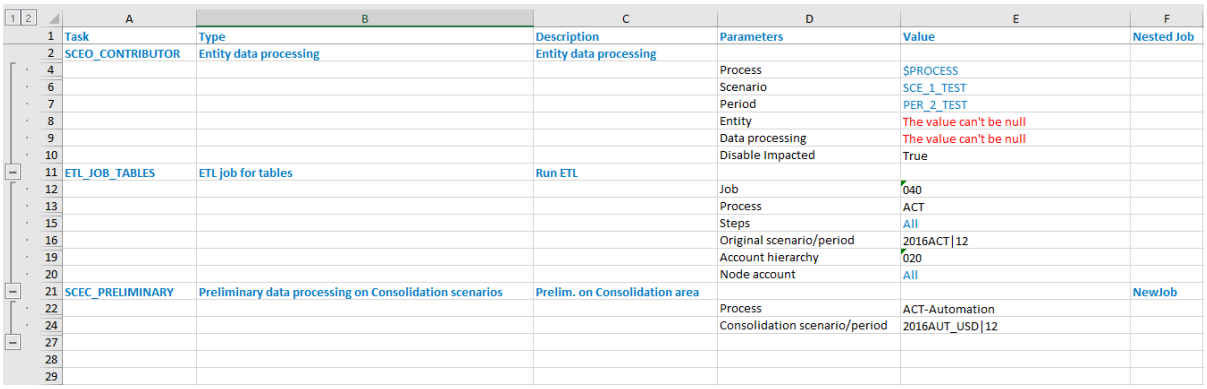
- **table.** the field is mandatory and contains the name of the table from which the system will read the information on the emails to be sent.

The table must contains the following columns:

- "OID": represents a unique ID of the single row of string type. The data type must be VARCHAR(36).
- "DATA_INVIO": date from which the email can be sent. If set to null, the email can be sent immediately. The data type must be TIMESTAMP or an equivalent type.
- "DESTINATARI_TO", "DESTINATARI_CC", "DESTINATARI_BCC": recipients of the email. These can be either email addresses or user names. To enter multiple recipients, use the comma as a separator. The data type must be VARCHAR(MAX) or an equivalent type.
- "OGGETTO": subject of the email that will be sent. The data type must be VARCHAR(MAX) or an equivalent type.
- "FLAG_INVIO": normally it is 0 and turns into 1 after the email is sent. The system ignores all rows where this flag is 1. The data type must be SMALLINT or an equivalent type.
- "MESSAGGIO_HTML": if it is 1, the "MESSAGGIO" field is considered as HTML. The data type must be SMALLINT or an equivalent type.
- "MESSAGGIO": message of the email. The data type must be VARCHAR(MAX) or an equivalent type.

Export to Excel

When you click on the ***export to excel*** icon, the system downloads an excel file (job_example_20210111_1524.xlsx).



	A	B	C	D	E	F
1	Task	Type	Description	Parameters	Value	Nested Job
2	SCEO_CONTRIBUTOR	Entity data processing	Entity data processing			
4				Process	$PROCESS	
6				Scenario	SCE_1_TEST	
7				Period	PER_2_TEST	
8				Entity	The value can't be null	
9				Data processing	The value can't be null	
10				Disable Impacted	True	
11	ETL_JOB_TABLES	ETL job for tables	Run ETL			
12				Job	040	
13				Process	ACT	
15				Steps	All	
16				Original scenario/period	2016ACT|12	
19				Account hierarchy	020	
20				Node account	All	
21	SCEC_PRELIMINARY	Preliminary data processing on Consolidation scenarios	Prelim. on Consolidation area			NewJob
22				Process	ACT-Automation	
24				Consolidation scenario/period	2016AUT_USD|12	
27						
28						
29						

The downloaded file consists of the following columns:

- ***Task***: task's code;
- ***Type***: description of the data processing run in the task;

- ***Description***: description of the task in the language of the current user;
- ***Parameters***: parameter's name;
- ***Value***: parameter's value;
    - if the value of the parameter is set to ***All***, then the value is represented by a blue label;
    - if the parameter's value contains a validation error, then the value is represented by a red label;
    - if the parameter's value is a parametric value, then the value is represented by a blue label ($PROCESS, SCE_1_TEST, PER_2_TEST).
- ***Nested Job***: code of the job contained in the task.

When you click on the Export to Excel icon and the job does not contain any task, the system will display a warning message: ***Please, add a task to the job before launching the export to Excel.***

Additional Tools

CCH® Tagetik provides the following additional tools.

Data Tables Clean-up

This tool allows users to remove incorrect or outdated data from tables, improving the overall quality of the information (see Data Tables Clean-up).

Monthly Splitting data processing

It allows to define periodic data of one or more accounts from the cumulated data of a specific period (see Monthly Splitting data processing).

Store data offline

Utility for transferring historical data, organized by process or by scenario/period (see Store data offline).

Recoding

Procedure that allows changing the codes assigned to master data records (see Recoding).

Snapshot

Functionality that enables users to define specific scenarios called “snapshots” in which data is “frozen” before further modifications (see Snapshot).

Unified SQL Syntax

Functionality that allows the definition of parameterizations independent of the database vendor, making them exportable and importable across different application environments, including heterogeneous databases, without the need to manually adapt the SQL syntax (see Unified SQL Syntax).

Store data offline

The term "Store data offline" refers to the transfer of offline data, divided by process or by scenario/period, from the original tables (called "standard") to equivalent tables (called "offline") containing the same information. This allows to reduce the tables size in order to improve the form and data processing run time.

Data moved to the offline tables will be available for the reporting but not for the data processing or for the data entry. The store data offline operation is reversible and consists in moving back the offline content to the standard tables.

The offline tables are identified by the prefix H_ followed by the original name of the tables.

Store data offline

The Store Data Offline utility can be accessed from the process cockpit (access the Admin home page, click on the **Process** tile, select the desired process > use the *Actions* (⋮) > *Process definition* menu.

The **Business Cycles** section contains the **Manage status** hyperlink

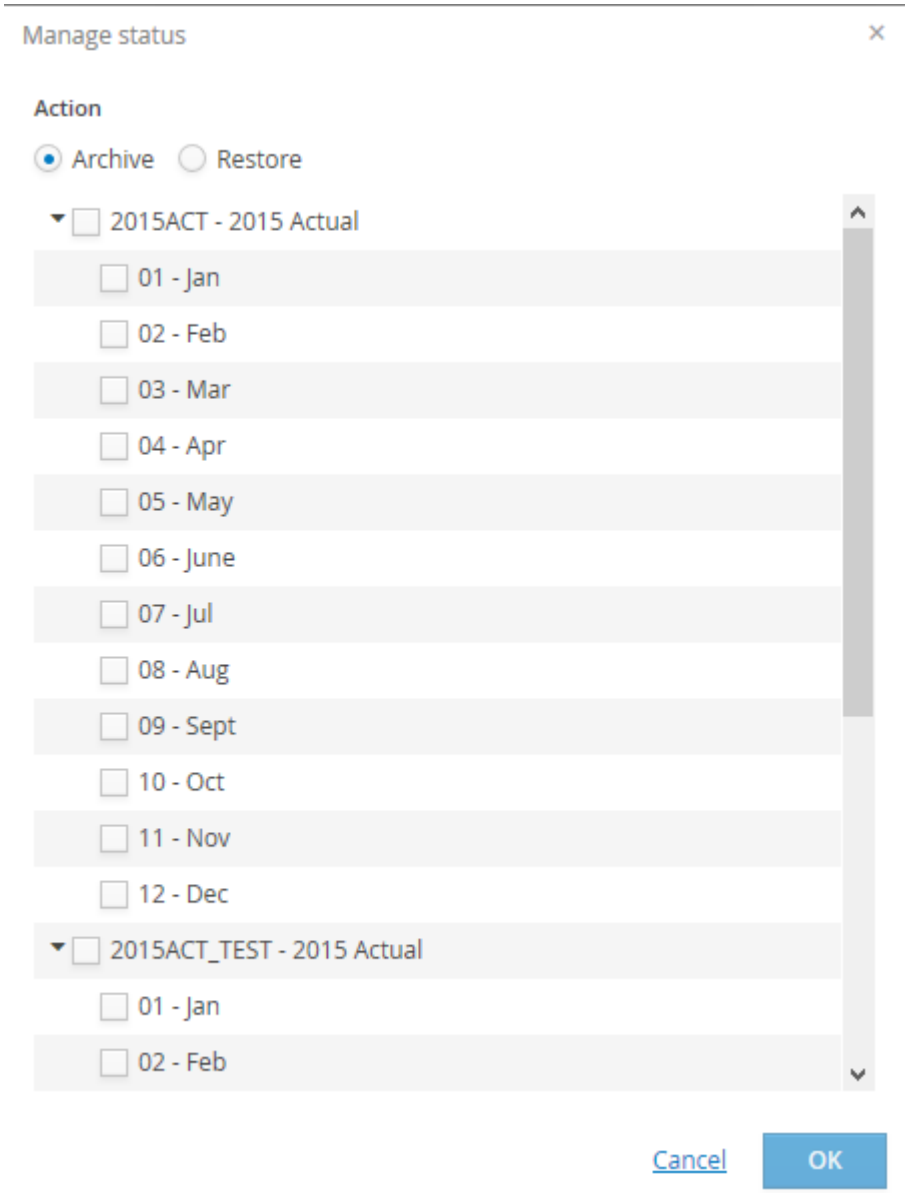
The system opens the Manage Status window; here the user has to select the **Archive** field

Then, the user must select the **scenario**/**period** for which the data must be stored offline.

In case of single submission processes, the data are stored offline and locked for all the scenarios / periods related to the process without the possibility of partialization

In case of processes with submission for scenario/period, besides the process it is necessary to choose the scenario / period.

The system shows only scenarios / periods which are not stored offline.

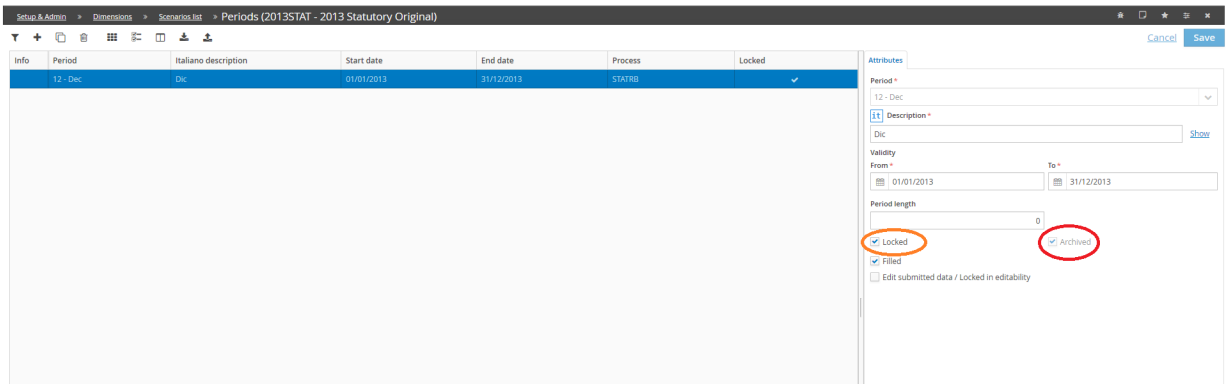


The system locks all the scenarios periods it stores offline.

To store data offline, it is necessary to click on **ok**.

Once the procedure is completed, the system labels as "**Offline**" all the periods (original and consolidation) it has stored offline.

A period can be stored offline from the scenarios elements table, in the scenarios elements table, in the window of the periods related to them.



If all the scenarios periods of a process are stored offline, the process will be locked and stored offline as well.

The **Offline** filed is defined also on the process header if all the underlying scenarios periods have been stored offline

Restore data online

Data can be restored online at any time with the same distinction between single submission processes and processes with submission for scenario / period specified above.

The **Restore data online** utility can be accessed from the processes cockpit.

If the process is single submission or submission for scenarios periods and all the existing periods have been stored offline, a banner saying "The process and all its data are stored offline" with the **Restore** button (Restore)is displayed on the upper part of the process cockpit.

The "Restore Data online" utility does not unlock the scenarios /periods.

Recoding

Recode tables

The Wizard icon in the toolbar of the tables window provides access to various functions according to the specific window. One of these is the **Recoding** function.

The **Recoding** data processing allows you to modify the codes assigned to the tables

To do that, it is necessary to:

- Access the recoding menu
- Create a row for each code to be modified by specifying: the current **Code** and the **New code** that will replace it. The **Primary** flag indicates that the row will prevail in the event of conflict with another row
- Save
- click on **Run**.

In the **Categories** tables it is possible to choose between **Recode Elements** and **Recode nodes**

On the **Scenarios** table it is necessary to choose between **Recode Original Scenarios** and **Recode Consolidation Scenarios**

To recode a node of a Custom dimension, it is necessary to access the windows for the management of the dimension's hierarchies. The wizard icon  in the window's toolbar allows you to recode a node of any hierarchy.

 The recoding for the analytical dimensions is temporarily suspended from SP17.

At runtime, the data processing verifies that:

- you are not trying to recode a user whose code starts with $;
- the new codes do not contain any forbidden character;
- the new codes do not already exist in the table, namely if you are trying to replace a user code with an existing one

 The recoding operation includes the recoding of forms that may contain the element whose code is being changed. This operation can be time consuming. Please note that direct references to the codes contained in the textual scripts internal to the report are not recoded: cell fields, Advanced selection scripts, queries do not support the recoding operation.

Recode Journal number

This data processing can be accessed from *Setup and Admin > Data processing tile > Purge data > Data tables*

Navigation panel> Data processing > Purge data > Data tables

In the window that pops up, create a row for each journal to be recoded and then specify **Scenario**, **Journal number**, **New code** (journal).

The **Recode** flag indicates that the row it refers to would prevail in the event of conflict with another row

 Recoding is not possible on AIH.

Snapshot

When preparing the budget, it is useful to keep several versions of data related to the same original scenario before choosing the final version. For this purpose, Tagetik allows the user to define some particular scenarios named "**snapshot**" where data are "frozen" before making further modifications.

In order to use this utility, the parametrizer user will have to open:

- an "official" original scenario linked to a specific application process on which the official budget version has to be loaded;
- N snapshot scenarios (where N must be <= 999), not necessarily linked to a process, on which the various test versions have to be saved (in order to "freeze" a result obtained before the modifications) specifying that such scenarios are "linked" to the "official" scenario.

In this way:

- all data entries will be performed on the "official" scenario;
- by means of copy programs, data can be saved on the snapshot scenarios or reloaded from a snapshot scenario on the "official" scenario;
- at the end of the operations, final data must be on the official scenario.

Snapshot scenario definition

In order to define a snapshot scenario, it is necessary to:

- access the original scenarios' elements table (accessible from the administrator user web interface by following the path *Setup & Admin > Dimensions tile > Scenarios > Elements*) *Navgation panel> Dimensions > Scenarios > Elements*)and then select the original scenario for which the snapshot scenarios have to be defined;
- select the "*Snapshot*" option from the *Actions* (⋮) window menu

Each snapshot scenario is identified by:

- a version number (**Version number**. This is a unique ID of one of the n versions managed for the same original scenario. It is possible to manage up to 999 versions for the same original scenario
- the snapshot scenario's code (**Snapshot Scenario**). This is an active original scenario created beforehand in the elements table.

 The snapshot scenarios, unlike the "official" original scenario, do not necessarily have to be related to an application process.

Process setup for the "snapshot" management

In order to enable the creation (copy of data from the official scenario to the snapshot scenario) and the restoration (copy of data from the snapshot scenario to the official scenario) of a snapshot for a given process, the user just has to:

- open the processes table accessible from the administrator user web interface by following the path *Setup Admin > Processes > select the process to set up* and using the *Actions (⋮) > Process Definition* path *Navigation panel > Processes & Workflows > Process > select the process to set up* and using the *Actions (⋮) > Process Definition* window menu;
- Select the **Snapshot** data processing from **Data processing**
- The **Requirements** (⚙) button allows the user to specify the dimensions for which the snapshots will be managed, that is:

Run by time dimension with which the snapshot has to be run:

- *by process*
- *by scenario* includes all the periods of the specified scenario
- *by scenario/period* includes one period of one scenario

Dimensions on which to filter the snapshot is run on a partialized set of data

- **Snapshot by Entity**. The creation or the restoration of the snapshot is partialized by entity, or rather it is possible to run the snapshot of only the data of the entities selected upon creation or restoration;
- **Category**. The creation or the restoration of the snapshot is partialized by category, or rather it is possible to run the snapshot of only the data of the categories selected upon creation or restoration;
- **Snapshot by custom dimension 1,...,5**. The creation or the restoration of the snapshot is partialized by custom dimension, or rather it is possible to run the snapshot of only the elements of the custom dimensions selected upon creation or restoration.

 If the scenario is managed *By process*, then the snapshot scenario CANNOT be linked to the process containing the scenarios the user wants to copy with the snapshot data processing.

 If you are working on a financial planning process it is necessary to create a snapshot Scenario also for the *Event Scenario* (for the snapshot linked to the *Event Scenario* it is not necessary to open the periods as it is a system scenario and it does not need to be loaded by the user).

Creation

The creation of a snapshot scenario, namely the copy of data from the official scenario to a snapshot scenario, can be accessed from the **contributor web interface through the process cockpit**.

If the snapshot is defined by Entity, it is possible to run the data processing selecting either one or more entities or a node (in this case, the data processing is run for all the entities of the node).

If the snapshot is not defined by entity, the data processing can be run only from the root.

If order to run the data processing, it is necessary to click on the **Snapshot** button and then select the **Create** option. Obviously, the management of the snapshots needs to be enabled for the process whose cockpit is being displayed.

If you run the snapshot from the detail overview tab, the system runs the snapshot on all the datasets linked to the process.

If you run the snapshot from one of the process steps, the system runs the snapshot only on the datasets linked to the selected step.

The user has to specify:

- the version number (**Version number**) of the target snapshot scenario
- a **description** of the copy operation they want to run
- a **comment**

Moreover, according to the process settings, the user has to specify

- one or more original source scenarios if the time dimension of the snapshot is "By scenario" or "By Scenario/period"
- one or more original source periods if the time dimension of the snapshot is "By Scenario/period"
- one or more source entities if the snapshot management is "By entity"
- one or more source categories if the snapshot management is "By category"
- one or more elements of the source Custom dimension X if the snapshot management is "By custom dimension X"

After setting the filters, it is necessary to click on "**Run**" to run the creation of the snapshot.


The target *snapshot scenario* must have the same **Version** number as the original scenarios, including the *Event Scenario*


The data processing runs the copy of the FX rates, of the Tax rates and of the Interest rates only if no "Dimensions on which to filter" have been defined and the snapshot is run from the root. In case of peripheral use, such data have to be copied beforehand by the admin.

The example below shows how the snapshot creation window works.

Let's assume that we have to copy the data on the snapshot scenarios related to the BUDGET process.

The snapshot scenarios related to the original scenarios of the BUDGET process are defined as shown below:

Original scenario	Version number	Snapshot scenario
2013ACT	Version 001	2013ACTV1
2014BDG	Version 001	2014BDGV1
	Version 002	2014BDGV2
2015BDG	Version 001	2015BDGV1

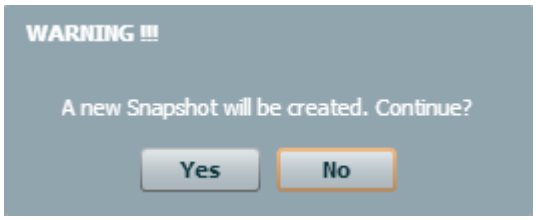
For example, if we select the 2014BDG original scenario, when creating the snapshot, the system will display the versions 001 and 002 in the "Version number" field.

Whereas, if no original scenario is selected, or the "time dimension" defined to manage the snapshot of the BUDGET process is “By process”, when creating the snapshot, the system will display the Version 001 in the "Version number" field as it is the only one which is defined on all three original scenarios.



- The copy does not perform any totalization and keeps the origins of data.

- The creation is irreversible therefore when the "**Run**" button is clicked the system sends a confirmation message to the user before running the copy.



Preliminary check

Before creating the snapshot scenario, the system checks:

- that each selected original scenario has not been used by another user for the creation of a snapshot , otherwise the system reports a warning
- that all the periods of the source scenario are not stored offline
- that the opened periods of each snapshot scenario are the same as those of the corresponding source scenario and that such periods are not blocked
- that the snapshot scenario of the event scenario related to the process exists, if the source scenario is related to a financial planning process

Restore

The restoration of a snapshot scenario, that is the copy of data from a snapshot scenario to the "official" scenario, can be accessed from **the end-user web interface through the process cockpit**.

If the snapshot is defined by entity, it is possible to run the data processing selecting either one or more entities or a node (in this case, the data processing will be run for all the entities of the node).

If the snapshot is not defined by entity, it is possible to run the data processing only from the root.

To run the data processing it is necessary to click on the **Snapshot** button and then select the **Restore** option. Obviously, the management of the snapshots must be enabled for the process whose cockpit is being displayed.

The user needs to specify the version number (**Version number**) of the source snapshot scenario

Moreover, according to the process settings, the user has to specify

- one or more original scenarios whose data have to be reloaded if the time dimension of the snapshot is "By scenario" or "By Scenario/period"
- one or more periods whose data have to be reloaded if the time dimension of the snapshot is "By Scenario / period"
- one or more entities whose data have to be reloaded if the snapshot management is "By entity"
- one or more categories whose data have to be reloaded if the snapshot management is "By category"
- one or more elements of the Custom dimension X whose data have to be reloaded if the snapshot management is "By custom dimension X"

After setting the filters, click on "**Run**" to restore the snapshot.

The data processing restores the FX rates, the Tax rates and the Interest rates only if no "Dimensions on which to filter" have been defined and the snapshot is run from the root. In case of peripheral use, such data have to be restored beforehand by the admin.

The example below shows how the snapshot restore window works.

Let's assume that we have to reload data on the official scenarios related to the BUDGET process.

The official original scenarios related to the BUDGET process are defined as follows:

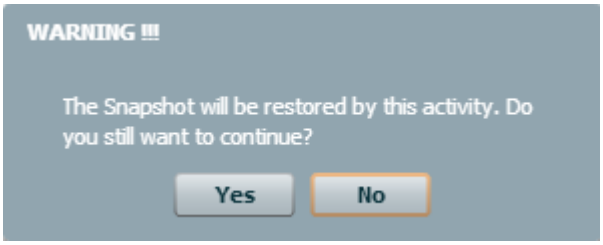
Original scenario	Version number	Snapshot scenario
2013ACT	Version 001	2013ACTV1
2014BDG	Version 001	2014BDGV1
	Version 002	2014BDGV2
2015BDG	Version 001	2015BDGV1

For example, if we select the 2014BDG original scenario, upon restoration the system will display the versions 001 and 002 in the "Version number" field .

Whereas, if either no original scenario is selected or the defined "time dimension" for the management of the snapshot of the BUDGET process is “By process”, upon restoration the system will display the Version 001 in the "Version number" field as it is the only one which is defined on all three original scenarios.

If no entity filter is applied, the system checks all the entities defined for the selected process and whether even just one of them is not defined on the source snapshot scenario, the system reports it and prevents the user from carrying out the "restore" operation. In this case, in order to complete the restoration, it is necessary to define the "Entity" filter by selecting all the entities provided by the system except the undefined ones.

The restoration is irreversible therefore when the "**Run**" button is clicked the system sends a confirmation message to the user before running the copy.



Preliminary check

Before restoring the "official" scenario, the system verifies that:

- the source scenarios have been copied on a snapshot scenario
- all the periods of the scenarios to "reload" are not stored offline
- the opened periods of each snapshot scenario are the same as those of the corresponding scenario to "reload" and that such periods are not blocked
- the snapshot scenario of the event scenario related to the process exists, if the scenario to "reload" is related to a financial planning process

Unified SQL syntax

In some CCH Tagetik definitions, where SQL language can be entered manually, you can use a unified custom syntax for certain functions (see table below), and this will then be translated into the language of the current vendor runtime DB. This functionality allows you to create “independent vendor DB” setups, which can be exported and imported on other application environments installed on other vendor DB without having to then convert the SQL syntax manually.

The following tools allow this syntax to be used:

- ETL (Extraction Transformation and Loading),
- DTP (Data Transformation Package),
- QDL (Quick Data Loader)
- Query-type application objects used in the QDL

 Square brackets indicate that the function’s topic is optional.

Syntax	SQLServer	Oracle	Postgres	Hana
F_SUBSTR(FIELD, 1, 3)	SUBSTRING(FIELD, 1, 3)	SUBSTR(FIELD, 1, 3)	SUBSTRING(FIELD, 1, 3)	SUBSTRING(FIELD, 1, 3)
F_LENGTH(FIELD)	LEN(FIELD)	LENGTH(FIELD)	LENGTH(FIELD)	LENGTH(FIELD)
F_INSTR(FIELD, WHAT)	CHARINDEX(WHAT, FIELD)	INSTR(FIELD, WHAT)	POSITION(WHAT IN FIELD)	INSTR(FIELD, WHAT)
F_CONCAT	+	||	||	||
F_LPAD(FIELD, N, EXPR)	RIGHT(REPLICATE (EXPR, N) + FIELD, N)	LPAD(FIELD, N, EXPR)	LPAD(FIELD, N, EXPR)	LPAD(FIELD, N, EXPR)
F_TO_CHAR(FIELD, [format])	CONVERT(VARCHAR, FIELD, [format_ code])	TO_CHAR(FIELD, [format])	TO_CHAR(FIELD, format)	TO_CHAR(FIELD, [format])
F_TO_DATE(FIELD, [format])	CONVERT(DATETIME, FIELD,[format_code])	TO_DATE(FIELD, [format])	TO_DATE(FIELD, format)	TO_DATE(FIELD, [format])
F_TO_NUMBER(FIELD, [CifreTotali [, CifreDecimali]])	CONVERT(NUMERIC [(CifreTotali [,CifreDecimali])], FIELD)	TO_NUMBER(FIELD)	TO_NUMBER(FIELD, format)	TO_NUMBER(FIELD)
F_TO_TIMESTAMP (FIELD, [format])	CONVER (DATETIME, FIELD, [format])	TO_TIMESTAMP (FIELD, [format])	TO_TIMESTAMP (FIELD, [format])	TO_TIMESTAMP (FIELD, [format])
F_DATE_TO_ TIMESTAMP(FIELD)	CONVERT(DATETIME, FIELD)	TO_TIMESTAMP (FIELD)	CAST (FIELD AS TIMESTAMP)	TO_TIMESTAMP (FIELD)
F_NVL(FIELD, VALUE)	ISNULL(FIELD, VALUE)	NVL(FIELD, VALUE)	COALESCE(FIELD, VALUE)	IFNULL(FIELD, VALUE)
F_NEWDATE	GETDATE()	SYSDATE	current_timestamp	current_timestamp
F_EXCELDATE(FIELD, [format])	CONVERT(VARCHAR (10), DATEADD(d, CAST(FIELD AS FLOAT),'1899-12-30'), [format])	TO_CHAR(TO_DATE ('1899-12-30', 'YYYY-MM-DD') + FIELD, [format])	TO_CHAR(TO_DATE ('1899-12-30', 'YYYY-MM-DD') + FIELD::int, format)	TO_CHAR(ADD_DAYS (TO_DATE ('1899-12-30', 'YYYY-MM-DD'), FIELD), 'YYYY-MM-DD')
F_DATEADD(day, N, FIELD)	DATEADD(day, N, FIELD)	FIELD + INTERVAL 'N' day	FIELD + INTERVAL 'N day'	ADD_DAYS(FIELD, N)
F_DATEADD(month, N, FIELD)	DATEADD(month, N, FIELD)	FIELD + INTERVAL 'N' month	FIELD + INTERVAL 'N month'	ADD_MONTHS(FIELD, N)
F_DATEADD(year, N, FIELD)	DATEADD(year, N, FIELD)	FIELD + INTERVAL 'N' year	FIELD + INTERVAL 'N year'	ADD_YEARS(FIELD, N)
F_YEAR(FIELD)	YEAR(FIELD)	EXTRACT(year FROM FIELD)	EXTRACT(year FROM FIELD)	YEAR(FIELD)
F_MONTH(FIELD)	MONTH(FIELD)	EXTRACT(month FROM FIELD)	EXTRACT(month FROM FIELD)	MONTH(FIELD)

Syntax	SQLServer	Oracle	Postgres	Hana
F_DAY(FIELD)	DAY(FIELD)	EXTRACT(day FROM FIELD)	EXTRACT(day FROM FIELD)	DAYOFMONTH(FIELD)
F_EOMONTH (theDateTime)	EOMONTH (theDateTime)	TRUNC(LAST_DAY (theDateTime))	(date_trunc ('MONTH', theDateTime) + INTERVAL '1 MONTH - 1 day')::date)	LAST_DAY (theDateTime)
F_EOMONTH (theDatetime, Offset)	EOMONTH (theDatetime, Offset)	TRUNC(LAST_DAY (ADD_MONTHS (theDatetime, theOffset)))	((date_trunc ('MONTH', theDatetime + INTERVAL 'theOffset MONTH') + INTERVAL '1 MONTH - 1 day')::date)	LAST_DAY(ADD_ MONTHS (theDateTime, theOffset))
F_DATEDIFF(DAY, firstdate,seconddate)	DATEDIFF(DAY, firstdate, seconddate)	(TO_DATE(TO_CHAR (seconddate,'DD-MON-YYYY')) - TO_ DATE(TO_CHAR (firstdate,'DD-MON-YYYY')))	(seconddate::date - firstdate::date)	DAYS_BETWEEN(TO_ DATE(firstdate), TO_ DATE(seconddate))
F_DATEDIFF(MONTH, firstdate,seconddate)	DATEDIFF(MONTH, firstdate, seconddate)	((EXTRACT(YEAR from seconddate) - EXTRACT(YEAR from firstdate)) * 12 + (EXTRACT(MONTH from seconddate) - EXTRACT(MONTH from firstdate)))	((DATE_PART('year', seconddate) - DATE_ PART('year', firstdate)) * 12 + (DATE_PART('month', seconddate) - DATE_ PART('month', firstdate)))	((YEAR(seconddate) - YEAR(firstdate)) * 12 + (MONTH (seconddate) - MONTH(firstdate)))
F_DATEDIFF(YEAR, firstdate,seconddate)	DATEDIFF(YEAR, firstdate, seconddate)	(EXTRACT(YEAR from seconddate) - EXTRACT(YEAR from firstdate))	(DATE_PART('year', seconddate) - DATE_ PART('year', firstdate))	(YEAR(seconddate) - YEAR(firstdate))
F_STUFF(inputString, startIndex, length, replaceWithString)	STUFF (character_ expression , start , length , replaceWith_ expression)	substr(pExpr, 1, pStart - 1)||pReplace||substr (pExpr, pStart + pLength)	overlay(<main_ string> placing COALESCE (<replacing_ string>,'') from <starting_position> [for <number_of_ characters>])	SUBSTRING(pExpr, 1, pStart - 1) || COALESCE (pReplace,'') || SUBSTRING(pExpr, pStart + pLength, LENGTH(pExpr))
F_dbo.	dbo.			
F_STRING_AGG (expression, separator)	STRING_AGG (expression, separator)	LISTAGG(expression, separator)	STRING_AGG (expression, separator)	STRING_AGG (expression, separator)
F_CEILING(numeric)	CEILING(numeric)	CEIL(numeric)	CEILING(numeric)	CEIL(numeric)
F_CHAR(numeric)	CHAR(numeric)	CHR(numeric)	CHR(numeric)	CHAR(numeric)
F_ISNUMERIC (expression)	ISNUMERIC (expression)	CASE WHEN regexp_ like(%1$s, '^(-)?\d+ (\.\d+)?(\,\d+)?$') THEN 1 ELSE 0 END	CASE WHEN %1$s ~ '^ (-)?\d+(\.\d+)? (\,\d+)?$' is true THEN 1 ELSE 0 END	locate_regexpr (START '^(-)?\d+ (\.\d+)?(\,\d+)?$' IN %1$s)
F_EQUAL(expr1, expr2)	CAST(expr1 AS VARBINARY(MAX)) = CAST(expr2 AS VARBINARY(MAX))	expr1 = expr2	expr1 = expr2	expr1 = expr2
F_DOUBLEQUOTE ([Campo1])	"Campo1"	"Campo1"	"Campo1"	"Campo1"
F_MD5(String)	HASHBYTES('MD5', String)	STANDARD_HASH (String, 'MD5')	('\x' || MD5 (String))::bytea	HASH_MD5(TO_ BINARY(String))
F_MOD(num1,num2)	num1 % num2	MOD(num1,num2)	MOD(num1,num2)	MOD(num1,num2)
"F_EXT_ DOUBLEQUOTE" on page 165	"FIELD"	"FIELD"	"FIELD"	"FIELD"

Syntax	SQLServer	Oracle	Postgres	Hana
"F_PREVIEW" on page 165	JSON_VALUE(FIELD, '$.preview')	JSON_VALUE(FIELD, '$.preview')	JSON_EXTRACT_PATH_TEXT (FIELD::json, 'preview')	JSON_VALUE(FIELD, '$.preview')

 To use unified sql syntax, such as an EXECUTE like:
EXECUTE 'select cod_periodo,**F_NVL(note, "A")** from dati_saldi_ic where cod_azienda=**"A3"** '
it is necessary to replace the superscripts with the F_DOUBLEQUOTE as follows:
EXECUTE 'select cod_periodo,**F_NVL(note, F_DOUBLEQUOTE(A))** from dati_saldi_ic where cod_azienda=**"A3"** '

 Note: the F_DOUBLEQUOTE must be used when, in one of the functions listed above, it is necessary to enclose a parameter between two single quotes (F_NVL(notes, ''A'')).

 Attention: When writing queries, the following system tables cannot be used for the Postgres DB:

- pg_role
- pg_user
- pg_group
- pg_database
- pg_stat_activity
- pg_stat_database
- pg_stat_database_conflicts
- information_schema

Examples:

- F_TO_CHAR

SELECT * FROM DATI_SALDI_LORDI WHERE F_TO_CHAR(DATEUPD,'DD/MM/YYYY') = '01/09/2017'

Translations:

SQLServer: CONVERT(VARCHAR,DATEUPD,103)

ORACLE: TO_CHAR(DATEUPD,'DD/MM/YYYY')

POSTGRES: TO_CHAR(DATEUPD,'DD/MM/YYYY')

HANA: TO_CHAR(DATEUPD,'DD/MM/YYYY')

SELECT * FROM DATI_SALDI_LORDI WHERE F_TO_CHAR(DATEUPD) = '01/09/2017'

Translations:

SQLServer: CONVERT(VARCHAR,DATEUPD)

ORACLE: TO_CHAR(DATEUPD)

POSTGRES: error (as reported in the table, the two topics are mandatory)

HANA: TO_CHAR(DATEUPD)

SELECT F_TO_CHAR(IMPORTO, '999999.999') FROM DATI_SALDI_LORDI

Translations:

SQLServer: CONVERT(VARCHAR, IMPORTO)

ORACLE: TO_CHAR(IMPORTO, '999999.999')
POSTGRES: TO_CHAR(IMPORTO, '999999.999')
HANA: TO_CHAR(IMPORTO, '999999.999')

- **F_TO_DATE**

SELECT F_TO_DATE('01/09/2017','DD/MM/YYYY') FROM DATI_SALDI_LORDI

 Note: the separator character of the first and second parameter must coincide.

Translations:

SQLServer: CONVERT(DATETIME,'01/09/2017,103)

ORACLE: TO_DATE('01/09/2017,'DD/MM/YYYY')

POSTGRES: TO_DATE('01/09/2017,'DD/MM/YYYY')

HANA: TO_DATE('01/09/2017,'DD/MM/YYYY')

SELECT F_TO_DATE('01/09/2017') FROM DATI_SALDI_LORDI

Translations:

SQLServer: CONVERT(DATETIME,'01/09/2017')

ORACLE: TO_DATE('01/09/2017')

POSTGRES: error (as reported in the table, the two topics are mandatory)

HANA: TO_DATE('01/09/2017')

- **F_TO_NUMBER**

SELECT *, F_TO_NUMBER('11998.261', 7, 2) FROM DATI_SALDI_LORDI

Translations:

SQLServer: CONVERT(NUMERIC(7, 2),'11998.261')

ORACLE: TO_NUMBER('11998.261') non tiene conto dei parametri cifre totali e cifre decimali

POSTGRES: TO_NUMBER('11998.261', '99999.99')

HANA: TO_NUMBER('11998.261') does not take account of the total figure or decimal digit parameters

 Caution: an error will be returned if the total number of figures specified is less than the number of decimal figures (e.g. F_TO_NUMBER('11998.261', 2, 7)), with the exception of Oracle and Hana, which do not take account of these two parameters.

SELECT *, F_TO_NUMBER('11998.261', 5) FROM DATI_SALDI_LORDI

Translations:

SQLServer: CONVERT(NUMERIC(5),'11998.261')

ORACLE: TO_NUMBER('11998.261') non tiene conto dei parametri cifre totali e cifre decimali

POSTGRES: TO_NUMBER('11998.261', '99999.')

HANA: TO_NUMBER('11998.261') does not take account of the total figure or decimal digit parameters

 Caution: an error will be returned if the number of figures comprising entire part of the first parameter is greater than the specified number of total figures (e.g. F_TO_NUMBER('11998.261', 3)), with the exception of Oracle and Hana.

SELECT *, F_TO_NUMBER('11998.261') FROM DATI_SALDI_LORDI
Translations:
SQLServer: CONVERT(NUMERIC,'11998.261')
ORACLE: TO_NUMBER('11998.261')
POSTGRES: error, at least the number of total figures must be specified.
HANA: TO_NUMBER('11998.261')

- **F_EXT_DOUBLEQUOTE**

SELECT F_EXT_DOUBLEQUOTE(F_TO_DATE('01/01/2024', 'DD/MM/YYYY')) FROM DATI_SALDI_LORDI

Translations:

SQLServer: CONVERT(DATETIME,''01/01/2024'',103)

ORACLE:TO_DATE(''01/01/2024'',''DD/MM/YYYY'')

POSTGRES: TO_DATE(''01/01/2024'',''DD/MM/YYYY'')

HANA: TO_DATE(''01/01/2024'',''DD/MM/YYYY'')

 Attention: F_EXT_DOUBLEQUOTE applies to strings or other single functions. If you have several disjointed functions, it must be repeated externally for each block.

F_EXT_DOUBLEQUOTE(FIELD)

 Attention: the F_EXT_DOUBLEQUOTE only affects single quotes. If an argument without quotes or with double quotes is passed to it, it does not perform any transformation.

- F_EXT_DOUBLEQUOTE(FIELD) -> restituisce FIELD
- F_EXT_DOUBLEQUOTE('FIELD') -> restituisce ''FIELD''
- F_EXT_DOUBLEQUOTE(''FIELD'') -> restituisce ''FIELD''

- **F_PREVIEW**

 Note: the FIELD field must contain a json string structured as follows:

```
{
    "key": {
        "storage": "storage type",
        "id": "rich text oid",
        "reference": "[db]\\\\[aw]\\\\[dataset]",
        "version": 1
    },
    "preview": "preview text"
}
```

Multi File Syntax

CCH Tagetik allows you to use, in certain scenarios, an advanced syntax to specify the files to be processed.

Note:
When using this syntax for file retrieval, the following parameters must already be defined:

- File type: Specifies the type of file that will be considered in the process (e.g. CSV, XLS, XLSX, etc.).
- Base folder: Defines the starting point for searching and retrieving files.

It is usually possible to specify these parameters; some tools work with these parameters pre-set.

From here on, files corresponding to the requested type will be called **'requested**'.

The syntax for file recovery offers two main modes:

- Specific file: allows you to indicate a specific file with its full name including the extension.
- File with partial name: allows the use of a partial name pattern to select multiple files.

In both cases, one or more files can be specified. File names must be separated by a semicolon (`;`) (e.g.: `file1.csv;file2.csv`).

Dynamic syntax structure

Dynamic syntax must be enclosed in square brackets (`[ ]`) and consists of three elements:

1. Recursion on subfolders (optional):**
   By using '**/', the file search will be extended to all sub-folders. If not specified, only files in the defined folder will be processed.
2. File template
   Allows you to specify a template for selecting the required files. For example, 'data*.csv' identifies all files starting with 'date' and ending with the extension '.csv'.
3. ZIP file recursion (optional):
   Using '/**', the search for **'required**' files is performed on the entire contents of the ZIP archives, including any nested ZIPs. If not specified, only files in the root of ZIP archives will be processed.

File Recovery Syntax

- Specific Files
  Specifies one or more files using the exact name and extension, separated by a semicolon (';') (e.g.: 'data1.csv;sales.csv').
- ZIP files
  Specifies one or more ZIP files with an extension, separated by a semicolon (';') (e.g.: 'collection.zip;archive.zip'.
- Filters
  - '[filename*.csv]': Retrieves all files starting with 'filename' and ending with '.csv' in the defined folder.
  - '[archive*.zip]': Retrieves all **requested** files by searching the root of ZIP files starting with 'archive' and ending with '.zip' in the defined folder.
- Recursive Filters
  - '[**/filename*.csv]': Retrieves all files starting with 'filename' and ending with '.csv' in the folder and all subfolders.
  - '[archive*.zip/**]': Searches all ZIP files starting with 'archive' in the defined folder, retrieving all **required** files that are contained in these ZIP files.
  - '[**/archive*.zip/**]': Searches all ZIP files starting with 'archive' in the defined folder and subfolders, retrieving all **requested** files that are contained in these ZIP files.
- Combined Syntax

  File names and filter syntax can be merged using semicolon (';'). For example: 'collection.csv; [archive*.zip/**]'.
- Special Syntaxes
  The dynamic syntax provides three special strings that do not require square brackets and should be used with care:
  - '*': All **requested** files in the defined folder and the root of the ZIP files in the defined folder.
  - **': All **requested** files in the defined folder, subfolders and the root of the ZIP files in the defined folder and all subfolders.

- '**/**': All **requested** files in the defined folder, subfolders and all contents of ZIP files (including nested ZIPs).

Best Practices:

When using recursive expressions, it is advisable to minimise the number of folders to be processed. A good practice is to create a dedicated folder in which to copy the structure of files and folders of interest. This is beneficial both for improving performance, especially with the FTP protocol, and for preventing the processing of unwanted files.

Note:

- If a file of a certain type is retrieved through the syntax (e.g. XLS) but the content of the file does not comply with the extension (e.g. is a txt) an incorrect MIME type error will be reported).
- If, depending on the type of file requested, no files are retrieved, an error will be generated indicating that no data is available (e.g. xLS-type files are requested, but are not present).

Examples:

Syntax	Search path	ZIP Content	Description
`data1.csv;sales.csv`	specified folder	no	exact file name match
`collection.zip;archive.zip`	specified folder	zIP file root only	**Requested** files contained in ZIP files with exact name
`[filename*.csv]`	specified folder	no	Files whose names begin with 'filename' and end with '.csv'.
`[archive*.zip]`	specified folder	zIP file root only	**Requested** files contained in ZIP files with partial name/ wildcard.
`[**/filename*.csv]`	specified folder and all subfolders	no	Files whose names begin with 'filename' and end with '.csv'.
`[archive*.zip/**]`	specified folder	subfolders and nested ZIP files	**Requested** files contained in ZIP files with partial name/wildcard.
`[**/archive*.zip/**]`	specified folder and all subfolders	subfolders and nested ZIP files	**Requested** files contained in ZIP files with partial name/ wildcard.
`*`	specified folder	zIP file root only	All files **requested**
`**`	specified folder and all subfolders	zIP file root only	All files **requested**
`**/**`	specified folder and all subfolders	subfolders and nested ZIP files	All files **requested**

Monthly splitting

When it is necessary to define periodic data of one or more accounts from the cumulated data of a specific period, it is possible to use CCH Tagetik's **Monthly splitting** data processing or the **Custom Splitting on Periods** data processing .

The first one (Monthly splitting) is defined for each single account and allows you to split per month from the last period of a scenario, on all previous periods of the Scenario, while the second one (Custom Splitting on Periods) allows you to define in much more detail the data to split, the Scenarios /Periods on which to split the data and the driver to use

In order to use the **Monthly splitting**, it is necessary to define some rules which are not linked to a specific process nor to an entity, but to the account to which the amount to be split belongs.

The **monthly splitting** rules allow the user to generate entries on the target periods.

Monthly splitting rules

The monthly splitting rules can be defined for the "P&L", "Other variations" accounts or for the "Variations" accounts of "BS" or "Other stocks" nature.

For the other account types / natures it is not possible to define the monthly splitting rules since they are always treated as cumulated accounts.

In order to define a monthly splitting rule, it is necessary to:

- access the accounts elements table (accessible from the administrator user web interface by following the path *Setup & Admin > Data Model > Dimensions > Account > Elements*) *Navigation panel> Dimensions > Account > Elements*) and select the account for which the the monthly splitting rule has to be defined;
- place the cursor on the "**Monthly splitting rules**" tab

The **Monthly splitting type** property specifies how the cumulated amount has to be split. Possible splitting types:

- *No*. The system does not run any monthly splitting
- *Flat*. The account's value is shared equally on each period (e.g. the amount of December is equally split on 12 months)
- *Coefficient 445*. The account's value is distributed according to a coefficient which follows the 4,4,5 progression of the weeks during the months. Considering that in one year there are 52 weeks and that in each quarter there are 13 weeks, the "weights" of the weeks in the quarter will be 4/52 8/52 13/52 17/52 .... 47/52 52/52
- *Coefficient 454*. The value of the account is distributed according to a coefficient which follows the 4, 5, 4 progression of the weeks during the months. Considering that in one year there are 52 weeks and that in each quarter there are 13 weeks, the "weights" of the weeks in the quarter will be 4/52 9/52 13/52 17/52 .... 48/52 52/52
- *Coefficient 544*. The value of the account is distributed according to a coefficient which follows the 5,4,4 progression of the weeks during the months. Considering that in one year there are 52 weeks and that in each quarter there are 13 weeks, the "weights" of the weeks in the quarter will be 5/52 9/52 13/52 18/52 .... 48/52 52/52

   The rules using the coefficients 445, 454 and 544 can be applied only if the monthly splitting is done on 12 periods.

- *Adv. management* The system splits per month the value of an account using a driver. The driver to use is defined according to the following information

Reference:

the **Reference account**. Account used to calculate the monthly splitting driver. If none is specified, the system uses the account for which the monthly splitting rule is being defined.

As an alternative to a single account, it is possible to indicate an entire hierarchy of accounts (**Reference accounts hierarchy**)

the **reference scenario**. The definition of a reference scenario is required when the scenario to use for the calculation is different from the data processing scenario. The reference scenario for the driver is parametric, in other words determined by the system during the rule execution. The available parametric values are:

- PREV. The reference scenario for the driver is the one before the processed scenario
- NEXT. The reference scenario for the driver is the one after the processed scenario
- REF1...REF5. The reference scenario for the driver is the 1...5 reference scenario related to the processed scenario

a possible filter on the managed **custom dimensions 1...5**, on the **category**, on the**Ctp entity** and on the **Ctp custom dimension 2** to apply when defining the monthly splitting driver

Process setup for the "Monthly splitting" management

In order to enable the "Monthly splitting" data processing for a given process the user needs to:

- open the process elements table accessible from the admin user interface from *Setup & Admin > Processes > select the process to parametrize* and then *Actions (⋮) > Define process.* ; *Navigation panel> Processes& Workflows > Process > select the process to parametrize* and then *Actions (⋮) > Define process.* ;
- in the **Data Processing** area, enable the **Others** > **Monthly splitting by Account** data processing
- save

“Monthly splitting” data processing

Once the parametrization of the monthly splitting rules is complete, it is possible to run the monthly splitting data processing. This data processing can be run only if its execution has been enabled for the selected process (for further details see chapter **Other** data processing of the manual on the process parametrization).

The system calculates data to be split per month according to the information specified in the initial filter:

- the **last period** of the scenario to run. In order to run the monthly splitting it is necessary to "launch" the data processing on the last open period of a scenario: usually it is run on the period 12 but, if the periods represent the weeks, it is run on the period 52. If several scenarios/periods are specified in the initial filter, the system puts together the monthly splitting periods: for example, scenarios 2010C/12 and 2010B/12 will be run together if they have the same target periods
- the **entities**
- other possible **optional filters**
- the **accounts** to which a monthly splitting rule is linked

The data processing "operates" only on the gross amounts and on the IC amounts and takes into account all origins but those generated:

- by a previous monthly splitting (PROC_MONTHLY_SPLIT_NEW)
- by the preconsolidation reclassifications (PROC_RECLASSIFICATION)
- by the depreciation calculation (PROC_DEPRECIATION)
- by the basic calculations (PROC_CALCULATED_BASIC)
- by the deferred taxes calculation (PROC_DEFERRED_TAXES)
- by the balancing (PROC_BALANCING)
- by the equity reclassification (PROC_NETEQUITY_RECL)
- by the application of calculation logics (PROC_CALCULATED)
- by the deconsolidation (PROC_DECONSOLIDATION3)

The system applies the monthly splitting rule to each data to be split per month, as follows:

- if the "Monthly splitting type" defined for the account is "NO", then data is not split per month
- if the "Monthly splitting type" defined for the account is "simple" type, then data is split per month by applying the constant coefficient specified in the rule
- if the "Monthly splitting type" defined for the account is “advanced”, then the system calculates the monthly splitting drivers according to the information specified in the rule. The calculated drivers are used by the system to calculate the weights to apply to data to be split per month. Once the weights have been calculated, they can be applied to data to split per month.

Before entering the monthly splitting entries, the system removes the existing ones.

Run "Monthly splitting"

The monthly splitting can be run:

- from the contributor user's **process cockpit**. After selecting one or more entities or one or more nodes, it is necessary to select the **Entity data processing** button from the *Actions* menu and then select **Monthly splitting** in the data processing tree under the "Entity data processing" node. This data processing is visible only if the Monthly splitting data processing is enabled for the selected process.
- **from a data entry form**
- **from a workflow task**

- **from a job task**

From form

The "Monthly splitting" can be run also from a data entry form as a single data processing as a part of the Entity data processes enabled for a given process.

Single data processing

Open the form in Design mode, click on  **Data processing** on the Design ribbon bar (or right click on the Open Items panel).

The system displays a window in which the user will select the data processing to run: select **Monthly splitting** and click on **Next** to set up the parameters and define the data processing execution area on the Entity, Scenario, Period and Category dimension and on the Custom dimensions. The parameters in bold are mandatory.

The parameters definition modes are:

- **Automatic**. The system retrieves automatically the parameter's value from a form filter, a tab filter, a matrix filter, a row/column filter. If the filter is defined by a parameter (therefore its value is chosen by the user at run-time) the choice of the data processing value will be up to the user
- **Excel cell**. The system retrieves the parameter's value from an Excel cell. The user must specify the cell containing the value by using the syntax ${H30} (cell H30).
- **Insert**. The user must enter the parameter's value into the corresponding text box
- **Select**. The user must select the parameter's value from a list of possible values
- **Cell field**. The system retrieves the parameter's value from a cell field.

After setting up the parameters, click on "Next" to configure a series of options which allow the system to understand how the data processing has to be run within the form (e.g. if it has to be run upon saving or manually, for further details see **Data processing**).

Eventually it is necessary to assign a code and a description to the data processing that will be inserted in the form.

Entity data processing

The monthly splitting belongs to the Entity data processing and can be run also as a part of such data processing just by selecting it from the *Entity data processing* selection window.

In this case, the following parameters have to be set up:

- the original **scenario / period** on which the data processing has to be run;
- the **entities** to which data to run are allocated;
- **Disable impacted**. By default this option is disabled.

 In this case, it is not possible either to select the accounts to split per month or specify filters on the other dimensions..

From workflow

Tagetik allows the user to run the monthly splitting also from a workflow model task. Also in this case, it can be run in two ways, namely either as a part of the entity data processing or individually.

 In this case, the process, the period and the entities to run are pre configured in the process cockpit.

Single data processing

Open the Workflow definition page in edit mode, define a new Task by dragging the *Monthly splitting* icon, *Other data processing* tab into the workflow area.

Besides the regular options of any Workflow Task, in the Parameters area it is necessary to specify the data processing execution area on the Account, Category and Custom dimensions.

Entity data processing

Define a new Task by selecting it from the top bar, in the *Core data processing* tab, *Entity Data processing*.

Select **Monthly Splitting** in the parameters.

 In this case, it is not possible either to select the accounts to split per month or to specify filters on the other dimensions.

From job

Tagetik allows the user to run the monthly splitting also from a job task. Also in this case, it can be run in two ways, namely either as a part of the entity data processing or individually.

Single data processing

Open the Job definition page in edit mode, define a new Task by dragging the *Monthly splitting* icon, *Other data processing* tab;

Besides the regular options of any Job Task, in the Parameters area it is necessary to specify the Process and the data processing execution area on the Account, Entity, Scenario, Period, Category and Custom dimensions.

 By selecting the **All** option (Entity and Scenario/Period) the values of the corresponding parameter will be determined by the system when running the job and they will coincide with all the available values, at that time, for that specific parameter.

Entity data processing

Define a new Task by selecting it from the top bar, in the Core data processing tab, *Entity data processing*.

In the parameters area, it is necessary to specify the process, the Scenarios / Periods and the Entities to run and then select the **Monthly splitting** data processing.

 In this case, it is not possible either to select the accounts to be split per month or specify filter s on the other dimensions.

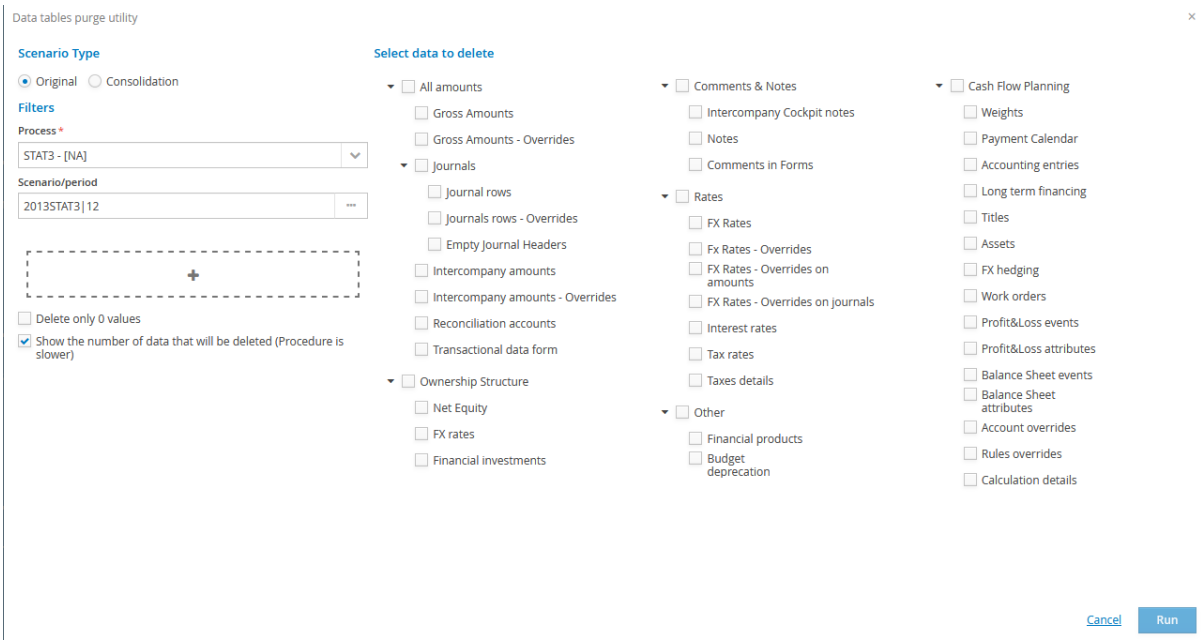
 By selecting the **All** option (Entity and Scenario/Period) the values of the corresponding parameter will be determined by the system when running the job and they will coincide with all the available values, at that time, for that specific parameter.

Data tables clean-up

Tagetik provides a convenient data processing to delete wrong data or data that are no longer useful

Setup & Admin > tile Data processing > Data Clean Up > Data tables

Navbar> Data processing > Data Clean Up > Data tables



It is necessary to filter the data to delete by **Process** and **Scenario/period** and then add, if necessary, additional specific filters : User (UserUpd), Category, Account, Entity, Entity node, Custom dimension 1-5, Date (DateUpd), Origin (this option is very useful when you want to massively delete all data with a specific origin for the specified process)

For some deletions it is not necessary to select the Scenario/Period:

Cash flow, Weights
Cash flow, Payment Calendar
Cash flow, Accounting entries
Cash flow, L/T Financing
Cash flow, Titles
Cash flow, Assets
Cash flow, FX hedging
Cash flow, Work Orders
Cash flow, Calculation details
Cash flow, Account overrides
Cash flow, Rules overrides
Ownership structure register, Net equity
Ownership structure register, FX rates
Ownership structure register, Financial investments
Other, Financial products
Other, Budget deprecation

Some specific filters are not available for certain types of deletion

Delete only zero values if enabled, it deletes only the rows with amount zero.

The deletion can be run on the following elements:

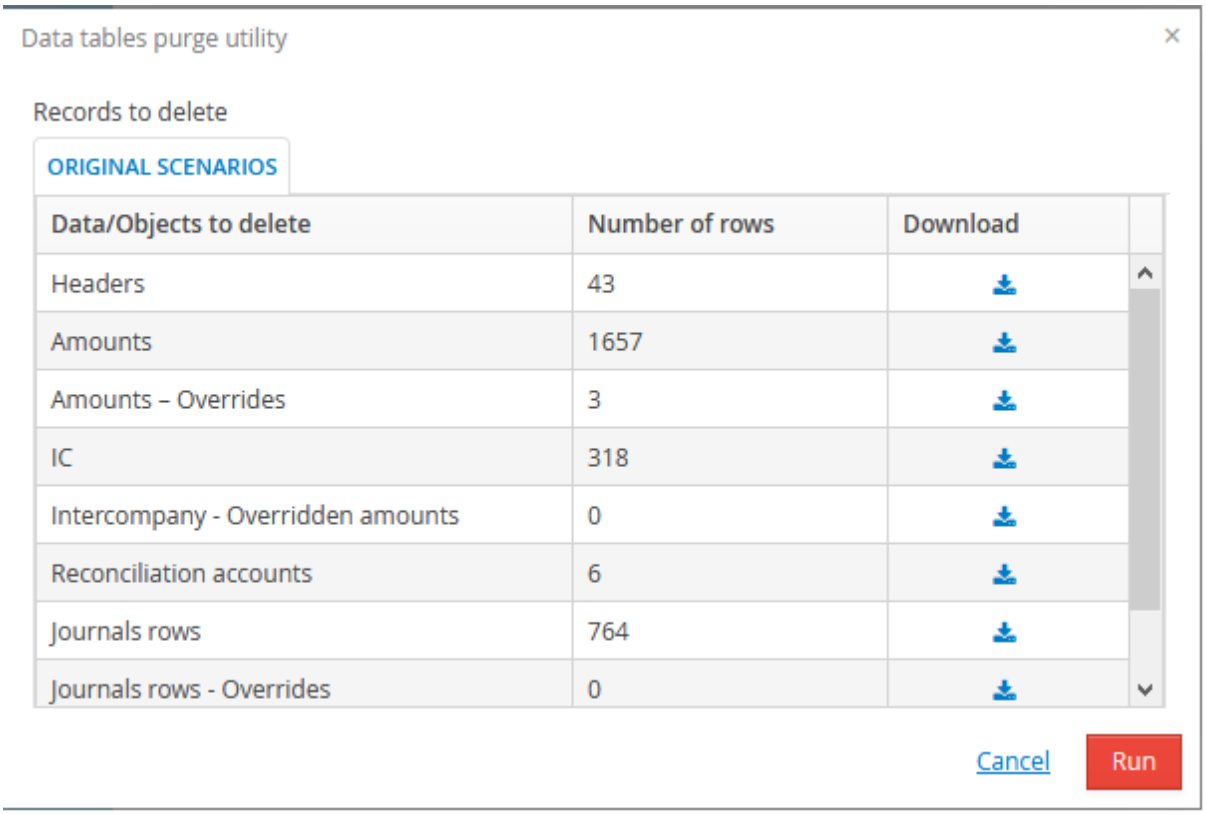
- **Amounts**

    - Gross amounts
    - Gross amounts - Overrides
    - Adjustments
    - Journals rows
    - Journals rows - overrides
    - Delete the journal headers if empty
    - IC amounts
    - IC amounts - overrides
    - Reconciliation accounts
    - Transactional form data
- **Ownership structure register**
    - Net Equity
    - FX rates
    - Financial investments
    - Other information
- **Comments and Notes**
    - Intercompany Cockpit notes
    - Notes Forms
    - Comments Forms
- **FX rates**
    - FX rate type
    - Overridden FX rates
    - Specific overridden FX rates on amounts
    - Specific overridden FX rates on journals
    - Interest Rates
    - Tax Rates
    - Taxes detail
- **Other**
    - Financial product: calculation details
    - Depreciation: calculation details
- **Cash Flow Planning**
    - Weights
    - Payments calendar
    - Accounting entries
    - L/T Financing
    - Titles
    - Assets
    - FX hedging
    - Work orders
    - P&L events
    - P&L attributes
    - Balance Sheet events
    - Balance Sheet attributes
    - Account overrides
    - Rules overrides
    - Calculation details

After specifying the process and the filters to identify the data to delete, it is possible to run the clean-up operation just by clicking on **Run**.

Download Purge Queries

When the **"Show the number of data that will be deleted (Procedure is slower)"** option is enabled, the window below pops up



For each "Data/ Objects to delete" it is possible to Download the related queries that can be run by clicking on Run.

After downloading the file, you can run each query (on any Query Analyzer) to display the rows that will be deleted.

For each download file, if all queries are run, the total number of rows will correspond to the value of the "Number of rows" column.

Then you can decide whether to run or to abort the operation. If you decide to proceed with the deletion, the following applies.

Once the operation is completed, the system displays a *Data processing is complete* window from which it is possible to extract the detail log or export such log in Excel format

It is possible to hide this window by flagging the "**Show only in case of error**" field